# ZlibValidation

Generated on Fri Mar 28 2025 12:09:33 for ZlibValidation by Doxygen 1.9.6

Fri Mar 28 2025 12:09:33

# Chapter 1

# ZlibValidation

## 1.1 Description

Command line tool to validate standard cell libraries in `.lib` format.

## 1.2 Usage

```
ZlibValidation
Usage: ./zlibvalidation [OPTIONS] [SUBCOMMAND]

Options:
  -h,--help                   Print this help message and exit
  -v,--version                Display program version information and exit

Subcommands:
  parse                       Parse the Liberty file and write JSON to a file
  mono                        Check the monotonicity of timing arc values
  compare                     Compare the comparison library against the reference library and report
      differences
  supercell                   Generate supercells for the given Liberty file
  zlibboost                   ZlibBoost - Multi-threaded Library Processing Tool
  clear                       Clear the log, JSON, map, markdown files in this directory
  verilog                     Generate Verilog file for given Liberty file
  spice                       Generate SPICE file for given Liberty file
  func                        Check functional equivalence of two Liberty files or Verilog files
```

## 1.3 Development Diary

### 1.3.1 2025-01-27

- C++CMake `libsi2dr_liberty.a`
- `CLI11`
- `spdlog`

### 1.3.2 2025-01-28

- nlohmann/json JSON
- version.h CMake
- --version --mode
- spdlog debug info
- LibFile PVT
  - PVT
- C++ #include

### 1.3.3 2025-02-01

- OOPCLibAttribute
  - char *std::string
- AttributesIteratorRAII

### 1.3.4 2025-02-10

- GroupsIterator
- LibGroup
- srcincludeCMakeLists.txt

### 1.3.5 2025-02-11

- hppCMakeLists.txtCMake
- LibFilejsonLibFile::writeJsonToFilePVTcell_namecell_footprintcell_area
- [x] voltage

### 1.3.6 2025-02-14

- json_utils.cpp JSON (lib) generateCellJson, generatePinJson, generate↩ PowerJson, generateLutJson hpp
- [x] (timing)
- parseStringToVector Look Up Table (LUT)
- LibFile lib_json_ (lib) JSON
- ValuesIterator
- LibAttribute::isComplex() bool
- LibGroup::getName() std::string
- si2drGroupsIdT sub_groups lib_group.getGroups()GroupsIterator

### 1.3.7 2025-02-15

- `generateTimingJson`

### 1.3.8 2025-02-17

- `begin()`

### 1.3.9 2025-02-18

- `LibFile::mono()` values
- `setupLogger()`

### 1.3.10 2025-02-19

- `mode == "mono"` C++17 `std::filesystem::exists()` JSON JSON JSON
- `LibFile::mono()` cell_risecell_fallrise_transitionfall_transitiontiming LUT value

### 1.3.11 2025-02-20

- `LibFile::mono()` `passed()` `failed()` cell `liberate_lv`
- `sl018_ff_3.96_-40.lib liberate_lv` 205 out of 559 cells failed
- when

### 1.3.12 2025-02-25

- `CLI11 parsemono`

### 1.3.13 2025-02-26

-
- Logger `-h-v`
- `LibFile::mono()` `input_slew` value `tcbn65lpbc.lib liberate_lv`

### 1.3.14 2025-02-27

- GDB
  - si2dr_liberty Liberty
  - si2dr_liberty
  - si2dr_liberty
  - strcmp

### 1.3.15 2025-02-28

- LibFilereadparsesi2drparsemonosi2dr
- [x] si2drPIGetGroupsbug: WARNING: si2drPIQuit: GetGroups called 1 more times than IterQuit
- CMakeLists.txtset(CMAKE_BUILD_TYPE Release)GDB

### 1.3.16 2025-03-01

- sub_groups_iterforsi2drPIQuit
- group_iterparseparsesi2drPIQuitgroups
- si2drPIQuitgroupsparse
- [ ] parselogPVT(parse 6s mono 1s)monolog
- LibFile

### 1.3.17 2025-03-07

- compare JSON
- [ ]

### 1.3.18 2025-03-10

- supercell LibFile::supercell
- zlibboost zlibboost
-
- clear JSON

### 1.3.19 2025-03-14

- tabulate/table CMake FetchContent
- compare.cpp compare.hpp LibraryComparator JSON

### 1.3.20 2025-03-15

- std::filesystem::path LibFile LibFile basename_filename_libname_↩ jsonname_ loggername_JSON
- (scope) LibFile::parse si2drPIQuit
- clear .map .md
- mono JSON JSON
- spdlog logger logger APP_NAME logger
- LibFile logger
- supercell mono
- LibraryComparator::generateReport() tabulate markdown

### 1.3.21 2025-03-16

- `compare .md .txt .md`
- `LibraryComparator::generateReport()` cell cell `LibraryComparator::compareCell()` cell
- `LibraryComparator::compareCell()` cell cell cell `comparePin` cell
- `LibraryComparator::comparePin()` cell pin timing arc pin timing arc `compareTimingArc` pin timing arc
- `LibraryComparator::compareTimingArc()` timing arc JSON timing arc "related_pin" timing arc "cell_rise", "cell_fall", "rise_transition", "fall_transition" timing arc timing arc `compareLut` timing arc
- `LibraryComparator::compareLut()` `compareValue` JSON value JSON "index_1" "index_2" LUT value (`reltol`) (`abstol`)
- `cell_namepin_nametiming_typerelated_pin arc_name`
- `LibraryComparator::compareValue()` `LibraryComparator::compareLut()`
- 
- `abstol` 0.002ns

### 1.3.22 2025-03-17

- `verilog spice` Verilog SPICE
- `LibraryComparator`
  - cell
  - cell Timing Delay

### 1.3.23 2025-03-18

- | Cell Name | Data Type | Failed Count | Avg Diff | Avg Diff% | Max Diff | Max Diff% | Outliers |
- | Pin Name | Reference | Comparison | Diff | Diff % | Type | Arc Name | Row # | Index_1 | Column # | Index_2 | Note |
- stdout
- `si2drSimpleAttrGetBooleanValue`
- `LibFile::mono()` `min_pulse_width`
  - "input_pins" cell
  - "timing_arcs"
  - "timing_type" "min_pulse_width" timing arc "rise_constraint" "fall_constraint" `checkTimingArc↩Monotonicity`
- `checkTimingArcMonotonicity` when
- `checkTimingArcMonotonicity`
- `checkTimingArcMonotonicity related_pin` pin

### 1.3.24 2025-03-19

- supercell1
- voltage std::round(voltage_ * 100) / 100.0

### 1.3.25 2025-03-21

- func Verilog

### 1.3.26 2025-03-24

- C++ Verilog Verilog `slang` (AST) CMakeLists slang API

### 1.3.27 2025-03-25

- `verilog_utils.cpp` `verilog_utils.hpp` `VerilogVisitor` Verilog AST
- slang
  - `AND2D0` `CMPE42D1` `DFQD1`
  - (UDP) `tsmc_dff` "Invalid instance declaration"
  - `inputoutput`
  -
  -

### 1.3.28 2025-03-26

- UDP `tsmc_dff` Entry
- `slang::syntax::SyntaxRewriter` `CellExtractor` `slang::syntax::SyntaxPrinter::printFile()` Verilog
- `CellPrinter` `slang::syntax::SyntaxVisitor` `module.toString()` Verilog
- `LibFile::supercell()` `cell_names` supercell
- supercell `input_pins`
- Doxygen Doxygen HTML LaTeX
- [ ] LaTeX

### 1.3.29 2025-03-27

- Git ****
- LibFile::verilog
  - this->supercell()
  - .map std::pair<std::string, std::string> ->/
  - lib_json_/
  - Verilog instance 1 chain length
  - ANSI fullModuleText
  - slang::syntax::SyntaxTree::fromText fullModuleText
  - ModuleRewriter transform handle()
  - slang::syntax::SyntaxPrinter::printFile(*tree)
- LibFile::verilog ANSI

### 1.3.30 2025-03-28

-
- ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module) insertAtBack(module.members, newDataNode) wire OP_i;

# Chapter 2

# Hierarchical Index

## 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

# Chapter 3

# Class Index

## 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

# Chapter 4

# File Index

## 4.1 File List

Here is a list of all files with brief descriptions:

# Chapter 5

# Class Documentation

## 5.1 AttributesIterator Class Reference

`#include <iterators.hpp>`

Collaboration diagram for AttributesIterator:

## 5.2 CellExtractor Class Reference

`#include <verilog_utils.hpp>`

Inheritance diagram for CellExtractor:

Collaboration diagram for CellExtractor:

### Public Member Functions

- CellExtractor (const std::string &targetCell)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)
- bool foundTargetCell () const

### Private Attributes

- const std::string & targetCell_
- bool foundTarget_

### 5.2.1 Detailed Description

Definition at line 32 of file verilog_utils.hpp.

### 5.2.2 Constructor & Destructor Documentation

#### 5.2.2.1 CellExtractor()

```
CellExtractor::CellExtractor (
            const std::string & targetCell )  [inline], [explicit]
```

Definition at line 34 of file verilog_utils.hpp.

### 5.2.3 Member Function Documentation

#### 5.2.3.1 foundTargetCell()

```
bool CellExtractor::foundTargetCell ( ) const
```

Definition at line 364 of file verilog_utils.cpp.

Here is the caller graph for this function:

#### 5.2.3.2 handle()

```
void CellExtractor::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 348 of file verilog_utils.cpp.

### 5.2.4 Member Data Documentation

#### 5.2.4.1 foundTarget_

```
bool CellExtractor::foundTarget_  [private]
```

Definition at line 40 of file verilog_utils.hpp.

#### 5.2.4.2 targetCell_

```
const std::string& CellExtractor::targetCell_  [private]
```

Definition at line 39 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

## 5.3 CellPrinter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for CellPrinter:

Collaboration diagram for CellPrinter:

### Public Member Functions

- CellPrinter (const std::string &targetCell, std::ostream &out)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)

### Private Attributes

- const std::string & targetCell_
- std::ostream & out_
- bool foundTarget_

### 5.3.1 Detailed Description

Definition at line 44 of file verilog_utils.hpp.

### 5.3.2 Constructor & Destructor Documentation

#### 5.3.2.1 CellPrinter()

```
CellPrinter::CellPrinter (
            const std::string & targetCell,
            std::ostream & out )  [inline], [explicit]
```

Definition at line 46 of file verilog_utils.hpp.

### 5.3.3 Member Function Documentation

#### 5.3.3.1 handle()

```
void CellPrinter::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 367 of file verilog_utils.cpp.

### 5.3.4 Member Data Documentation

#### 5.3.4.1 foundTarget_

```
bool CellPrinter::foundTarget_  [private]
```

Definition at line 53 of file verilog_utils.hpp.

#### 5.3.4.2 out_

```
std::ostream& CellPrinter::out_  [private]
```

Definition at line 52 of file verilog_utils.hpp.

#### 5.3.4.3 targetCell_

```
const std::string& CellPrinter::targetCell_  [private]
```

Definition at line 51 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

## 5.4 GroupsIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for GroupsIterator:

## Public Member Functions

- GroupsIterator (si2drGroupsIdT groups, si2drErrorT &err)
- ~GroupsIterator ()
- void next ()
- bool end ()
- LibGroup get ()

## Private Attributes

- si2drGroupsIdT groups_
- si2drGroupIdT group_
- si2drErrorT & err_

### 5.4.1 Detailed Description

Definition at line 9 of file iterators.hpp.

### 5.4.2 Constructor & Destructor Documentation

#### 5.4.2.1 GroupsIterator()

```
GroupsIterator::GroupsIterator (
            si2drGroupsIdT groups,
            si2drErrorT & err )
```

Definition at line 3 of file iterators.cpp.

#### 5.4.2.2 ~GroupsIterator()

```
GroupsIterator::~GroupsIterator ( )
```

Definition at line 6 of file iterators.cpp.

### 5.4.3 Member Function Documentation

#### 5.4.3.1 end()

```
bool GroupsIterator::end ( )
```

Definition at line 9 of file iterators.cpp.

Here is the caller graph for this function:

#### 5.4.3.2 get()

```
LibGroup GroupsIterator::get ( )
```

Definition at line 11 of file iterators.cpp.

Here is the caller graph for this function:

#### 5.4.3.3 next()

```
void GroupsIterator::next ( )
```

Definition at line 8 of file iterators.cpp.

Here is the caller graph for this function:

### 5.4.4 Member Data Documentation

#### 5.4.4.1 err_

```
si2drErrorT& GroupsIterator::err_  [private]
```

Definition at line 20 of file iterators.hpp.

#### 5.4.4.2 group_

```
si2drGroupIdT GroupsIterator::group_  [private]
```

Definition at line 19 of file iterators.hpp.

#### 5.4.4.3 groups_

```
si2drGroupsIdT GroupsIterator::groups_  [private]
```

Definition at line 18 of file iterators.hpp.

The documentation for this class was generated from the following files:

- include/iterators.hpp
- src/iterators.cpp

## 5.5 LibAttribute Class Reference

```
#include <lib_attribute.hpp>
```

Collaboration diagram for LibAttribute:

### Public Member Functions

- LibAttribute (si2drAttrIdT attr, si2drErrorT &err)
- ~LibAttribute ()
- std::string getName ()
- bool isComplex ()
- si2drValuesIdT getValues ()
- long int getInt ()
- double getFloat ()
- std::string getString ()
- bool getBoolean ()

### Private Attributes

- si2drAttrIdT attr_
- si2drErrorT & err_

### 5.5.1 Detailed Description

Definition at line 8 of file lib_attribute.hpp.

### 5.5.2 Constructor & Destructor Documentation

#### 5.5.2.1 LibAttribute()

```
LibAttribute::LibAttribute (
            si2drAttrIdT attr,
            si2drErrorT & err )
```

Definition at line 3 of file lib_attribute.cpp.

#### 5.5.2.2 ~LibAttribute()

```
LibAttribute::~LibAttribute ( )
```

Definition at line 5 of file lib_attribute.cpp.

### 5.5.3 Member Function Documentation

#### 5.5.3.1 getBoolean()

```
bool LibAttribute::getBoolean ( )
```

Definition at line 25 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.2 getFloat()

```
double LibAttribute::getFloat ( )
```

Definition at line 18 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.3 getInt()

```
long int LibAttribute::getInt ( )
```

Definition at line 16 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.4 getName()

```
std::string LibAttribute::getName ( )
```

Definition at line 7 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.5 getString()

```
std::string LibAttribute::getString ( )
```

Definition at line 20 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.6 getValues()

```
si2drValuesIdT LibAttribute::getValues ( )
```

Definition at line 14 of file lib_attribute.cpp.

Here is the caller graph for this function:

#### 5.5.3.7 isComplex()

```
bool LibAttribute::isComplex ( )
```

Definition at line 12 of file lib_attribute.cpp.

### 5.5.4 Member Data Documentation

#### 5.5.4.1 attr_

```
si2drAttrIdT LibAttribute::attr_  [private]
```

Definition at line 21 of file lib_attribute.hpp.

#### 5.5.4.2 err_

```
si2drErrorT& LibAttribute::err_  [private]
```

Definition at line 22 of file lib_attribute.hpp.

The documentation for this class was generated from the following files:

- include/lib_attribute.hpp
- src/lib_attribute.cpp

## 5.6 liberty_value_data Struct Reference

```
#include <si2dr_liberty.h>
```

### Public Attributes

- int dimensions
- int * dim_sizes
- LONG_DOUBLE ** index_info
- LONG_DOUBLE * values

### 5.6.1 Detailed Description

Definition at line 612 of file si2dr_liberty.h.

### 5.6.2 Member Data Documentation

#### 5.6.2.1 dim_sizes

```
int* liberty_value_data::dim_sizes
```

Definition at line 615 of file si2dr_liberty.h.

#### 5.6.2.2 dimensions

```
int liberty_value_data::dimensions
```

Definition at line 614 of file si2dr_liberty.h.

#### 5.6.2.3 index_info

```
LONG_DOUBLE** liberty_value_data::index_info
```

Definition at line 617 of file si2dr_liberty.h.

#### 5.6.2.4 values

```
LONG_DOUBLE* liberty_value_data::values
```

Definition at line 621 of file si2dr_liberty.h.

The documentation for this struct was generated from the following file:

- include/si2dr_liberty.h

## 5.7 LibFile Class Reference

```
#include <lib_file.hpp>
```

### Public Member Functions

- LibFile (const std::string &filepath, const std::string &loggername)
  *Constructs a LibFile object with specified file path and logger name.*
- ~LibFile ()
- void writeJsonToFile ()
  *Writes the JSON data stored in the lib_json_ member to a file.*
- void parse ()
  *Parses the Liberty file and extracts library information into a JSON structure.*
- void modify ()
- void mono (const bool is_slew)
  *Validates that lookup tables are monotonically increasing with respect to the output load.*
- void supercell (const int chain_length, const std::vector< std::string > &cell_names)
  *Creates supercells based on standard cells in the library file.*
- void verilog (const int chain_length, const std::vector< std::string > &cell_names)

### Public Attributes

- std::shared_ptr< spdlog::logger > logger_
- std::filesystem::path filepath_
- std::string basename_
- std::string filename_
- std::string libname_ = ""
- std::string jsonname_ = ""
- std::string loggername_ = ""
- json lib_json_ = json::object()

## Private Member Functions

- void read ()

  *Reads the Liberty file specified by the filename_ member variable.*
- bool checkTimingArcMonotonicity (const json &cell, const json &pin, const json &arc, const std::string &type, bool is_slew)

  *Checks if the values in a timing arc are monotonically increasing.*

## Private Attributes

- si2drErrorT err_
- int process_ = 0
- float voltage_ = 0.0
- int temperature_ = 0

### 5.7.1 Detailed Description

Definition at line 13 of file lib_file.hpp.

### 5.7.2 Constructor & Destructor Documentation

#### 5.7.2.1 LibFile()

```
LibFile::LibFile (
            const std::string & filepath,
            const std::string & loggername )
```

Constructs a LibFile object with specified file path and logger name.

This constructor initializes a LibFile object with the given file path and creates a logger with the specified name. It sets up:

- Two logging sinks: a colored console sink and a file sink
- File path information including filename, basename, and a corresponding JSON filename
- Log levels (info for console, debug for file)

**Parameters**

| | |
|---|---|
| *filepath* | The path to the file to be processed |
| *loggername* | The name for the logger and the log file |

Definition at line 28 of file lib_file.cpp.

#### 5.7.2.2 ~LibFile()

```
LibFile::~LibFile ( )
```

Definition at line 48 of file lib_file.cpp.

### 5.7.3 Member Function Documentation

#### 5.7.3.1 checkTimingArcMonotonicity()

```
bool LibFile::checkTimingArcMonotonicity (
            const json & cell,
            const json & pin,
            const json & arc,
            const std::string & timing_arc_name,
            bool is_slew )  [private]
```

Checks if the values in a timing arc are monotonically increasing.

This function examines the values in the specified timing arc to ensure they are monotonically increasing. It checks monotonicity in two dimensions:

1. Across rows (by output load capacitance)
2. Across columns (by input slew, only if is_slew is true)

The function logs detailed information about any non-monotonic values found, including cell name, pin names, and conditional statements (when clause) if present.

**Parameters**

| | |
|---|---|
| *cell* | The JSON object representing the cell being checked |
| *pin* | The JSON object representing the pin being checked |
| *arc* | The JSON object representing the timing arc being checked |
| *timing_arc_name* | The name of the timing arc to check (e.g., "cell_rise", "cell_fall") |
| *is_slew* | Boolean flag indicating whether to check monotonicity across input slew values |

**Returns**

true if all values in the timing arc are monotonic, false otherwise

**Exceptions**

| | |
|---|---|
| *None,but* | logs errors or warnings for invalid data formats or non-monotonic values |

Definition at line 219 of file lib_file.cpp.

Here is the caller graph for this function:

#### 5.7.3.2 modify()

```
void LibFile::modify ( )
```

Definition at line 196 of file lib_file.cpp.

#### 5.7.3.3 mono()

```
void LibFile::mono (
            const bool is_slew )
```

Validates that lookup tables are monotonically increasing with respect to the output load.

This function checks the following timing data in the current library to ensure monotonicity:

- cell_rise and retaining_rise
- cell_fall and retaining_fall
- rise_transition and retain_rise_slew
- fall_transition and retain_fall_slew
- min_pulse_width constraints (rise_constraint, fall_constraint)

The function first checks if a parsed JSON representation exists. If not, it parses the Liberty file and creates the JSON file. It then evaluates each timing arc in all cells and reports the pass/fail status at the end.

**Parameters**

| | |
|---|---|
| *is_slew* | Boolean flag indicating whether to check slew values rather than delay values |

The function outputs:

- Number of cells that passed/failed the monotonicity check
- List of cells that failed the check (if any)

Definition at line 340 of file lib_file.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.7.3.4 parse()

```
void LibFile::parse ( )
```

Parses the Liberty file and extracts library information into a JSON structure.

This method handles the parsing of a Liberty file by:

1. Initializing the SI2 parser interface error handler
2. Reading the Liberty file contents
3. Extracting the library name
4. Processing second-level groups including:
    - Cell definitions, which are added to the JSON structure
    - Operating conditions, extracting PVT (Process, Voltage, Temperature) values

The parsed data is stored in the lib_json_ member variable, with the following structure:

- "library_name": Name of the library
- "cells": Array of cell definitions
- "process": Process value
- "voltage": Voltage value (rounded to 2 decimal places)
- "temperature": Temperature value

**Note**

This method creates local scopes to ensure proper cleanup of SI2 iterators.

Definition at line 130 of file lib_file.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.7.3.5 read()

```
void LibFile::read ( )  [private]
```

Reads the Liberty file specified by the filename_ member variable.

This function attempts to read a Liberty file and logs the process. It measures the time taken to read the file and logs any errors encountered during the read operation. If an error occurs, the function logs the error and terminates the program.

- Logs the start of the read operation.
- Measures the duration of the read operation.
- Checks for specific errors (invalid name or syntax error) and logs them.
- Terminates the program with specific exit codes if errors are detected.
- Logs the completion of the read operation and the time taken.

**Note**

This function uses the spdlog library for logging and the si2drReadLibertyFile function for reading the Liberty file.

**Exceptions**

| | |
|---|---|
| *This* | function will terminate the program with exit codes 301 or 401 if specific errors are encountered. |

Definition at line 91 of file lib_file.cpp.

Here is the caller graph for this function:

#### 5.7.3.6 supercell()

```
void LibFile::supercell (
            const int chain_length,
            const std::vector< std::string > & cell_names )
```

Creates supercells based on standard cells in the library file.

This method creates supercells by combining standard cells in chains. For each input-output pin pair of a cell, it creates a supercell entry with naming format: <cellname>**X**<**chain_length**><input_pin>__<output_pin>. The results are written to a .map file.

For sequential cells (with clock pins), the chain length is always set to 1 regardless of the requested chain length.

**Parameters**

| | |
|---|---|
| *chain_length* | The length of the chain for combinational cells |
| *cell_names* | Vector of specific cell names to process. If empty, all cells will be processed. |

**Note**

Before creating supercells, this method checks for the existence of a JSON representation of the Liberty file and parses it if not found.

The method will report any requested cells that were not found in the library.

Definition at line 467 of file lib_file.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.7.3.7 verilog()

```
void LibFile::verilog (
            const int chain_length,
            const std::vector< std::string > & cell_names )
```

Definition at line 596 of file lib_file.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.7.3.8 writeJsonToFile()

```
void LibFile::writeJsonToFile ( )
```

Writes the JSON data stored in the lib_json_ member to a file.

This function attempts to open the specified file for writing. If the file cannot be opened, an error message is logged. If the file is successfully opened, the JSON data is written to the file with an indentation of 2 spaces. After writing, the file is closed and a success message is logged.

**Parameters**

| | |
|---|---|
| *jsonname*↩ _ | The name of the file to which the JSON data will be written. |

Definition at line 60 of file lib_file.cpp.

Here is the caller graph for this function:

## 5.7.4 Member Data Documentation

### 5.7.4.1 basename_

```
std::string LibFile::basename_
```

Definition at line 19 of file lib_file.hpp.

### 5.7.4.2 err_

```
si2drErrorT LibFile::err_  [private]
```

Definition at line 33 of file lib_file.hpp.

### 5.7.4.3 filename_

```
std::string LibFile::filename_
```

Definition at line 20 of file lib_file.hpp.

### 5.7.4.4 filepath_

```
std::filesystem::path LibFile::filepath_
```

Definition at line 18 of file lib_file.hpp.

#### 5.7.4.5 jsonname_

```
std::string LibFile::jsonname_ = ""
```

Definition at line 22 of file lib_file.hpp.

#### 5.7.4.6 lib_json_

```
json LibFile::lib_json_ = json::object()
```

Definition at line 24 of file lib_file.hpp.

#### 5.7.4.7 libname_

```
std::string LibFile::libname_ = ""
```

Definition at line 21 of file lib_file.hpp.

#### 5.7.4.8 logger_

```
std::shared_ptr<spdlog::logger> LibFile::logger_
```

Definition at line 17 of file lib_file.hpp.

#### 5.7.4.9 loggername_

```
std::string LibFile::loggername_ = ""
```

Definition at line 23 of file lib_file.hpp.

#### 5.7.4.10 process_

```
int LibFile::process_ = 0  [private]
```

Definition at line 34 of file lib_file.hpp.

#### 5.7.4.11 temperature_

```
int LibFile::temperature_ = 0  [private]
```

Definition at line 36 of file lib_file.hpp.

#### 5.7.4.12 voltage_

```
float LibFile::voltage_ = 0.0  [private]
```

Definition at line 35 of file lib_file.hpp.

The documentation for this class was generated from the following files:

- include/lib_file.hpp
- src/lib_file.cpp

## 5.8 LibGroup Class Reference

```
#include <lib_group.hpp>
```

Collaboration diagram for LibGroup:

### Public Member Functions

- LibGroup (si2drGroupIdT group, si2drErrorT &err)
- ~LibGroup ()
- std::string getName ()
- std::string getType ()
- si2drAttrsIdT getAttrs ()
- si2drGroupsIdT getGroups ()

### Private Attributes

- si2drGroupIdT group_
- si2drErrorT & err_

### 5.8.1 Detailed Description

Definition at line 8 of file lib_group.hpp.

### 5.8.2 Constructor & Destructor Documentation

#### 5.8.2.1 LibGroup()

```
LibGroup::LibGroup (
            si2drGroupIdT group,
            si2drErrorT & err )
```

Definition at line 3 of file lib_group.cpp.

#### 5.8.2.2 ~LibGroup()

```
LibGroup::~LibGroup ( )
```

Definition at line 5 of file lib_group.cpp.

### 5.8.3 Member Function Documentation

#### 5.8.3.1 getAttrs()

```
si2drAttrsIdT LibGroup::getAttrs ( )
```

Definition at line 19 of file lib_group.cpp.

Here is the caller graph for this function:

#### 5.8.3.2 getGroups()

```
si2drGroupsIdT LibGroup::getGroups ( )
```

Definition at line 21 of file lib_group.cpp.

Here is the caller graph for this function:

#### 5.8.3.3 getName()

```
std::string LibGroup::getName ( )
```

Definition at line 7 of file lib_group.cpp.

Here is the caller graph for this function:

#### 5.8.3.4 getType()

```
std::string LibGroup::getType ( )
```

Definition at line 14 of file lib_group.cpp.

Here is the caller graph for this function:

### 5.8.4 Member Data Documentation

#### 5.8.4.1 err_

```
si2drErrorT& LibGroup::err_  [private]
```

Definition at line 19 of file lib_group.hpp.

#### 5.8.4.2 group_

```
si2drGroupIdT LibGroup::group_  [private]
```

Definition at line 18 of file lib_group.hpp.

The documentation for this class was generated from the following files:

- include/lib_group.hpp
- src/lib_group.cpp

## 5.9 LibraryComparator Class Reference

```
#include <compare.hpp>
```

### Public Member Functions

- LibraryComparator (LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol)

  *Constructs a LibraryComparator object.*
- void generateReport (const std::string &output_file)

  *Generates a comparison report between the reference and comparison libraries.*

### Public Attributes

- std::filesystem::path ref_lib_path_
- std::filesystem::path comp_lib_path_

## Private Member Functions

- void compareCell (const std::string &cell_name, const json &ref_cell, const json &comp_cell, Table &table)

  *Compares the output pins of a cell between a reference JSON and a comparison JSON.*
- void comparePin (const std::string &cell_name, const std::string &pin_name, const json &ref_pin, const json &comp_pin, Table &table)

  *Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.*
- void compareTimingArc (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const json &ref_timing_arc, const json &comp_timing_arc, Table &table)

  *Compares a timing arc between two JSON objects.*
- void compareLut (const std::string &cell_name, const std::string &pin_name, const std::string &timing_type, const std::string &related_pin, const std::string &arc_name, const json &ref_lut, const json &comp_lut, Table &table)

  *Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.*

## Private Attributes

- json ref_json_
- json comp_json_
- double reltol_
- double abstol_

### 5.9.1 Detailed Description

Definition at line 12 of file compare.hpp.

### 5.9.2 Constructor & Destructor Documentation

#### 5.9.2.1 LibraryComparator()

```
LibraryComparator::LibraryComparator (
            LibFile & ref_libfile,
            LibFile & comp_libfile,
            double reltol,
            double abstol )
```

Constructs a LibraryComparator object.

This constructor initializes a LibraryComparator object by loading and parsing JSON files associated with the provided reference and comparison library files. If the JSON files do not exist, they are generated by parsing the library files.

**Parameters**

| | |
|---|---|
| *ref_libfile* | Reference to the reference library file object. |
| *comp_libfile* | Reference to the comparison library file object. |
| *reltol* | Relative tolerance value for comparison. |

Definition at line 23 of file compare.cpp.

Here is the call graph for this function:

### 5.9.3 Member Function Documentation

#### 5.9.3.1 compareCell()

```
void LibraryComparator::compareCell (
            const std::string & cell_name,
            const json & ref_cell,
            const json & comp_cell,
            Table & table )  [private]
```

Compares the output pins of a cell between a reference JSON and a comparison JSON.

This function iterates through the output pins of the comparison cell and checks if each pin exists in the reference cell. If a pin is found in both cells, it calls the comparePin function to compare the details of the pin. If a pin is not found in the reference cell, a warning is logged. If the comparison cell does not contain any output pins, an informational message is logged.

**Parameters**

| | |
|---|---|
| *cell_name* | The name of the cell being compared. |
| *ref_cell* | The reference JSON object containing the cell data. |
| *comp_cell* | The comparison JSON object containing the cell data. |
| *table* | The table object where the comparison results are stored. |

Definition at line 300 of file compare.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.9.3.2 compareLut()

```
void LibraryComparator::compareLut (
            const std::string & cell_name,
            const std::string & pin_name,
            const std::string & timing_type,
            const std::string & related_pin,
            const std::string & arc_name,
            const json & ref_lut,
            const json & comp_lut,
            Table & table )  [private]
```

Compares lookup tables (LUTs) between reference and comparison libraries for a specific timing arc.

This function compares the lookup table values between reference and comparison libraries for a given timing arc, checking the consistency of indices and values. It verifies that the indices match between libraries and then compares each value in the LUT against relative and absolute tolerance thresholds. Any discrepancies that exceed the tolerance limits are recorded in the provided table.

**Parameters**

| | |
|---|---|
| *cell_name* | The name of the cell containing the LUT. |
| *pin_name* | The name of the pin associated with the LUT. |
| *timing_type* | The timing type of the arc (e.g., "rise_transition", "fall_transition"). |
| *related_pin* | The related pin for the timing arc. |
| *arc_name* | The name of the specific timing arc being compared. |
| *ref_lut* | The lookup table from the reference library (in JSON format). |
| *comp_lut* | The lookup table from the comparison library (in JSON format). |
| *table* | Table object to store comparison results for any mismatches found. |

**Note**

The function logs various information/warning/error messages:

- Logs index mismatches as warnings
- Logs value mismatches as debug messages
- Logs format errors in LUT structure as errors
- Records values exceeding tolerance in the provided table

The comparison uses both relative tolerance (reltol_) and absolute tolerance (abstol_) to determine if values differ significantly.

Definition at line 99 of file compare.cpp.

Here is the caller graph for this function:

#### 5.9.3.3 comparePin()

```
void LibraryComparator::comparePin (
            const std::string & cell_name,
            const std::string & pin_name,
            const json & ref_pin,
            const json & comp_pin,
            Table & table )  [private]
```

Compares the timing arcs of a given pin between a reference JSON and a comparison JSON.

This function iterates through the timing arcs of the comparison pin and looks for matching timing types in the reference pin. If a matching timing type is found, it calls the compareTimingArc function to compare the timing arcs. If a timing type is not found in the reference JSON, a warning is logged.

**Parameters**

| | |
|---|---|
| *cell_name* | The name of the cell containing the pin. |
| *pin_name* | The name of the pin to compare. |
| *ref_pin* | The reference JSON object containing the pin's data. |
| *comp_pin* | The comparison JSON object containing the pin's data. |
| *table* | A reference to a Table object where comparison results are stored. |

Definition at line 261 of file compare.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.9.3.4 compareTimingArc()

```
void LibraryComparator::compareTimingArc (
            const std::string & cell_name,
            const std::string & pin_name,
            const std::string & timing_type,
            const json & ref_timing_arc,
            const json & comp_timing_arc,
            Table & table )  [private]
```

Compares a timing arc between two JSON objects.

This function compares a specific timing arc (e.g., cell_rise, cell_fall, rise_transition, fall_transition) between a reference JSON object and a comparison JSON object. It extracts the related pin from the reference timing arc and iterates through a predefined list of arc names. If an arc name is found in the comparison timing arc, it checks if the same arc name exists in the reference timing arc. If both exist, it calls the compareLut function to compare the LUT data associated with the arc. If the arc name is found in the comparison JSON but not in the reference JSON, a warning message is logged.

**Parameters**

| | |
|---|---|
| *cell_name* | The name of the cell. |
| *pin_name* | The name of the pin. |
| *timing_type* | The type of timing (e.g., "combinational", "sequential"). |
| *ref_timing_arc* | The reference JSON object containing the timing arc information. |
| *comp_timing_arc* | The comparison JSON object containing the timing arc information. |
| *table* | The table to store comparison results. |

Definition at line 221 of file compare.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 5.9.3.5 generateReport()

```
void LibraryComparator::generateReport (
            const std::string & output_file )
```

Generates a comparison report between the reference and comparison libraries.

This function generates a detailed comparison report between the reference library and the comparison library. The report includes metadata such as the reference and comparison library paths, absolute and relative tolerances, and the application details. It also includes a legend explaining various symbols used in the report.

The function iterates through each cell in the comparison library, compares it with the corresponding cell in the reference library, and generates a table of differences. If there are any differences, it calculates summary statistics such as the average and maximum differences, and the number of outliers. The results are written to the specified output file in either Markdown or plain text format.

**Parameters**

| | |
|---|---|
| *output_file* | The path to the output file where the report will be written. |

Definition at line 342 of file compare.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

### 5.9.4 Member Data Documentation

#### 5.9.4.1 abstol_

```
double LibraryComparator::abstol_  [private]
```

Definition at line 23 of file compare.hpp.

#### 5.9.4.2 comp_json_

```
json LibraryComparator::comp_json_  [private]
```

Definition at line 21 of file compare.hpp.

#### 5.9.4.3 comp_lib_path_

```
std::filesystem::path LibraryComparator::comp_lib_path_
```

Definition at line 16 of file compare.hpp.

#### 5.9.4.4 ref_json_

```
json LibraryComparator::ref_json_  [private]
```

Definition at line 20 of file compare.hpp.

#### 5.9.4.5 ref_lib_path_

```
std::filesystem::path LibraryComparator::ref_lib_path_
```

Definition at line 15 of file compare.hpp.

#### 5.9.4.6 reltol_

```
double LibraryComparator::reltol_  [private]
```

Definition at line 22 of file compare.hpp.

The documentation for this class was generated from the following files:

- include/compare.hpp
- src/compare.cpp

## 5.10 ModuleRewriter Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for ModuleRewriter:

Collaboration diagram for ModuleRewriter:

### Public Member Functions

- ModuleRewriter (const std::vector< std::string > &inputPins, const std::vector< std::string > &outputPins, const std::pair< std::string, std::string > &supercell_entry, int instance_count, std::shared_ptr< spdlog↩ ::logger > logger)
- void handle (const slang::syntax::SyntaxNode &node)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)

### Public Attributes

- std::shared_ptr< spdlog::logger > logger_

### Private Attributes

- const std::vector< std::string > & inputPins_
- const std::vector< std::string > & outputPins_
- const std::string cellName_
- const std::string moduleName_
- int depth_
- int instance_count_

### 5.10.1 Detailed Description

Definition at line 57 of file verilog_utils.hpp.

### 5.10.2 Constructor & Destructor Documentation

#### 5.10.2.1 ModuleRewriter()

```
ModuleRewriter::ModuleRewriter (
            const std::vector< std::string > & inputPins,
            const std::vector< std::string > & outputPins,
            const std::pair< std::string, std::string > & supercell_entry,
            int instance_count,
            std::shared_ptr< spdlog::logger > logger )  [inline], [explicit]
```

Definition at line 59 of file verilog_utils.hpp.

### 5.10.3 Member Function Documentation

#### 5.10.3.1 handle() [1/2]

```
void ModuleRewriter::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 397 of file verilog_utils.cpp.

#### 5.10.3.2 handle() [2/2]

```
void ModuleRewriter::handle (
            const slang::syntax::SyntaxNode & node )
```

Definition at line 387 of file verilog_utils.cpp.

### 5.10.4 Member Data Documentation

#### 5.10.4.1 cellName_

```
const std::string ModuleRewriter::cellName_  [private]
```

Definition at line 73 of file verilog_utils.hpp.

#### 5.10.4.2 depth_

```
int ModuleRewriter::depth_  [private]
```

Definition at line 75 of file verilog_utils.hpp.

#### 5.10.4.3 inputPins_

```
const std::vector<std::string>& ModuleRewriter::inputPins_  [private]
```

Definition at line 71 of file verilog_utils.hpp.

#### 5.10.4.4 instance_count_

```
int ModuleRewriter::instance_count_  [private]
```

Definition at line 76 of file verilog_utils.hpp.

#### 5.10.4.5 logger_

```
std::shared_ptr<spdlog::logger> ModuleRewriter::logger_
```

Definition at line 66 of file verilog_utils.hpp.

#### 5.10.4.6 moduleName_

```
const std::string ModuleRewriter::moduleName_  [private]
```

Definition at line 74 of file verilog_utils.hpp.

#### 5.10.4.7 outputPins_

```
const std::vector<std::string>& ModuleRewriter::outputPins_  [private]
```

Definition at line 72 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

## 5.11 si2drExprT Struct Reference

```
#include <si2dr_liberty.h>
```

Collaboration diagram for si2drExprT:

### Public Attributes

- si2drExprTypeT type
- union {
  si2drInt32T i
  si2drFloat64T d
  si2drStringT s
  si2drBooleanT b
  } u
- si2drValueTypeT valuetype
- struct si2drExprT * left
- struct si2drExprT * right

### 5.11.1 Detailed Description

Definition at line 172 of file si2dr_liberty.h.

### 5.11.2 Member Data Documentation

#### 5.11.2.1 b

```
si2drBooleanT si2drExprT::b
```

Definition at line 180 of file si2dr_liberty.h.

#### 5.11.2.2 d

```
si2drFloat64T si2drExprT::d
```

Definition at line 178 of file si2dr_liberty.h.

#### 5.11.2.3 i

```
si2drInt32T si2drExprT::i
```

Definition at line 177 of file si2dr_liberty.h.

#### 5.11.2.4 left

```
struct si2drExprT* si2drExprT::left
```

Definition at line 185 of file si2dr_liberty.h.

#### 5.11.2.5 right

```
struct si2drExprT* si2drExprT::right
```

Definition at line 186 of file si2dr_liberty.h.

#### 5.11.2.6 s

```
si2drStringT si2drExprT::s
```

Definition at line 179 of file si2dr_liberty.h.

#### 5.11.2.7 type

```
si2drExprTypeT si2drExprT::type
```

Definition at line 174 of file si2dr_liberty.h.

#### 5.11.2.8

```
union { ... }  si2drExprT::u
```

#### 5.11.2.9 valuetype

```
si2drValueTypeT si2drExprT::valuetype
```

Definition at line 183 of file si2dr_liberty.h.

The documentation for this struct was generated from the following file:

- include/si2dr_liberty.h

## 5.12 si2OID Struct Reference

```
#include <si2dr_liberty.h>
```

### Public Attributes

- void * v1
- void * v2

### 5.12.1 Detailed Description

Definition at line 79 of file si2dr_liberty.h.

### 5.12.2 Member Data Documentation

#### 5.12.2.1 v1

```
void* si2OID::v1
```

Definition at line 79 of file si2dr_liberty.h.

#### 5.12.2.2 v2

```
void * si2OID::v2
```

Definition at line 79 of file si2dr_liberty.h.

The documentation for this struct was generated from the following file:

- include/si2dr_liberty.h

## 5.13 ValuesIterator Class Reference

```
#include <iterators.hpp>
```

Collaboration diagram for ValuesIterator:

### Public Member Functions

- ValuesIterator (si2drValuesIdT values, si2drErrorT &err)
- ~ValuesIterator ()
- void next ()
- bool end ()

### Public Attributes

- si2drValuesIdT values_
- si2drValueTypeT vtype_
- si2drInt32T int_
- si2drFloat64T float_
- si2drStringT str_
- si2drBooleanT bool_
- si2drExprT * exprp_
- si2drErrorT & err_

### 5.13.1 Detailed Description

Definition at line 37 of file iterators.hpp.

### 5.13.2 Constructor & Destructor Documentation

#### 5.13.2.1 ValuesIterator()

```
ValuesIterator::ValuesIterator (
            si2drValuesIdT values,
            si2drErrorT & err )
```

Definition at line 24 of file iterators.cpp.

#### 5.13.2.2 ~ValuesIterator()

```
ValuesIterator::~ValuesIterator ( )
```

Definition at line 27 of file iterators.cpp.

### 5.13.3 Member Function Documentation

#### 5.13.3.1 end()

```
bool ValuesIterator::end ( )
```

Definition at line 32 of file iterators.cpp.

Here is the caller graph for this function:

#### 5.13.3.2 next()

```
void ValuesIterator::next ( )
```

Definition at line 29 of file iterators.cpp.

Here is the caller graph for this function:

### 5.13.4 Member Data Documentation

#### 5.13.4.1 bool_

```
si2drBooleanT ValuesIterator::bool_
```

Definition at line 49 of file iterators.hpp.

#### 5.13.4.2 err_

```
si2drErrorT& ValuesIterator::err_
```

Definition at line 51 of file iterators.hpp.

#### 5.13.4.3 exprp_

```
si2drExprT* ValuesIterator::exprp_
```

Definition at line 50 of file iterators.hpp.

#### 5.13.4.4 float_

```
si2drFloat64T ValuesIterator::float_
```

Definition at line 47 of file iterators.hpp.

#### 5.13.4.5 int_

```
si2drInt32T ValuesIterator::int_
```

Definition at line 46 of file iterators.hpp.

#### 5.13.4.6 str_

```
si2drStringT ValuesIterator::str_
```

Definition at line 48 of file iterators.hpp.

#### 5.13.4.7 values_

```
si2drValuesIdT ValuesIterator::values_
```

Definition at line 44 of file iterators.hpp.

#### 5.13.4.8 vtype_

```
si2drValueTypeT ValuesIterator::vtype_
```

Definition at line 45 of file iterators.hpp.

The documentation for this class was generated from the following files:

- include/iterators.hpp
- src/iterators.cpp

## 5.14 VerilogVisitor Class Reference

```
#include <verilog_utils.hpp>
```

Inheritance diagram for VerilogVisitor:

Collaboration diagram for VerilogVisitor:

### Public Member Functions

- VerilogVisitor (const std::string &targetCell)
- void handle (const slang::syntax::SyntaxNode &node)
- void handle (const slang::syntax::ModuleDeclarationSyntax &module)
- void handle (const slang::syntax::PortDeclarationSyntax &portDecl)
- void handle (const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst)
- void handle (const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst)
- void handle (const slang::syntax::SpecifyBlockSyntax &specifyBlock)

### Private Attributes

- const std::string & targetCell_
- int depth_
- bool inTargetModule_

### 5.14.1 Detailed Description

Definition at line 14 of file verilog_utils.hpp.

### 5.14.2 Constructor & Destructor Documentation

#### 5.14.2.1 VerilogVisitor()

```
VerilogVisitor::VerilogVisitor (
            const std::string & targetCell )  [inline], [explicit]
```

Definition at line 16 of file verilog_utils.hpp.

### 5.14.3 Member Function Documentation

#### 5.14.3.1 handle() [1/6]

```
void VerilogVisitor::handle (
            const slang::syntax::HierarchyInstantiationSyntax & hierarchyInst )
```

Definition at line 80 of file verilog_utils.cpp.

#### 5.14.3.2 handle() [2/6]

```
void VerilogVisitor::handle (
            const slang::syntax::ModuleDeclarationSyntax & module )
```

Definition at line 19 of file verilog_utils.cpp.

#### 5.14.3.3 handle() [3/6]

```
void VerilogVisitor::handle (
            const slang::syntax::PortDeclarationSyntax & portDecl )
```

Definition at line 48 of file verilog_utils.cpp.

#### 5.14.3.4 handle() [4/6]

```
void VerilogVisitor::handle (
            const slang::syntax::PrimitiveInstantiationSyntax & primitiveInst )
```

Definition at line 192 of file verilog_utils.cpp.

#### 5.14.3.5 handle() [5/6]

```
void VerilogVisitor::handle (
            const slang::syntax::SpecifyBlockSyntax & specifyBlock )
```

Definition at line 259 of file verilog_utils.cpp.

Here is the call graph for this function:

#### 5.14.3.6 handle() [6/6]

```
void VerilogVisitor::handle (
            const slang::syntax::SyntaxNode & node )
```

Definition at line 8 of file verilog_utils.cpp.

Here is the caller graph for this function:

### 5.14.4 Member Data Documentation

#### 5.14.4.1 depth_

```
int VerilogVisitor::depth_  [private]
```

Definition at line 27 of file verilog_utils.hpp.

#### 5.14.4.2 inTargetModule_

```
bool VerilogVisitor::inTargetModule_  [private]
```

Definition at line 28 of file verilog_utils.hpp.

#### 5.14.4.3 targetCell_

```
const std::string& VerilogVisitor::targetCell_  [private]
```

Definition at line 26 of file verilog_utils.hpp.

The documentation for this class was generated from the following files:

- include/verilog_utils.hpp
- src/verilog_utils.cpp

# Chapter 6

# File Documentation

## 6.1 include/compare.hpp File Reference

```
#include "lib_file.hpp"
#include "nlohmann/json.hpp"
#include "tabulate/table.hpp"
```

Include dependency graph for compare.hpp: This graph shows which files directly or indirectly include this file:

### Classes

- class LibraryComparator

### Typedefs

- using json = nlohmann::json

### 6.1.1 Typedef Documentation

#### 6.1.1.1 json

```
using json = nlohmann::json
```

Definition at line 9 of file compare.hpp.

## 6.2 compare.hpp

Go to the documentation of this file.

```
00001 #ifndef COMPARE_HPP
00002 #define COMPARE_HPP
00003
00004 #include "lib_file.hpp"
00005
00006 #include "nlohmann/json.hpp"
00007 #include "tabulate/table.hpp"
00008
00009 using json = nlohmann::json;
00010 using namespace tabulate;
00011
00012 class LibraryComparator {
00013 public:
00014   LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol, double abstol);
00015   std::filesystem::path ref_lib_path_;
00016   std::filesystem::path comp_lib_path_;
00017   void generateReport(const std::string &output_file);
00018
00019 private:
00020   json ref_json_;
00021   json comp_json_;
00022   double reltol_;
00023   double abstol_;
00024
00025   void compareCell(const std::string &cell_name, const json &ref_cell, const json &comp_cell,
00026                    Table &table);
00027   void comparePin(const std::string &cell_name, const std::string &pin_name, const json &ref_pin,
00028                   const json &comp_pin, Table &table);
00029   void compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00030                         const std::string &timing_type, const json &ref_timing_arc,
00031                         const json &comp_timing_arc, Table &table);
00032   void compareLut(const std::string &cell_name, const std::string &pin_name,
00033                   const std::string &timing_type, const std::string &related_pin,
00034                   const std::string &arc_name, const json &ref_lut, const json &comp_lut, Table
      &table);
00035   // void configureTableFormat(Table &table);
00036 };
00037
00038 #endif // COMPARE_HPP
```

## 6.3 include/iterators.hpp File Reference

```
#include "si2dr_liberty.h"
#include "lib_attribute.hpp"
#include "lib_group.hpp"
```

Include dependency graph for iterators.hpp: This graph shows which files directly or indirectly include this file:

### Classes

- class GroupsIterator
- class AttributesIterator
- class ValuesIterator

## 6.4 iterators.hpp

Go to the documentation of this file.

```
00001 #ifndef ITERATORS_H
00002 #define ITERATORS_H
00003
00004 #include "si2dr_liberty.h"
00005
00006 #include "lib_attribute.hpp"
00007 #include "lib_group.hpp"
```

```
00008
00009 class GroupsIterator {
00010 public:
00011   GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err);
00012   ~GroupsIterator();
00013   void next();
00014   bool end();
00015   LibGroup get();
00016
00017 private:
00018   si2drGroupsIdT groups_;
00019   si2drGroupIdT group_;
00020   si2drErrorT &err_;
00021 };
00022
00023 class AttributesIterator {
00024 public:
00025   AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err);
00026   ~AttributesIterator();
00027   void next();
00028   bool end();
00029   LibAttribute get();
00030
00031 private:
00032   si2drAttrsIdT attrs_;
00033   si2drAttrIdT attr_;
00034   si2drErrorT &err_;
00035 };
00036
00037 class ValuesIterator {
00038 public:
00039   ValuesIterator(si2drValuesIdT values, si2drErrorT &err);
00040   ~ValuesIterator();
00041   void next();
00042   bool end();
00043
00044   si2drValuesIdT values_;
00045   si2drValueTypeT vtype_;
00046   si2drInt32T int_;
00047   si2drFloat64T float_;
00048   si2drStringT str_;
00049   si2drBooleanT bool_;
00050   si2drExprT *exprp_;
00051   si2drErrorT &err_;
00052 };
00053
00054 #endif // ITERATORS_H
```

## 6.5 include/json_utils.hpp File Reference

```
#include <string>
#include <utility>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
#include "lib_group.hpp"
```

Include dependency graph for json_utils.hpp: This graph shows which files directly or indirectly include this file:

### Typedefs

- using json = nlohmann::json

### Functions

- json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)

  *Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.*
- json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)

  *Converts a Liberty power group into a JSON representation.*
- std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)

  *Generates a JSON representation of a Liberty pin group.*
- json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)

  *Generates a JSON representation of a cell from a LibGroup object.*

### 6.5.1 Typedef Documentation

#### 6.5.1.1 json

```
using json = nlohmann::json
```

Definition at line 12 of file json_utils.hpp.

### 6.5.2 Function Documentation

#### 6.5.2.1 generateCellJson()

```
json generateCellJson (
            LibGroup & lib_cell_group,
            si2drErrorT & err )
```

Generates a JSON representation of a cell from a LibGroup object.

This function processes a LibGroup representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

**Parameters**

| | |
|---|---|
| *lib_cell_group* | The LibGroup object representing the cell |
| *err* | Reference to a si2drErrorT object for error handling |

**Returns**

json A JSON object containing the cell's information with the following structure:

- "cell_name": string - name of the cell
- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 272 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.5.2.2 generateLutJson()

```
json generateLutJson (
            LibGroup & lib_lut_group,
            si2drErrorT & err )
```

Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.

This function iterates through the attributes of the provided LibGroup object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

**Parameters**

| | |
|---|---|
| *lib_lut_group* | The LibGroup object containing the LUT data to be converted |
| *err* | Reference to an si2drErrorT object to track any errors during processing |

**Returns**

json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 62 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.5.2.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
            LibGroup & lib_pin_group,
            si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using generatePowerJson()
- timing: Converted using generateTimingJson()

**Parameters**

| | |
|---|---|
| *lib_pin_group* | The Liberty pin group to process |
| *err* | Reference to an error object for tracking Liberty API errors |

**Returns**

A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 206 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.5.2.4 generatePowerJson()

```
json generatePowerJson (
            LibGroup & lib_power_group,
            si2drErrorT & err )
```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

**Parameters**

| | |
|---|---|
| *lib_power_group* | The Liberty power group to convert |
| *err* | Reference to an si2drErrorT object for error handling |

**Returns**

json A JSON object representing the power group data

The function:

- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 154 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

## 6.6 json_utils.hpp

Go to the documentation of this file.

```
00001 #ifndef JSON_UTILS_HPP
00002 #define JSON_UTILS_HPP
00003
00004 #include <string>
00005 #include <utility>
00006
00007 #include "nlohmann/json.hpp"
00008 #include "si2dr_liberty.h"
00009
00010 #include "lib_group.hpp"
00011
00012 using json = nlohmann::json;
00013
00014 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err);
00015 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err);
00016 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err);
00017 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err);
00018
00019 #endif // JSON_UTILS_HPP
```

## 6.7 include/lib_attribute.hpp File Reference

```
#include <string>
#include "si2dr_liberty.h"
```

Include dependency graph for lib_attribute.hpp: This graph shows which files directly or indirectly include this file:

### Classes

- class LibAttribute

## 6.8 lib_attribute.hpp

Go to the documentation of this file.

```
00001 #ifndef LIB_ATTRIBUTE_H
00002 #define LIB_ATTRIBUTE_H
00003
00004 #include <string>
00005
00006 #include "si2dr_liberty.h"
00007
00008 class LibAttribute {
00009 public:
00010   LibAttribute(si2drAttrIdT attr, si2drErrorT &err);
00011   ~LibAttribute();
00012   std::string getName();
00013   bool isComplex();
00014   si2drValuesIdT getValues();
00015   long int getInt();
00016   double getFloat();
00017   std::string getString();
00018   bool getBoolean();
00019
00020 private:
00021   si2drAttrIdT attr_;
00022   si2drErrorT &err_;
00023 };
00024
00025 #endif // LIB_ATTRIBUTE_H
```

## 6.9 include/lib_file.hpp File Reference

```
#include <filesystem>
#include <string>
#include <vector>
#include "nlohmann/json.hpp"
#include "si2dr_liberty.h"
```

Include dependency graph for lib_file.hpp: This graph shows which files directly or indirectly include this file:

### Classes

- class LibFile

### Typedefs

- using json = nlohmann::json

### 6.9.1 Typedef Documentation

#### 6.9.1.1 json

```
using json = nlohmann::json
```

Definition at line 11 of file lib_file.hpp.

## 6.10 lib_file.hpp

Go to the documentation of this file.

```
00001 #ifndef LIB_FILE_H
00002 #define LIB_FILE_H
00003
00004 #include <filesystem>
00005 #include <string>
00006 #include <vector>
00007
00008 #include "nlohmann/json.hpp"
00009 #include "si2dr_liberty.h"
00010
00011 using json = nlohmann::json;
00012
00013 class LibFile {
00014 public:
00015   LibFile(const std::string &filepath, const std::string &loggername);
00016   ~LibFile();
00017   std::shared_ptr<spdlog::logger> logger_;
00018   std::filesystem::path filepath_; // full path to the file
00019   std::string basename_;           // file name without extension
00020   std::string filename_;           // full file name with extension
00021   std::string libname_ = "";       // library name obtained through parsing
00022   std::string jsonname_ = "";      // json file name to store parsed data
00023   std::string loggername_ = "";    // log file name
00024   json lib_json_ = json::object();
00025   void writeJsonToFile();
00026   void parse();
00027   void modify();
00028   void mono(const bool is_slew);
```

```
00029   void supercell(const int chain_length, const std::vector<std::string> &cell_names);
00030   void verilog(const int chain_length, const std::vector<std::string> &cell_names);
00031
00032 private:
00033   si2drErrorT err_;
00034   int process_ = 0;
00035   float voltage_ = 0.0;
00036   int temperature_ = 0;
00037   void read();
00038   bool checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
00039                                   const std::string &type, bool is_slew);
00040 };
00041
00042 #endif // LIB_FILE_H
```

## 6.11 include/lib_group.hpp File Reference

```
#include <string>
#include "si2dr_liberty.h"
```

Include dependency graph for lib_group.hpp: This graph shows which files directly or indirectly include this file:

### Classes

- class LibGroup

## 6.12 lib_group.hpp

Go to the documentation of this file.

```
00001 #ifndef LIB_GROUP_H
00002 #define LIB_GROUP_H
00003
00004 #include <string>
00005
00006 #include "si2dr_liberty.h"
00007
00008 class LibGroup {
00009 public:
00010   LibGroup(si2drGroupIdT group, si2drErrorT &err);
00011   ~LibGroup();
00012   std::string getName();
00013   std::string getType();
00014   si2drAttrsIdT getAttrs();
00015   si2drGroupsIdT getGroups();
00016
00017 private:
00018   si2drGroupIdT group_;
00019   si2drErrorT &err_;
00020 };
00021
00022 #endif // LIB_GROUP_H
```

## 6.13 include/si2dr_liberty.h File Reference

This graph shows which files directly or indirectly include this file:

### Classes

- struct si2OID
- struct si2drExprT
- struct liberty_value_data

## Macros

- #define SI2_ARGS(args) ()
- #define SI2_LONG_MAX 2147483647
- #define SI2_LONG_MIN -2147483647
- #define SI2_ULONG_MAX 4294967295
- #define SI2DR_MIN_FLOAT32 -1E+37
- #define SI2DR_MAX_FLOAT32 1E+37
- #define SI2DR_MIN_FLOAT64 -1.797693e+308
- #define SI2DR_MAX_FLOAT64 1.797693e+308
- #define SI2DR_MIN_INT32 -2147483647
- #define SI2DR_MAX_INT32 2147483647
- #define SI2DR_MAX_STRING_LEN 1048576 /∗1024∗1024∗/
- #define SI2DR_TRUE SI2_TRUE
- #define SI2DR_FALSE SI2_FALSE
- #define LONG_DOUBLE long double

## Typedefs

- typedef enum si2BooleanEnum si2BooleanT
- typedef long int si2LongT
- typedef float si2FloatT
- typedef double si2DoubleT
- typedef char ∗ si2StringT
- typedef void si2VoidT
- typedef enum si2TypeEnum si2TypeT
- typedef struct si2OID si2ObjectIdT
- typedef void ∗ si2IteratorIdT
- typedef si2BooleanT si2drBooleanT
- typedef si2LongT si2drInt32T
- typedef si2FloatT si2drFloat32T
- typedef si2DoubleT si2drFloat64T
- typedef si2StringT si2drStringT
- typedef si2VoidT si2drVoidT
- typedef si2IteratorIdT si2drIterIdT
- typedef si2ObjectIdT si2drObjectIdT
- typedef si2drObjectIdT si2drGroupIdT
- typedef si2drObjectIdT si2drAttrIdT
- typedef si2drObjectIdT si2drAttrComplexValIdT
- typedef si2drObjectIdT si2drDefineIdT
- typedef si2drIterIdT si2drObjectsIdT
- typedef si2drIterIdT si2drGroupsIdT
- typedef si2drIterIdT si2drAttrsIdT
- typedef si2drIterIdT si2drDefinesIdT
- typedef si2drIterIdT si2drNamesIdT
- typedef si2drIterIdT si2drValuesIdT
- typedef enum si2drSeverityT si2drSeverityT
- typedef enum si2drObjectTypeT si2drObjectTypeT
- typedef enum si2drAttrTypeT si2drAttrTypeT
- typedef enum si2drValueTypeT si2drValueTypeT
- typedef enum si2drExprTypeT si2drExprTypeT
- typedef struct si2drExprT si2drExprT
- typedef enum si2drErrorT si2drErrorT
- typedef si2drVoidT(∗ si2drMessageHandlerT) (si2drSeverityT sev, si2drErrorT errToPrint, si2drStringT auxText, si2drErrorT ∗err)

## Enumerations

- enum si2BooleanEnum { SI2_FALSE = 0 , SI2_TRUE = 1 }
- enum si2TypeEnum {
  SI2_UNDEFINED_VALUETYPE = 0 , SI2_BOOLEAN = 1 , SI2_LONG = 2 , SI2_FLOAT = 3 ,
  SI2_DOUBLE = 4 , SI2_STRING = 5 , SI2_VOID = 6 , SI2_OID = 7 ,
  SI2_ITER = 8 , SI2_EXPR = 9 , SI2_MAX_VALUETYPE = 10 }
- enum si2drSeverityT { SI2DR_SEVERITY_NOTE = 0 , SI2DR_SEVERITY_WARN = 1 , SI2DR_SEVERITY_ERR = 2 }
- enum si2drObjectTypeT {
  SI2DR_UNDEFINED_OBJECTTYPE = 0 , SI2DR_GROUP = 1 , SI2DR_ATTR = 2 , SI2DR_DEFINE = 3 ,
  SI2DR_COMPLEXVAL = 4 , SI2DR_MAX_OBJECTTYPE = 5 }
- enum si2drAttrTypeT { SI2DR_SIMPLE , SI2DR_COMPLEX }
- enum si2drValueTypeT {
  SI2DR_UNDEFINED_VALUETYPE = SI2_UNDEFINED_VALUETYPE , SI2DR_INT32 = SI2_LONG ,
  SI2DR_STRING = SI2_STRING , SI2DR_FLOAT64 = SI2_DOUBLE ,
  SI2DR_BOOLEAN = SI2_BOOLEAN , SI2DR_EXPR = SI2_EXPR , SI2DR_MAX_VALUETYPE = SI2_↩
  MAX_VALUETYPE }
- enum si2drExprTypeT {
  SI2DR_EXPR_VAL = 0 , SI2DR_EXPR_OP_ADD = 1 , SI2DR_EXPR_OP_SUB = 2 , SI2DR_EXPR_OP_MUL = 3 ,
  SI2DR_EXPR_OP_DIV = 4 , SI2DR_EXPR_OP_PAREN = 5 , SI2DR_EXPR_OP_LOG2 = 6 ,
  SI2DR_EXPR_OP_LOG10 = 7 ,
  SI2DR_EXPR_OP_EXP = 8 }
- enum si2drErrorT {
  SI2DR_NO_ERROR = 0 , SI2DR_INTERNAL_SYSTEM_ERROR = 1 , SI2DR_INVALID_VALUE = 4 ,
  SI2DR_INVALID_NAME = 5 ,
  SI2DR_INVALID_OBJECTTYPE = 6 , SI2DR_INVALID_ATTRTYPE = 10 , SI2DR_UNUSABLE_OID = 7 ,
  SI2DR_OBJECT_ALREADY_EXISTS = 8 ,
  SI2DR_OBJECT_NOT_FOUND = 9 , SI2DR_SYNTAX_ERROR = 2 , SI2DR_TRACE_FILES_CANNOT_BE_OPENED = 3 , SI2DR_PIINIT_NOT_CALLED = 11 ,
  SI2DR_SEMANTIC_ERROR = 12 , SI2DR_REFERENCE_ERROR = 13 , SI2DR_MAX_ERROR = 14 }

## Functions

- si2drVoidT si2drDefaultMessageHandler (si2drSeverityT sev, si2drErrorT errToPrint, si2drStringT auxText, si2drErrorT *err)
- si2drMessageHandlerT si2drPIGetMessageHandler (si2drErrorT *err)
- si2drVoidT si2drPISetMessageHandler (si2drMessageHandlerT MsgHandFuncPtr, si2drErrorT *err)
- si2drGroupIdT si2drPICreateGroup SI2_ARGS ((si2drStringT name, si2drStringT group_type, si2drErrorT *err))
- si2drAttrIdT si2drGroupCreateAttr SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drAttrTypeT type, si2drErrorT *err))
- si2drAttrTypeT si2drAttrGetAttrType SI2_ARGS ((si2drAttrIdT attr, si2drErrorT *err))
- si2drVoidT si2drComplexAttrAddInt32Value SI2_ARGS ((si2drAttrIdT attr, si2drInt32T intgr, si2drErrorT *err))
- si2drVoidT si2drComplexAttrAddStringValue SI2_ARGS ((si2drAttrIdT attr, si2drStringT string, si2drErrorT *err))
- si2drVoidT si2drComplexAttrAddBooleanValue SI2_ARGS ((si2drAttrIdT attr, si2drBooleanT boolval, si2drErrorT *err))
- si2drVoidT si2drComplexAttrAddFloat64Value SI2_ARGS ((si2drAttrIdT attr, si2drFloat64T float64, si2drErrorT *err))
- si2drVoidT si2drComplexAttrAddExprValue SI2_ARGS ((si2drAttrIdT attr, si2drExprT *expr, si2drErrorT *err))
- si2drVoidT si2drIterNextComplexValue SI2_ARGS ((si2drValuesIdT iter, si2drValueTypeT *type, si2drInt32T *intgr, si2drFloat64T *float64, si2drStringT *string, si2drBooleanT *boolval, si2drExprT **expr, si2drErrorT *err))
- si2drAttrComplexValIdT si2drIterNextComplex SI2_ARGS ((si2drValuesIdT iter, si2drErrorT *err))

- si2drVoidT si2drSimpleAttrSetBooleanValue SI2_ARGS ((si2drAttrIdT attr, si2drBooleanT intgr, si2drErrorT *err))
- si2drVoidT si2drExprDestroy SI2_ARGS ((si2drExprT *expr, si2drErrorT *err))
- si2drExprT *si2drCreateExpr SI2_ARGS ((si2drExprTypeT type, si2drErrorT *err))
- si2drExprT *si2drCreateBooleanValExpr SI2_ARGS ((si2drBooleanT b, si2drErrorT *err))
- si2drExprT *si2drCreateDoubleValExpr SI2_ARGS ((si2drFloat64T d, si2drErrorT *err))
- si2drExprT *si2drCreateStringValExpr SI2_ARGS ((si2drStringT s, si2drErrorT *err))
- si2drExprT *si2drCreateIntValExpr SI2_ARGS ((si2drInt32T i, si2drErrorT *err))
- si2drExprT *si2drCreateBinaryOpExpr SI2_ARGS ((si2drExprT *left, si2drExprTypeT optype, si2drExprT *right, si2drErrorT *err))
- si2drExprT *si2drCreateUnaryOpExpr SI2_ARGS ((si2drExprTypeT optype, si2drExprT *expr, si2drErrorT *err))
- si2drDefineIdT si2drGroupCreateDefine SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT allowed_group_name, si2drValueTypeT valtype, si2drErrorT *err))
- si2drVoidT si2drDefineGetInfo SI2_ARGS ((si2drDefineIdT def, si2drStringT *name, si2drStringT *allowed↩ _group_name, si2drValueTypeT *valtype, si2drErrorT *err))
- si2drStringT si2drDefineGetName SI2_ARGS ((si2drDefineIdT def, si2drErrorT *err))
- si2drGroupIdT si2drGroupCreateGroup SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT group_type, si2drErrorT *err))
- si2drStringT si2drGroupGetGroupType SI2_ARGS ((si2drGroupIdT group, si2drErrorT *err))
- si2drVoidT si2drGroupSetComment SI2_ARGS ((si2drGroupIdT group, si2drStringT comment, si2drErrorT *err))
- si2drVoidT si2drAttrSetComment SI2_ARGS ((si2drAttrIdT attr, si2drStringT comment, si2drErrorT *err))
- si2drVoidT si2drDefineSetComment SI2_ARGS ((si2drDefineIdT def, si2drStringT comment, si2drErrorT *err))
- si2drVoidT si2drGroupAddName SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drErrorT *err))
- si2drGroupIdT si2drPIFindGroupByName SI2_ARGS ((si2drStringT name, si2drStringT type, si2drErrorT *err))
- si2drGroupIdT si2drGroupFindGroupByName SI2_ARGS ((si2drGroupIdT group, si2drStringT name, si2drStringT type, si2drErrorT *err))
- si2drDefineIdT si2drPIFindDefineByName SI2_ARGS ((si2drStringT name, si2drErrorT *err))
- si2drGroupsIdT si2drPIGetGroups SI2_ARGS ((si2drErrorT *err))
- si2drVoidT si2drObjectDelete SI2_ARGS ((si2drObjectIdT object, si2drErrorT *err))
- si2drStringT si2drPIGetErrorText SI2_ARGS ((si2drErrorT errorCode, si2drErrorT *err))
- si2drBooleanT si2drObjectIsSame SI2_ARGS ((si2drObjectIdT object1, si2drObjectIdT object2, si2drErrorT *err))
- si2drVoidT si2drObjectSetFileName SI2_ARGS ((si2drObjectIdT object, si2drStringT filename, si2drErrorT *err))
- si2drVoidT si2drObjectSetLineNo SI2_ARGS ((si2drObjectIdT object, si2drInt32T lineno, si2drErrorT *err))
- si2drVoidT si2drReadLibertyFile SI2_ARGS ((char *filename, si2drErrorT *err))
- si2drVoidT si2drWriteLibertyFile SI2_ARGS ((char *filename, si2drGroupIdT group, char *cellname, si2drErrorT *err))
- si2drVoidT si2drPISetTraceMode SI2_ARGS ((si2drStringT fname, si2drErrorT *err))
- si2drVoidT si2drGroupMoveAfter SI2_ARGS ((si2drGroupIdT groupToMove, si2drGroupIdT targetGroup, si2drErrorT *err))
- LONG_DOUBLE liberty_get_element (struct liberty_value_data *vd,...)
- void liberty_destroy_value_data (struct liberty_value_data *vd)
- struct liberty_value_data * liberty_get_values_data (si2drGroupIdT table_group)
- void print_version ()
- void usage ()
- int main_liberty_parser (int argc, char *argv[])
- void get_function (char *filename)

### 6.13.1 Macro Definition Documentation

#### 6.13.1.1 LONG_DOUBLE

```
#define LONG_DOUBLE long double
```

Definition at line 609 of file si2dr_liberty.h.

#### 6.13.1.2 SI2_ARGS

```
#define SI2_ARGS(
              args ) ()
```

Definition at line 34 of file si2dr_liberty.h.

#### 6.13.1.3 SI2_LONG_MAX

```
#define SI2_LONG_MAX 2147483647
```

Definition at line 74 of file si2dr_liberty.h.

#### 6.13.1.4 SI2_LONG_MIN

```
#define SI2_LONG_MIN -2147483647
```

Definition at line 75 of file si2dr_liberty.h.

#### 6.13.1.5 SI2_ULONG_MAX

```
#define SI2_ULONG_MAX 4294967295
```

Definition at line 76 of file si2dr_liberty.h.

#### 6.13.1.6 SI2DR_FALSE

```
#define SI2DR_FALSE SI2_FALSE
```

Definition at line 96 of file si2dr_liberty.h.

#### 6.13.1.7 SI2DR_MAX_FLOAT32

```
#define SI2DR_MAX_FLOAT32 1E+37
```

Definition at line 87 of file si2dr_liberty.h.

#### 6.13.1.8 SI2DR_MAX_FLOAT64

```
#define SI2DR_MAX_FLOAT64 1.797693e+308
```

Definition at line 89 of file si2dr_liberty.h.

#### 6.13.1.9 SI2DR_MAX_INT32

```
#define SI2DR_MAX_INT32 2147483647
```

Definition at line 91 of file si2dr_liberty.h.

#### 6.13.1.10 SI2DR_MAX_STRING_LEN

```
#define SI2DR_MAX_STRING_LEN 1048576 /*1024*1024*/
```

Definition at line 93 of file si2dr_liberty.h.

#### 6.13.1.11 SI2DR_MIN_FLOAT32

```
#define SI2DR_MIN_FLOAT32 -1E+37
```

Definition at line 86 of file si2dr_liberty.h.

#### 6.13.1.12 SI2DR_MIN_FLOAT64

```
#define SI2DR_MIN_FLOAT64 -1.797693e+308
```

Definition at line 88 of file si2dr_liberty.h.

#### 6.13.1.13 SI2DR_MIN_INT32

```
#define SI2DR_MIN_INT32 -2147483647
```

Definition at line 90 of file si2dr_liberty.h.

#### 6.13.1.14 SI2DR_TRUE

```
#define SI2DR_TRUE SI2_TRUE
```

Definition at line 95 of file si2dr_liberty.h.

### 6.13.2 Typedef Documentation

#### 6.13.2.1 si2BooleanT

```
typedef enum si2BooleanEnum si2BooleanT
```

#### 6.13.2.2 si2DoubleT

```
typedef double si2DoubleT
```

Definition at line 52 of file si2dr_liberty.h.

#### 6.13.2.3 si2drAttrComplexValIdT

```
typedef si2drObjectIdT si2drAttrComplexValIdT
```

Definition at line 111 of file si2dr_liberty.h.

#### 6.13.2.4 si2drAttrIdT

```
typedef si2drObjectIdT si2drAttrIdT
```

Definition at line 110 of file si2dr_liberty.h.

#### 6.13.2.5 si2drAttrsIdT

```
typedef si2drIterIdT si2drAttrsIdT
```

Definition at line 116 of file si2dr_liberty.h.

#### 6.13.2.6 si2drAttrTypeT

```
typedef enum si2drAttrTypeT si2drAttrTypeT
```

#### 6.13.2.7 si2drBooleanT

```
typedef si2BooleanT si2drBooleanT
```

Definition at line 98 of file si2dr_liberty.h.

#### 6.13.2.8 si2drDefineIdT

```
typedef si2drObjectIdT si2drDefineIdT
```

Definition at line 112 of file si2dr_liberty.h.

#### 6.13.2.9 si2drDefinesIdT

```
typedef si2drIterIdT si2drDefinesIdT
```

Definition at line 117 of file si2dr_liberty.h.

#### 6.13.2.10 si2drErrorT

```
typedef enum si2drErrorT si2drErrorT
```

#### 6.13.2.11 si2drExprT

```
typedef struct si2drExprT si2drExprT
```

#### 6.13.2.12 si2drExprTypeT

```
typedef enum si2drExprTypeT si2drExprTypeT
```

#### 6.13.2.13 si2drFloat32T

```
typedef si2FloatT si2drFloat32T
```

Definition at line 101 of file si2dr_liberty.h.

#### 6.13.2.14 si2drFloat64T

```
typedef si2DoubleT si2drFloat64T
```

Definition at line 102 of file si2dr_liberty.h.

#### 6.13.2.15 si2drGroupIdT

```
typedef si2drObjectIdT si2drGroupIdT
```

Definition at line 109 of file si2dr_liberty.h.

#### 6.13.2.16 si2drGroupsIdT

```
typedef si2drIterIdT si2drGroupsIdT
```

Definition at line 115 of file si2dr_liberty.h.

#### 6.13.2.17 si2drInt32T

```
typedef si2LongT si2drInt32T
```

Definition at line 100 of file si2dr_liberty.h.

#### 6.13.2.18 si2drIterIdT

```
typedef si2IteratorIdT si2drIterIdT
```

Definition at line 106 of file si2dr_liberty.h.

#### 6.13.2.19 si2drMessageHandlerT

```
typedef si2drVoidT(* si2drMessageHandlerT) (si2drSeverityT sev, si2drErrorT errToPrint, si2drStringT
auxText, si2drErrorT *err)
```

Definition at line 215 of file si2dr_liberty.h.

#### 6.13.2.20 si2drNamesIdT

```
typedef si2drIterIdT si2drNamesIdT
```

Definition at line 118 of file si2dr_liberty.h.

#### 6.13.2.21 si2drObjectIdT

```
typedef si2ObjectIdT si2drObjectIdT
```

Definition at line 107 of file si2dr_liberty.h.

#### 6.13.2.22 si2drObjectsIdT

```
typedef si2drIterIdT si2drObjectsIdT
```

Definition at line 114 of file si2dr_liberty.h.

#### 6.13.2.23 si2drObjectTypeT

```
typedef enum si2drObjectTypeT si2drObjectTypeT
```

#### 6.13.2.24 si2drSeverityT

```
typedef enum si2drSeverityT si2drSeverityT
```

#### 6.13.2.25 si2drStringT

```
typedef si2StringT si2drStringT
```

Definition at line 103 of file si2dr_liberty.h.

#### 6.13.2.26 si2drValuesIdT

```
typedef si2drIterIdT si2drValuesIdT
```

Definition at line 119 of file si2dr_liberty.h.

#### 6.13.2.27 si2drValueTypeT

```
typedef enum si2drValueTypeT si2drValueTypeT
```

#### 6.13.2.28 si2drVoidT

```
typedef si2VoidT si2drVoidT
```

Definition at line 104 of file si2dr_liberty.h.

#### 6.13.2.29 si2FloatT

```
typedef float si2FloatT
```

Definition at line 51 of file si2dr_liberty.h.

#### 6.13.2.30 si2IteratorIdT

```
typedef void* si2IteratorIdT
```

Definition at line 83 of file si2dr_liberty.h.

#### 6.13.2.31 si2LongT

```
typedef long int si2LongT
```

Definition at line 50 of file si2dr_liberty.h.

#### 6.13.2.32 si2ObjectIdT

```
typedef struct si2OID si2ObjectIdT
```

#### 6.13.2.33 si2StringT

```
typedef char* si2StringT
```

Definition at line 53 of file si2dr_liberty.h.

#### 6.13.2.34 si2TypeT

```
typedef enum si2TypeEnum si2TypeT
```

#### 6.13.2.35 si2VoidT

```
typedef void si2VoidT
```

Definition at line 54 of file si2dr_liberty.h.

### 6.13.3 Enumeration Type Documentation

#### 6.13.3.1 si2BooleanEnum

```
enum si2BooleanEnum
```

**Enumerator**

| | |
|---|---|
| SI2_FALSE | |
| SI2_TRUE | |

Definition at line 49 of file si2dr_liberty.h.

#### 6.13.3.2 si2drAttrTypeT

`enum si2drAttrTypeT`

**Enumerator**

| | |
|---|---|
| SI2DR_SIMPLE | |
| SI2DR_COMPLEX | |

Definition at line 139 of file si2dr_liberty.h.

#### 6.13.3.3 si2drErrorT

`enum si2drErrorT`

**Enumerator**

| | |
|---|---|
| SI2DR_NO_ERROR | |
| SI2DR_INTERNAL_SYSTEM_ERROR | |
| SI2DR_INVALID_VALUE | |
| SI2DR_INVALID_NAME | |
| SI2DR_INVALID_OBJECTTYPE | |
| SI2DR_INVALID_ATTRTYPE | |
| SI2DR_UNUSABLE_OID | |
| SI2DR_OBJECT_ALREADY_EXISTS | |
| SI2DR_OBJECT_NOT_FOUND | |
| SI2DR_SYNTAX_ERROR | |
| SI2DR_TRACE_FILES_CANNOT_BE_OPENED | |
| SI2DR_PIINIT_NOT_CALLED | |
| SI2DR_SEMANTIC_ERROR | |
| SI2DR_REFERENCE_ERROR | |
| SI2DR_MAX_ERROR | |

Definition at line 190 of file si2dr_liberty.h.

#### 6.13.3.4 si2drExprTypeT

```
enum si2drExprTypeT
```

**Enumerator**

| | |
|---|---|
| SI2DR_EXPR_VAL | |
| SI2DR_EXPR_OP_ADD | |
| SI2DR_EXPR_OP_SUB | |
| SI2DR_EXPR_OP_MUL | |
| SI2DR_EXPR_OP_DIV | |
| SI2DR_EXPR_OP_PAREN | |
| SI2DR_EXPR_OP_LOG2 | |
| SI2DR_EXPR_OP_LOG10 | |
| SI2DR_EXPR_OP_EXP | |

Definition at line 157 of file si2dr_liberty.h.

#### 6.13.3.5 si2drObjectTypeT

```
enum si2drObjectTypeT
```

**Enumerator**

| | |
|---|---|
| SI2DR_UNDEFINED_OBJECTTYPE | |
| SI2DR_GROUP | |
| SI2DR_ATTR | |
| SI2DR_DEFINE | |
| SI2DR_COMPLEXVAL | |
| SI2DR_MAX_OBJECTTYPE | |

Definition at line 129 of file si2dr_liberty.h.

#### 6.13.3.6 si2drSeverityT

```
enum si2drSeverityT
```

**Enumerator**

| | |
|---|---|
| SI2DR_SEVERITY_NOTE | |
| SI2DR_SEVERITY_WARN | |
| SI2DR_SEVERITY_ERR | |

Definition at line 121 of file si2dr_liberty.h.

#### 6.13.3.7 si2drValueTypeT

```
enum si2drValueTypeT
```

**Enumerator**

| | |
|---|---|
| SI2DR_UNDEFINED_VALUETYPE | |
| SI2DR_INT32 | |
| SI2DR_STRING | |
| SI2DR_FLOAT64 | |
| SI2DR_BOOLEAN | |
| SI2DR_EXPR | |
| SI2DR_MAX_VALUETYPE | |

Definition at line 146 of file si2dr_liberty.h.

#### 6.13.3.8 si2TypeEnum

```
enum si2TypeEnum
```

**Enumerator**

| | |
|---|---|
| SI2_UNDEFINED_VALUETYPE | |
| SI2_BOOLEAN | |
| SI2_LONG | |
| SI2_FLOAT | |
| SI2_DOUBLE | |
| SI2_STRING | |
| SI2_VOID | |
| SI2_OID | |
| SI2_ITER | |
| SI2_EXPR | |
| SI2_MAX_VALUETYPE | |

Definition at line 56 of file si2dr_liberty.h.

### 6.13.4 Function Documentation

#### 6.13.4.1 get_function()

```
void get_function (
            char * filename )
```

#### 6.13.4.2 liberty_destroy_value_data()

```
void liberty_destroy_value_data (
            struct liberty_value_data * vd )
```

#### 6.13.4.3 liberty_get_element()

```
LONG_DOUBLE liberty_get_element (
            struct liberty_value_data * vd,
             ...  )
```

#### 6.13.4.4 liberty_get_values_data()

```
struct liberty_value_data * liberty_get_values_data (
            si2drGroupIdT table_group )
```

#### 6.13.4.5 main_liberty_parser()

```
int main_liberty_parser (
            int argc,
            char * argv[] )
```

#### 6.13.4.6 print_version()

```
void print_version ( )
```

#### 6.13.4.7 SI2_ARGS() [1/41]

```
si2drVoidT si2drReadLibertyFile SI2_ARGS (
            (char *filename, si2drErrorT *err)  )
```

#### 6.13.4.8 SI2_ARGS() [2/41]

```
si2drVoidT si2drWriteLibertyFile SI2_ARGS (
            (char *filename, si2drGroupIdT group, char *cellname, si2drErrorT *err)  )
```

#### 6.13.4.9 SI2_ARGS() [3/41]

```
si2drVoidT si2drComplexAttrAddBooleanValue SI2_ARGS (
             (si2drAttrIdT attr, si2drBooleanT boolval, si2drErrorT *err)  )
```

#### 6.13.4.10 SI2_ARGS() [4/41]

```
si2drVoidT si2drSimpleAttrSetBooleanValue SI2_ARGS (
             (si2drAttrIdT attr, si2drBooleanT intgr, si2drErrorT *err)  )
```

#### 6.13.4.11 SI2_ARGS() [5/41]

```
si2drStringT si2drAttrGetComment SI2_ARGS (
             (si2drAttrIdT attr, si2drErrorT *err)  )
```

#### 6.13.4.12 SI2_ARGS() [6/41]

```
si2drVoidT si2drSimpleAttrSetExprValue SI2_ARGS (
             (si2drAttrIdT attr, si2drExprT *expr, si2drErrorT *err)  )
```

#### 6.13.4.13 SI2_ARGS() [7/41]

```
si2drVoidT si2drSimpleAttrSetFloat64Value SI2_ARGS (
             (si2drAttrIdT attr, si2drFloat64T float64, si2drErrorT *err)  )
```

#### 6.13.4.14 SI2_ARGS() [8/41]

```
si2drVoidT si2drSimpleAttrSetInt32Value SI2_ARGS (
             (si2drAttrIdT attr, si2drInt32T intgr, si2drErrorT *err)  )
```

#### 6.13.4.15 SI2_ARGS() [9/41]

```
si2drVoidT si2drAttrSetComment SI2_ARGS (
             (si2drAttrIdT attr, si2drStringT comment, si2drErrorT *err)  )
```

#### 6.13.4.16 SI2_ARGS() [10/41]

```
si2drVoidT si2drSimpleAttrSetStringValue SI2_ARGS (
            (si2drAttrIdT attr, si2drStringT string, si2drErrorT *err)  )
```

#### 6.13.4.17 SI2_ARGS() [11/41]

```
si2drExprT *si2drCreateBooleanValExpr SI2_ARGS (
            (si2drBooleanT b, si2drErrorT *err)  )
```

#### 6.13.4.18 SI2_ARGS() [12/41]

```
si2drStringT si2drDefineGetComment SI2_ARGS (
            (si2drDefineIdT def, si2drErrorT *err)  )
```

#### 6.13.4.19 SI2_ARGS() [13/41]

```
si2drVoidT si2drDefineGetInfo SI2_ARGS (
            (si2drDefineIdT def, si2drStringT *name, si2drStringT *allowed_group_name, si2drValueTypeT
*valtype, si2drErrorT *err)  )
```

#### 6.13.4.20 SI2_ARGS() [14/41]

```
si2drVoidT si2drDefineSetComment SI2_ARGS (
            (si2drDefineIdT def, si2drStringT comment, si2drErrorT *err)  )
```

#### 6.13.4.21 SI2_ARGS() [15/41]

```
si2drBooleanT si2drPIGetNocheckMode SI2_ARGS (
            (si2drErrorT *err)  )
```

#### 6.13.4.22 SI2_ARGS() [16/41]

```
si2drStringT si2drPIGetErrorText SI2_ARGS (
            (si2drErrorT errorCode, si2drErrorT *err)  )
```

#### 6.13.4.23 SI2_ARGS() [17/41]

```
si2drExprT *si2drOpExprGetRightExpr SI2_ARGS (
             (si2drExprT *expr, si2drErrorT *err)  )
```

#### 6.13.4.24 SI2_ARGS() [18/41]

```
si2drExprT *si2drCreateBinaryOpExpr SI2_ARGS (
             (si2drExprT *left, si2drExprTypeT optype, si2drExprT *right, si2drErrorT *err)  )
```

#### 6.13.4.25 SI2_ARGS() [19/41]

```
si2drExprT *si2drCreateUnaryOpExpr SI2_ARGS (
             (si2drExprTypeT optype, si2drExprT *expr, si2drErrorT *err)  )
```

#### 6.13.4.26 SI2_ARGS() [20/41]

```
si2drExprT *si2drCreateExpr SI2_ARGS (
             (si2drExprTypeT type, si2drErrorT *err)  )
```

#### 6.13.4.27 SI2_ARGS() [21/41]

```
si2drExprT *si2drCreateDoubleValExpr SI2_ARGS (
             (si2drFloat64T d, si2drErrorT *err)  )
```

#### 6.13.4.28 SI2_ARGS() [22/41]

```
si2drVoidT si2drCheckLibertyLibrary SI2_ARGS (
             (si2drGroupIdT group, si2drErrorT *err)  )
```

#### 6.13.4.29 SI2_ARGS() [23/41]

```
si2drVoidT si2drGroupSetComment SI2_ARGS (
             (si2drGroupIdT group, si2drStringT comment, si2drErrorT *err)  )
```

#### 6.13.4.30 SI2_ARGS() [24/41]

```
si2drAttrIdT si2drGroupCreateAttr SI2_ARGS (
            (si2drGroupIdT group, si2drStringT name, si2drAttrTypeT type, si2drErrorT *err)  )
```

#### 6.13.4.31 SI2_ARGS() [25/41]

```
si2drDefineIdT si2drGroupFindDefineByName SI2_ARGS (
            (si2drGroupIdT group, si2drStringT name, si2drErrorT *err)  )
```

#### 6.13.4.32 SI2_ARGS() [26/41]

```
si2drDefineIdT si2drGroupCreateDefine SI2_ARGS (
            (si2drGroupIdT group, si2drStringT name, si2drStringT allowed_group_name, si2drValueTypeT
valtype, si2drErrorT *err)  )
```

#### 6.13.4.33 SI2_ARGS() [27/41]

```
si2drGroupIdT si2drGroupCreateGroup SI2_ARGS (
            (si2drGroupIdT group, si2drStringT name, si2drStringT group_type, si2drErrorT
*err)  )
```

#### 6.13.4.34 SI2_ARGS() [28/41]

```
si2drGroupIdT si2drGroupFindGroupByName SI2_ARGS (
            (si2drGroupIdT group, si2drStringT name, si2drStringT type, si2drErrorT *err)  )
```

#### 6.13.4.35 SI2_ARGS() [29/41]

```
si2drVoidT si2drGroupMoveBefore SI2_ARGS (
            (si2drGroupIdT groupToMove, si2drGroupIdT targetGroup, si2drErrorT *err)  )
```

#### 6.13.4.36 SI2_ARGS() [30/41]

```
si2drExprT *si2drCreateIntValExpr SI2_ARGS (
            (si2drInt32T i, si2drErrorT *err)  )
```

#### 6.13.4.37 SI2_ARGS() [31/41]

```
si2drStringT si2drObjectGetFileName SI2_ARGS (
             (si2drObjectIdT object, si2drErrorT *err)  )
```

#### 6.13.4.38 SI2_ARGS() [32/41]

```
si2drVoidT si2drObjectSetLineNo SI2_ARGS (
             (si2drObjectIdT object, si2drInt32T lineno, si2drErrorT *err)  )
```

#### 6.13.4.39 SI2_ARGS() [33/41]

```
si2drVoidT si2drObjectSetFileName SI2_ARGS (
             (si2drObjectIdT object, si2drStringT filename, si2drErrorT *err)  )
```

#### 6.13.4.40 SI2_ARGS() [34/41]

```
si2drBooleanT si2drObjectIsSame SI2_ARGS (
             (si2drObjectIdT object1, si2drObjectIdT object2, si2drErrorT *err)  )
```

#### 6.13.4.41 SI2_ARGS() [35/41]

```
si2drVoidT si2drPISetTraceMode SI2_ARGS (
             (si2drStringT fname, si2drErrorT *err)  )
```

#### 6.13.4.42 SI2_ARGS() [36/41]

```
si2drDefineIdT si2drPIFindDefineByName SI2_ARGS (
             (si2drStringT name, si2drErrorT *err)  )
```

#### 6.13.4.43 SI2_ARGS() [37/41]

```
si2drGroupIdT si2drPICreateGroup SI2_ARGS (
             (si2drStringT name, si2drStringT group_type, si2drErrorT *err)  )
```

#### 6.13.4.44 SI2_ARGS() [38/41]

```
si2drGroupIdT si2drPIFindGroupByName SI2_ARGS (
             (si2drStringT name, si2drStringT type, si2drErrorT *err)  )
```

#### 6.13.4.45 SI2_ARGS() [39/41]

```
si2drExprT *si2drCreateStringValExpr SI2_ARGS (
             (si2drStringT s, si2drErrorT *err)  )
```

#### 6.13.4.46 SI2_ARGS() [40/41]

```
si2drVoidT si2drIterQuit SI2_ARGS (
             (si2drValuesIdT iter, si2drErrorT *err)  )
```

#### 6.13.4.47 SI2_ARGS() [41/41]

```
si2drVoidT si2drIterNextComplexValue SI2_ARGS (
             (si2drValuesIdT iter, si2drValueTypeT *type, si2drInt32T *intgr, si2drFloat64T
*float64, si2drStringT *string, si2drBooleanT *boolval, si2drExprT **expr, si2drErrorT *err)  )
```

#### 6.13.4.48 si2drDefaultMessageHandler()

```
si2drVoidT si2drDefaultMessageHandler (
             si2drSeverityT sev,
             si2drErrorT errToPrint,
             si2drStringT auxText,
             si2drErrorT * err )
```

#### 6.13.4.49 si2drPIGetMessageHandler()

```
si2drMessageHandlerT si2drPIGetMessageHandler (
             si2drErrorT * err )
```

#### 6.13.4.50 si2drPISetMessageHandler()

```
si2drVoidT si2drPISetMessageHandler (
             si2drMessageHandlerT MsgHandFuncPtr,
             si2drErrorT * err )
```

#### 6.13.4.51 usage()

```
void usage ( )
```

## 6.14 si2dr_liberty.h

Go to the documentation of this file.

```
00001 /*****************************************************************************
00002     Copyright (c) 1996-2005 Synopsys, Inc.    ALL RIGHTS RESERVED
00003
00004   The contents of this file are subject to the restrictions and limitations
00005   set forth in the SYNOPSYS Open Source License Version 1.0  (the "License");
00006   you may not use this file except in compliance with such restrictions
00007   and limitations. You may obtain instructions on how to receive a copy of
00008   the License at
00009
00010   http://www.synopsys.com/partners/tapin/tapinprogram.html.
00011
00012   Software distributed by Original Contributor under the License is
00013   distributed on an "AS IS" basis, WITHOUT WARRANTY OF ANY KIND, either
00014   expressed or implied. See the License for the specific language governing
00015   rights and limitations under the License.
00016
00017 *****************************************************************************/
00018
00019 /* define a PI for manipulating the liberty data */
00020
00021
00022 /* basic data types */
00023
00024
00025 #ifndef SI2DR_H
00026 #define SI2DR_H
00027
00028 #ifdef SI2_ARGS
00029 #undef SI2_ARGS
00030 #endif
00031 #if defined(__STDC__) || defined(HPPA) || defined(ibmpoweraix) || defined(_MSC_VER) ||
     defined(__cplusplus)
00032 #define SI2_ARGS(args)  args
00033 #else
00034 #define SI2_ARGS(args)  ()
00035 #endif
00036
00037
00038 /* ************************************************************************
00039  File contents : LIBERTY-DR type definitions.
00040 ************************************************************************** */
00041 #ifdef __cplusplus
00042 extern "C" {
00043 #endif
00044
00045 /*
00046  * These primitive types are compatible with the ANSI C Standard (ANSI/
00047  * X3.159-1990 Programming Language C Standard.)
00048  */
00049 typedef enum si2BooleanEnum { SI2_FALSE = 0, SI2_TRUE = 1 } si2BooleanT;
00050 typedef long int  si2LongT;
00051 typedef float     si2FloatT;
00052 typedef double    si2DoubleT;
00053 typedef char*     si2StringT;
00054 typedef void      si2VoidT;
00055
00056 typedef enum si2TypeEnum {
00057    SI2_UNDEFINED_VALUETYPE = 0,
```

```
00058    SI2_BOOLEAN = 1,
00059    SI2_LONG = 2,
00060    SI2_FLOAT = 3,
00061    SI2_DOUBLE = 4,
00062    SI2_STRING = 5,
00063    SI2_VOID = 6,
00064    SI2_OID = 7,
00065    SI2_ITER = 8,
00066    SI2_EXPR = 9,
00067    SI2_MAX_VALUETYPE = 10
00068    } si2TypeT;
00069
00070 /*
00071  * Size limits
00072  */
00073
00074 #define SI2_LONG_MAX   2147483647
00075 #define SI2_LONG_MIN  -2147483647
00076 #define SI2_ULONG_MAX  4294967295
00077
00078
00079 typedef struct si2OID  {void *v1, *v2;} si2ObjectIdT;
00080 /*
00081  * Iterators are blind handles the size of a single pointer.
00082  */
00083 typedef void *si2IteratorIdT;
00084
00085
00086 #define SI2DR_MIN_FLOAT32  -1E+37
00087 #define SI2DR_MAX_FLOAT32   1E+37
00088 #define SI2DR_MIN_FLOAT64  -1.797693e+308
00089 #define SI2DR_MAX_FLOAT64   1.797693e+308
00090 #define SI2DR_MIN_INT32    -2147483647
00091 #define SI2DR_MAX_INT32     2147483647
00092
00093 #define SI2DR_MAX_STRING_LEN  1048576 /*1024*1024*/
00094
00095 #define SI2DR_TRUE SI2_TRUE
00096 #define SI2DR_FALSE SI2_FALSE
00097
00098 typedef si2BooleanT si2drBooleanT;
00099
00100 typedef si2LongT    si2drInt32T;
00101 typedef si2FloatT   si2drFloat32T;
00102 typedef si2DoubleT   si2drFloat64T;
00103 typedef si2StringT  si2drStringT;
00104 typedef si2VoidT    si2drVoidT;
00105
00106 typedef si2IteratorIdT    si2drIterIdT;
00107 typedef si2ObjectIdT    si2drObjectIdT;
00108
00109 typedef si2drObjectIdT si2drGroupIdT;
00110 typedef si2drObjectIdT si2drAttrIdT;
00111 typedef si2drObjectIdT si2drAttrComplexValIdT;
00112 typedef si2drObjectIdT si2drDefineIdT;
00113
00114 typedef si2drIterIdT si2drObjectsIdT;
00115 typedef si2drIterIdT si2drGroupsIdT;
00116 typedef si2drIterIdT si2drAttrsIdT;
00117 typedef si2drIterIdT si2drDefinesIdT;
00118 typedef si2drIterIdT si2drNamesIdT;
00119 typedef si2drIterIdT si2drValuesIdT;
00120
00121 typedef enum si2drSeverityT
00122 {
00123     SI2DR_SEVERITY_NOTE = 0,
00124     SI2DR_SEVERITY_WARN = 1,
00125     SI2DR_SEVERITY_ERR  = 2
00126 } si2drSeverityT;
00127
00128
00129 typedef enum si2drObjectTypeT
00130 {
00131     SI2DR_UNDEFINED_OBJECTTYPE =  0,
00132     SI2DR_GROUP                =  1,
00133     SI2DR_ATTR                 =  2,
00134     SI2DR_DEFINE               =  3,
00135     SI2DR_COMPLEXVAL           =  4,
00136     SI2DR_MAX_OBJECTTYPE       =  5
00137 } si2drObjectTypeT;
00138
00139 typedef enum si2drAttrTypeT
00140 {
00141     SI2DR_SIMPLE,
00142     SI2DR_COMPLEX
00143 } si2drAttrTypeT;
00144
```

```
00145
00146 typedef enum si2drValueTypeT
00147 {
00148     SI2DR_UNDEFINED_VALUETYPE = SI2_UNDEFINED_VALUETYPE,
00149     SI2DR_INT32               = SI2_LONG,
00150     SI2DR_STRING              = SI2_STRING,
00151     SI2DR_FLOAT64             = SI2_DOUBLE,
00152     SI2DR_BOOLEAN             = SI2_BOOLEAN,
00153     SI2DR_EXPR                = SI2_EXPR,
00154     SI2DR_MAX_VALUETYPE       = SI2_MAX_VALUETYPE
00155 } si2drValueTypeT;
00156
00157 typedef enum si2drExprTypeT
00158 {
00159     SI2DR_EXPR_VAL = 0,     /* unary ops just have left arg set: add, sub, logs, paren */
00160     SI2DR_EXPR_OP_ADD =  1,
00161     SI2DR_EXPR_OP_SUB =  2,
00162     SI2DR_EXPR_OP_MUL = 3,
00163     SI2DR_EXPR_OP_DIV =  4,
00164     SI2DR_EXPR_OP_PAREN =  5,
00165     SI2DR_EXPR_OP_LOG2 =  6, /* pretty much flight of fancy from here on? */
00166     SI2DR_EXPR_OP_LOG10 =  7,
00167     SI2DR_EXPR_OP_EXP  = 8, /* exponent? */
00168
00169 } si2drExprTypeT;
00170
00171
00172 typedef struct si2drExprT
00173 {
00174     si2drExprTypeT type;
00175     union
00176     {
00177         si2drInt32T i;
00178         si2drFloat64T d;
00179         si2drStringT s;    /* most likely an identifier */
00180         si2drBooleanT b;
00181     } u;
00182
00183     si2drValueTypeT  valuetype; /* if the type is a fixed value */
00184
00185     struct si2drExprT *left; /* the exprs form a classic binary tree rep of an arithmetic expression
      */
00186     struct si2drExprT *right;
00187 } si2drExprT;
00188
00189
00190 typedef enum si2drErrorT
00191 {
00192     SI2DR_NO_ERROR                              = 0,
00193     SI2DR_INTERNAL_SYSTEM_ERROR                 = 1,
00194     SI2DR_INVALID_VALUE                         = 4,
00195     SI2DR_INVALID_NAME                          = 5,
00196     SI2DR_INVALID_OBJECTTYPE                    = 6,
00197     SI2DR_INVALID_ATTRTYPE                      = 10,
00198     SI2DR_UNUSABLE_OID                          = 7,
00199     SI2DR_OBJECT_ALREADY_EXISTS                 = 8,
00200     SI2DR_OBJECT_NOT_FOUND                      = 9,
00201     SI2DR_SYNTAX_ERROR                          = 2,
00202     SI2DR_TRACE_FILES_CANNOT_BE_OPENED          = 3,
00203     SI2DR_PIINIT_NOT_CALLED                     = 11,
00204     SI2DR_SEMANTIC_ERROR                        = 12,
00205     SI2DR_REFERENCE_ERROR                       = 13,
00206     SI2DR_MAX_ERROR                             = 14
00207 } si2drErrorT;
00208
00209 si2drVoidT  si2drDefaultMessageHandler(si2drSeverityT sev,
00210                                        si2drErrorT errToPrint,
00211                                        si2drStringT auxText,
00212                                        si2drErrorT *err);
00213
00214
00215     typedef si2drVoidT (*si2drMessageHandlerT)(si2drSeverityT sev,
00216                                                si2drErrorT errToPrint,
00217                                                si2drStringT auxText,
00218                                                si2drErrorT *err);
00219
00220
00221 si2drMessageHandlerT  si2drPIGetMessageHandler( si2drErrorT *err);
00222
00223
00224 si2drVoidT  si2drPISetMessageHandler( si2drMessageHandlerT MsgHandFuncPtr,
00225                                       si2drErrorT *err);
00226
00227
00228     si2drGroupIdT  si2drPICreateGroup     SI2_ARGS(( si2drStringT name,
00229                                                      si2drStringT group_type,
00230                                                      si2drErrorT  *err));
```

```
00231
00232
00233     si2drAttrIdT   si2drGroupCreateAttr   SI2_ARGS(( si2drGroupIdT  group,
00234                                                      si2drStringT   name,
00235                                                      si2drAttrTypeT type,
00236                                                      si2drErrorT   *err));
00237
00238     si2drAttrTypeT si2drAttrGetAttrType   SI2_ARGS(( si2drAttrIdT  attr,
00239                                                      si2drErrorT   *err));
00240
00241     si2drStringT   si2drAttrGetName       SI2_ARGS(( si2drAttrIdT  attr,
00242                                                      si2drErrorT   *err));
00243
00244     si2drVoidT     si2drComplexAttrAddInt32Value        SI2_ARGS(( si2drAttrIdT attr,
00245                                                                    si2drInt32T     intgr,
00246                                                                    si2drErrorT     *err ));
00247
00248     si2drVoidT     si2drComplexAttrAddStringValue       SI2_ARGS(( si2drAttrIdT attr,
00249                                                                     si2drStringT    string,
00250                                                                     si2drErrorT     *err ));
00251
00252     si2drVoidT     si2drComplexAttrAddBooleanValue      SI2_ARGS(( si2drAttrIdT attr,
00253                                                                      si2drBooleanT   boolval,
00254                                                                      si2drErrorT     *err ));
00255
00256     si2drVoidT     si2drComplexAttrAddFloat64Value      SI2_ARGS(( si2drAttrIdT attr,
00257                                                                      si2drFloat64T   float64,
00258                                                                      si2drErrorT     *err ));
00259
00260     si2drVoidT     si2drComplexAttrAddExprValue         SI2_ARGS(( si2drAttrIdT attr,
00261                                                                    si2drExprT    *expr,
00262                                                                    si2drErrorT     *err ));
00263
00264     si2drValuesIdT si2drComplexAttrGetValues         SI2_ARGS(( si2drAttrIdT     attr,
00265                                                                  si2drErrorT     *err ));
00266
00267     si2drVoidT     si2drIterNextComplexValue         SI2_ARGS(( si2drValuesIdT iter,
00268                                                                  si2drValueTypeT *type,
00269                                                                  si2drInt32T     *intgr,
00270                                                                  si2drFloat64T   *float64,
00271                                                                  si2drStringT    *string,
00272                                                                  si2drBooleanT   *boolval,
00273                                                                  si2drExprT      **expr,
00274                                                                  si2drErrorT     *err ));
00275
00276     si2drAttrComplexValIdT si2drIterNextComplex      SI2_ARGS(( si2drValuesIdT iter,
00277                                                                  si2drErrorT     *err ));
00278
00279     si2drValueTypeT  si2drComplexValGetValueType        SI2_ARGS(( si2drAttrComplexValIdT attr,
00280                                                                    si2drErrorT     *err ));
00281
00282     si2drInt32T      si2drComplexValGetInt32Value       SI2_ARGS(( si2drAttrComplexValIdT attr,
00283                                                                    si2drErrorT     *err ));
00284
00285     si2drFloat64T    si2drComplexValGetFloat64Value     SI2_ARGS(( si2drAttrComplexValIdT attr,
00286                                                                    si2drErrorT     *err ));
00287
00288     si2drStringT     si2drComplexValGetStringValue      SI2_ARGS(( si2drAttrComplexValIdT attr,
00289                                                                    si2drErrorT     *err ));
00290
00291     si2drBooleanT    si2drComplexValGetBooleanValue     SI2_ARGS(( si2drAttrComplexValIdT attr,
00292                                                                    si2drErrorT     *err ));
00293
00294     si2drExprT      *si2drComplexValGetExprValue        SI2_ARGS(( si2drAttrComplexValIdT attr,
00295                                                                    si2drErrorT     *err ));
00296
00297
00298
00299
00300
00301
00302     si2drValueTypeT  si2drSimpleAttrGetValueType        SI2_ARGS(( si2drAttrIdT attr,
00303                                                                    si2drErrorT     *err ));
00304
00305     si2drInt32T      si2drSimpleAttrGetInt32Value       SI2_ARGS(( si2drAttrIdT attr,
00306                                                                    si2drErrorT     *err ));
00307
00308     si2drFloat64T    si2drSimpleAttrGetFloat64Value     SI2_ARGS(( si2drAttrIdT attr,
00309                                                                    si2drErrorT     *err ));
00310
00311     si2drStringT     si2drSimpleAttrGetStringValue      SI2_ARGS(( si2drAttrIdT attr,
00312                                                                    si2drErrorT     *err ));
00313
00314     si2drBooleanT    si2drSimpleAttrGetBooleanValue     SI2_ARGS(( si2drAttrIdT attr,
00315                                                                    si2drErrorT     *err ));
00316
00317     si2drExprT      *si2drSimpleAttrGetExprValue        SI2_ARGS(( si2drAttrIdT attr,
```

```
00318                                                                   si2drErrorT     *err ));
00319
00320
00321
00322     si2drVoidT       si2drSimpleAttrSetInt32Value       SI2_ARGS(( si2drAttrIdT attr,
00323                                                                     si2drInt32T     intgr,
00324                                                                     si2drErrorT     *err ));
00325
00326     si2drVoidT       si2drSimpleAttrSetBooleanValue     SI2_ARGS(( si2drAttrIdT   attr,
00327                                                                     si2drBooleanT   intgr,
00328                                                                     si2drErrorT      *err ));
00329
00330     si2drVoidT       si2drSimpleAttrSetFloat64Value     SI2_ARGS(( si2drAttrIdT attr,
00331                                                                     si2drFloat64T   float64,
00332                                                                     si2drErrorT     *err ));
00333
00334     si2drVoidT       si2drSimpleAttrSetStringValue      SI2_ARGS(( si2drAttrIdT attr,
00335                                                                     si2drStringT    string,
00336                                                                     si2drErrorT     *err ));
00337
00338     si2drVoidT       si2drSimpleAttrSetExprValue        SI2_ARGS(( si2drAttrIdT   attr,
00339                                                                     si2drExprT    *expr,
00340                                                                     si2drErrorT   *err ));
00341
00342     si2drBooleanT    si2drSimpleAttrGetIsVar     SI2_ARGS(( si2drAttrIdT attr,
00343                                                                 si2drErrorT     *err ));
00344
00345     si2drVoidT       si2drSimpleAttrSetIsVar     SI2_ARGS(( si2drAttrIdT   attr,
00346                                                                 si2drErrorT      *err ));
00347
00348 /* helper functions for expr creation, building, and destruction */
00349
00350     si2drVoidT       si2drExprDestroy                   SI2_ARGS(( si2drExprT   *expr,  /* recursively
    free the structures */
00351                                                                     si2drErrorT  *err));
00352
00353     si2drExprT      *si2drCreateExpr                    SI2_ARGS(( si2drExprTypeT type, /* malloc an
    Expr and return it */
00354                                                                     si2drErrorT  *err));
00355
00356 si2drExprT  *si2drCreateBooleanValExpr    SI2_ARGS(( si2drBooleanT b,
00357                                                       si2drErrorT  *err ));
00358
00359 si2drExprT  *si2drCreateDoubleValExpr    SI2_ARGS(( si2drFloat64T d,
00360                                                     si2drErrorT  *err ));
00361
00362 si2drExprT  *si2drCreateStringValExpr    SI2_ARGS(( si2drStringT s,
00363                                                     si2drErrorT  *err ));
00364
00365 si2drExprT  *si2drCreateIntValExpr    SI2_ARGS(( si2drInt32T i,
00366                                                  si2drErrorT  *err ));
00367
00368 si2drExprT  *si2drCreateBinaryOpExpr    SI2_ARGS(( si2drExprT *left,
00369                                                    si2drExprTypeT optype,
00370                                                    si2drExprT *right,
00371                                                    si2drErrorT  *err ));
00372
00373 si2drExprT  *si2drCreateUnaryOpExpr    SI2_ARGS(( si2drExprTypeT optype,
00374                                                   si2drExprT *expr,
00375                                                   si2drErrorT  *err ));
00376
00377 si2drStringT    si2drExprToString           SI2_ARGS(( si2drExprT  *expr,
00378                                                         si2drErrorT  *err ));
00379
00380 si2drExprTypeT si2drExprGetType SI2_ARGS(( si2drExprT  *expr,
00381                                           si2drErrorT  *err ));
00382
00383 si2drValueTypeT si2drValExprGetValueType  SI2_ARGS(( si2drExprT  *expr,
00384                                                     si2drErrorT  *err ));
00385
00386 si2drInt32T    si2drIntValExprGetInt SI2_ARGS(( si2drExprT  *expr,
00387                                                 si2drErrorT  *err ));
00388
00389 si2drFloat64T    si2drDoubleValExprGetDouble SI2_ARGS(( si2drExprT  *expr,
00390                                                       si2drErrorT  *err ));
00391
00392 si2drBooleanT      si2drBooleanValExprGetBoolean SI2_ARGS(( si2drExprT  *expr,
00393                                                             si2drErrorT  *err ));
00394
00395 si2drStringT      si2drStringValExprGetString SI2_ARGS(( si2drExprT  *expr,
00396                                                          si2drErrorT  *err ));
00397
00398 si2drExprT   *si2drOpExprGetLeftExpr SI2_ARGS(( si2drExprT  *expr,  /* will apply to Unary & binary
    Ops */
00399                                                 si2drErrorT  *err ));
00400
00401 si2drExprT   *si2drOpExprGetRightExpr SI2_ARGS(( si2drExprT  *expr,
```

```
00402                                                 si2drErrorT  *err ));
00403
00404 /* Define related funcs */
00405
00406     si2drDefineIdT si2drGroupCreateDefine SI2_ARGS(( si2drGroupIdT group,
00407                                                      si2drStringT name,
00408                                                      si2drStringT allowed_group_name,
00409                                                      si2drValueTypeT valtype,
00410                                                      si2drErrorT  *err));
00411
00412     si2drVoidT     si2drDefineGetInfo     SI2_ARGS(( si2drDefineIdT def,
00413                                                      si2drStringT  *name,
00414                                                      si2drStringT  *allowed_group_name,
00415                                                      si2drValueTypeT *valtype,
00416                                                      si2drErrorT  *err));
00417
00418     si2drStringT   si2drDefineGetName     SI2_ARGS(( si2drDefineIdT def,
00419                                                      si2drErrorT  *err));
00420
00421     si2drStringT   si2drDefineGetAllowedGroupName     SI2_ARGS(( si2drDefineIdT def,
00422                                                                  si2drErrorT  *err));
00423
00424     si2drValueTypeT   si2drDefineGetValueType     SI2_ARGS(( si2drDefineIdT def,
00425                                                              si2drErrorT  *err));
00426
00427     si2drGroupIdT  si2drGroupCreateGroup  SI2_ARGS(( si2drGroupIdT group,
00428                                                      si2drStringT name,
00429                                                      si2drStringT group_type,
00430                                                      si2drErrorT  *err));
00431
00432     si2drStringT   si2drGroupGetGroupType SI2_ARGS(( si2drGroupIdT group,
00433                                                      si2drErrorT  *err));
00434
00435     si2drStringT   si2drGroupGetComment   SI2_ARGS(( si2drGroupIdT group,
00436                                                      si2drErrorT  *err));
00437
00438     si2drVoidT     si2drGroupSetComment   SI2_ARGS(( si2drGroupIdT group,
00439                                                      si2drStringT comment,
00440                                                      si2drErrorT  *err));
00441
00442     si2drStringT   si2drAttrGetComment      SI2_ARGS(( si2drAttrIdT attr,
00443                                                        si2drErrorT  *err));
00444
00445     si2drVoidT     si2drAttrSetComment      SI2_ARGS(( si2drAttrIdT attr,
00446                                                        si2drStringT comment,
00447                                                        si2drErrorT  *err));
00448
00449     si2drStringT   si2drDefineGetComment      SI2_ARGS(( si2drDefineIdT def,
00450                                                          si2drErrorT  *err));
00451
00452     si2drVoidT     si2drDefineSetComment      SI2_ARGS(( si2drDefineIdT def,
00453                                                          si2drStringT comment,
00454                                                          si2drErrorT  *err));
00455
00456     si2drVoidT     si2drGroupAddName      SI2_ARGS(( si2drGroupIdT group,
00457                                                      si2drStringT name,
00458                                                      si2drErrorT  *err));
00459
00460     si2drVoidT     si2drGroupDeleteName   SI2_ARGS(( si2drGroupIdT group,
00461                                                      si2drStringT name,
00462                                                      si2drErrorT  *err));
00463
00464
00465     si2drGroupIdT  si2drPIFindGroupByName     SI2_ARGS(( si2drStringT name,
00466                                                          si2drStringT type,
00467                                                          si2drErrorT  *err));
00468
00469     si2drGroupIdT  si2drGroupFindGroupByName  SI2_ARGS(( si2drGroupIdT group,
00470                                                          si2drStringT name,
00471                                                          si2drStringT type,
00472                                                          si2drErrorT  *err));
00473
00474     si2drAttrIdT   si2drGroupFindAttrByName   SI2_ARGS(( si2drGroupIdT group,
00475                                                          si2drStringT name,
00476                                                          si2drErrorT  *err));
00477
00478     si2drDefineIdT si2drGroupFindDefineByName SI2_ARGS(( si2drGroupIdT group,
00479                                                          si2drStringT name,
00480                                                          si2drErrorT  *err));
00481
00482
00483     si2drDefineIdT si2drPIFindDefineByName SI2_ARGS(( si2drStringT name,
00484                                                       si2drErrorT  *err));
00485
00486
00487
00488     si2drGroupsIdT  si2drPIGetGroups     SI2_ARGS(( si2drErrorT  *err));
```

```
00489
00490
00491     si2drGroupsIdT  si2drGroupGetGroups  SI2_ARGS(( si2drGroupIdT group,
00492                                                      si2drErrorT  *err));
00493
00494     si2drNamesIdT   si2drGroupGetNames   SI2_ARGS(( si2drGroupIdT group,
00495                                                      si2drErrorT  *err));
00496
00497     si2drAttrsIdT   si2drGroupGetAttrs   SI2_ARGS(( si2drGroupIdT group,
00498                                                      si2drErrorT  *err));
00499
00500     si2drDefinesIdT si2drGroupGetDefines SI2_ARGS(( si2drGroupIdT group,
00501                                                      si2drErrorT  *err));
00502
00503     si2drGroupIdT   si2drIterNextGroup  SI2_ARGS(( si2drGroupsIdT iter,
00504                                                    si2drErrorT  *err));
00505     si2drStringT    si2drIterNextName   SI2_ARGS(( si2drNamesIdT iter,
00506                                                    si2drErrorT  *err));
00507     si2drAttrIdT    si2drIterNextAttr   SI2_ARGS(( si2drAttrsIdT iter,
00508                                                    si2drErrorT  *err));
00509     si2drDefineIdT  si2drIterNextDefine SI2_ARGS(( si2drDefinesIdT iter,
00510                                                    si2drErrorT  *err));
00511     si2drVoidT      si2drIterQuit       SI2_ARGS(( si2drIterIdT iter,
00512                                                    si2drErrorT  *err));
00513
00514
00515
00516     si2drVoidT      si2drObjectDelete   SI2_ARGS(( si2drObjectIdT object,
00517                                                    si2drErrorT  *err));
00518
00519
00520
00521     si2drStringT     si2drPIGetErrorText          SI2_ARGS(( si2drErrorT errorCode,
00522                                                              si2drErrorT  *err));
00523
00524     si2drObjectIdT   si2drPIGetNullId             SI2_ARGS(( si2drErrorT  *err));
00525
00526     si2drVoidT       si2drPIInit                  SI2_ARGS(( si2drErrorT  *err));
00527
00528     si2drVoidT       si2drPIQuit                  SI2_ARGS(( si2drErrorT  *err));
00529
00530     si2drObjectTypeT si2drObjectGetObjectType     SI2_ARGS(( si2drObjectIdT object,
00531                                                              si2drErrorT  *err));
00532
00533     si2drObjectIdT   si2drObjectGetOwner          SI2_ARGS(( si2drObjectIdT object,
00534                                                              si2drErrorT  *err));
00535
00536     si2drBooleanT    si2drObjectIsNull            SI2_ARGS(( si2drObjectIdT object,
00537                                                              si2drErrorT  *err));
00538
00539     si2drBooleanT    si2drObjectIsSame            SI2_ARGS(( si2drObjectIdT object1,
00540                                                              si2drObjectIdT object2,
00541                                                              si2drErrorT  *err));
00542
00543     si2drBooleanT    si2drObjectIsUsable          SI2_ARGS(( si2drObjectIdT object,
00544                                                              si2drErrorT  *err));
00545
00546     si2drVoidT       si2drObjectSetFileName       SI2_ARGS(( si2drObjectIdT object,
00547                                                              si2drStringT   filename,
00548                                                              si2drErrorT  *err));
00549
00550     si2drVoidT       si2drObjectSetLineNo         SI2_ARGS(( si2drObjectIdT object,
00551                                                              si2drInt32T    lineno,
00552                                                              si2drErrorT  *err));
00553
00554     si2drInt32T      si2drObjectGetLineNo         SI2_ARGS(( si2drObjectIdT object,
00555                                                              si2drErrorT  *err));
00556
00557     si2drStringT     si2drObjectGetFileName       SI2_ARGS(( si2drObjectIdT object,
00558                                                              si2drErrorT  *err));
00559
00560     si2drVoidT       si2drReadLibertyFile         SI2_ARGS(( char *filename,
00561                                                              si2drErrorT  *err));
00562
00563     si2drVoidT       si2drWriteLibertyFile        SI2_ARGS(( char *filename,
00564                                                              si2drGroupIdT group,
00565                                                  char* cellname,
00566                                                              si2drErrorT  *err ));
00567
00568     si2drVoidT       si2drCheckLibertyLibrary     SI2_ARGS(( si2drGroupIdT group,
00569                                                              si2drErrorT  *err));
00570
00571
00572     si2drBooleanT  si2drPIGetTraceMode       SI2_ARGS((si2drErrorT  *err));
00573
00574     si2drVoidT     si2drPIUnSetTraceMode     SI2_ARGS((si2drErrorT  *err));
00575
```

```
00576     si2drVoidT     si2drPISetTraceMode       SI2_ARGS((si2drStringT fname,
00577                                                        si2drErrorT  *err));
00578
00579
00580     si2drVoidT       si2drPISetDebugMode       SI2_ARGS((si2drErrorT  *err));
00581
00582     si2drVoidT       si2drPIUnSetDebugMode     SI2_ARGS((si2drErrorT  *err));
00583
00584     si2drBooleanT    si2drPIGetDebugMode       SI2_ARGS((si2drErrorT  *err));
00585

00586
00587     si2drVoidT     si2drPISetNocheckMode     SI2_ARGS((si2drErrorT  *err));
00588
00589     si2drVoidT     si2drPIUnSetNocheckMode   SI2_ARGS((si2drErrorT  *err));
00590
00591     si2drBooleanT  si2drPIGetNocheckMode     SI2_ARGS((si2drErrorT  *err));
00592
00593
00594     si2drVoidT     si2drGroupMoveAfter SI2_ARGS((si2drGroupIdT groupToMove,
00595                                                  si2drGroupIdT targetGroup,
00596                                                  si2drErrorT   *err));
00597
00598     si2drVoidT     si2drGroupMoveBefore SI2_ARGS((si2drGroupIdT groupToMove,
00599                                                   si2drGroupIdT targetGroup,
00600                                                   si2drErrorT *err));
00601
00602
00603 /* HELPER FUNCTIONS/STRUCTURES */
00604
00605 #ifdef NO_LONG_DOUBLE
00606 #define LONG_DOUBLE double
00607 #define strtold strtod
00608 #else
00609 #define LONG_DOUBLE long double
00610 #endif
00611
00612 struct liberty_value_data
00613 {
00614     int dimensions;
00615     int *dim_sizes; /* a malloc'd array of <dimensions> numbers,
00616                        each number is the size of the array in that dim.*/
00617     LONG_DOUBLE **index_info; /* a Ptr to a malloc'd array of size <dimensions>, ptrs to
00618                                  malloc'd arrays of long double index values.
00619                                  Each array of long double is of length set by
00620                                  the corresponding dim_sizes array values */
00621     LONG_DOUBLE *values; /* a ptr to a malloc'd array of long doubles,
00622                             starting with [0,0,...,0,0], and ending with
00623                             [z-1,y-1,....b-1,a-1], where a-z are the max
00624                             number of elements in each dimension. */
00625 };
00626
00627 LONG_DOUBLE liberty_get_element(struct liberty_value_data *vd, ...);  /* returns NaN if bounds
    exceeded */
00628
00629 void liberty_destroy_value_data(struct liberty_value_data *vd);
00630
00631 struct liberty_value_data *liberty_get_values_data( si2drGroupIdT table_group);
00632
00633 // directly call the main function of liberty parser
00634 extern void print_version();
00635 extern void usage();
00636 extern int main_liberty_parser(int argc, char* argv[]);
00637 extern void get_function(char* filename);
00638
00639 #ifdef __cplusplus
00640 }
00641 #endif
00642
00643 #endif
00644
```

## 6.15 include/verilog_utils.hpp File Reference

```
#include <fstream>
#include <iostream>
#include <string>
#include <unordered_set>
#include "slang/syntax/SyntaxPrinter.h"
```

```
#include "slang/syntax/SyntaxTree.h"
#include "slang/syntax/SyntaxVisitor.h"
```

Include dependency graph for verilog_utils.hpp: This graph shows which files directly or indirectly include this file:

## Classes

- class VerilogVisitor
- class CellExtractor
- class CellPrinter
- class ModuleRewriter

## Functions

- void getAST (const std::string &verilog_file, const std::string &cell)

### 6.15.1 Function Documentation

#### 6.15.1.1 getAST()

```
void getAST (
            const std::string & verilog_file,
            const std::string & cell )
```

Definition at line 410 of file verilog_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

## 6.16 verilog_utils.hpp

Go to the documentation of this file.

```
00001 #ifndef VERILOG_UTILS_H
00002 #define VERILOG_UTILS_H
00003
00004 #include <fstream>
00005 #include <iostream> // For std::ostream
00006 #include <string>
00007 #include <unordered_set>
00008
00009 #include "slang/syntax/SyntaxPrinter.h"
00010 #include "slang/syntax/SyntaxTree.h"
00011 #include "slang/syntax/SyntaxVisitor.h"
00012
00013 // Creating custom visitor class
00014 class VerilogVisitor : public slang::syntax::SyntaxVisitor<VerilogVisitor> {
00015 public:
00016   explicit VerilogVisitor(const std::string &targetCell)
00017       : targetCell_(targetCell), depth_(0), inTargetModule_(false) {}
00018   void handle(const slang::syntax::SyntaxNode &node);
00019   void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00020   void handle(const slang::syntax::PortDeclarationSyntax &portDecl);
00021   void handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst);
00022   void handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst);
00023   void handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock);
00024
00025 private:
00026   const std::string &targetCell_;
```

```
00027   int depth_;
00028   bool inTargetModule_;
00029 };
00030
00031 // Creating a Rewriter to extract a specific cell
00032 class CellExtractor : public slang::syntax::SyntaxRewriter<CellExtractor> {
00033 public:
00034   explicit CellExtractor(const std::string &targetCell) : targetCell_(targetCell), foundTarget_(false)
      {}
00035   void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00036   bool foundTargetCell() const;
00037
00038 private:
00039   const std::string &targetCell_;
00040   bool foundTarget_;
00041 };
00042
00043 // Print specific cell when visiting SyntaxTree
00044 class CellPrinter : public slang::syntax::SyntaxVisitor<CellPrinter> {
00045 public:
00046   explicit CellPrinter(const std::string &targetCell, std::ostream &out)
00047       : targetCell_(targetCell), out_(out), foundTarget_(false) {}
00048   void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00049
00050 private:
00051   const std::string &targetCell_;
00052   std::ostream &out_;
00053   bool foundTarget_;
00054 };
00055
00056 // Comprehensive module rewriter for adding ports, instances, and connections
00057 class ModuleRewriter : public slang::syntax::SyntaxRewriter<ModuleRewriter> {
00058 public:
00059   explicit ModuleRewriter(const std::vector<std::string> &inputPins,
00060                           const std::vector<std::string> &outputPins,
00061                           const std::pair<std::string, std::string> &supercell_entry,
00062                           int instance_count,
00063                           std::shared_ptr<spdlog::logger> logger)
00064       : inputPins_(inputPins), outputPins_(outputPins), cellName_(supercell_entry.first),
00065         moduleName_(supercell_entry.second), logger_(logger), depth_(0),
      instance_count_(instance_count) {}
00066   std::shared_ptr<spdlog::logger> logger_;
00067   void handle(const slang::syntax::SyntaxNode &node);
00068   void handle(const slang::syntax::ModuleDeclarationSyntax &module);
00069
00070 private:
00071   const std::vector<std::string> &inputPins_;
00072   const std::vector<std::string> &outputPins_;
00073   const std::string cellName_;
00074   const std::string moduleName_;
00075   int depth_;
00076   int instance_count_;
00077 };
00078
00079 void getAST(const std::string &verilog_file, const std::string &cell);
00080
00081 #endif // VERILOG_UTILS_H
```

## 6.17 include/version.h File Reference

This graph shows which files directly or indirectly include this file:

### Macros

- #define APP_NAME "ZlibValidation"
- #define APP_VERSION_MAJOR 0
- #define APP_VERSION_MINOR 1
- #define APP_VERSION_PATCH 0
- #define APP_VERSION "0.1.0"
- #define APP_AUTHOR "Song Zixuan"
- #define APP_CONTACT "cedar@zju.edu.cn"
- #define BUILD_TIMESTAMP "2025-03-28 12:02:42"

## 6.17.1 Macro Definition Documentation

### 6.17.1.1 APP_AUTHOR

```
#define APP_AUTHOR "Song Zixuan"
```

Definition at line 10 of file version.h.

### 6.17.1.2 APP_CONTACT

```
#define APP_CONTACT "cedar@zju.edu.cn"
```

Definition at line 11 of file version.h.

### 6.17.1.3 APP_NAME

```
#define APP_NAME "ZlibValidation"
```

Definition at line 5 of file version.h.

### 6.17.1.4 APP_VERSION

```
#define APP_VERSION "0.1.0"
```

Definition at line 9 of file version.h.

### 6.17.1.5 APP_VERSION_MAJOR

```
#define APP_VERSION_MAJOR 0
```

Definition at line 6 of file version.h.

#### 6.17.1.6 APP_VERSION_MINOR

```
#define APP_VERSION_MINOR 1
```

Definition at line 7 of file version.h.

#### 6.17.1.7 APP_VERSION_PATCH

```
#define APP_VERSION_PATCH 0
```

Definition at line 8 of file version.h.

#### 6.17.1.8 BUILD_TIMESTAMP

```
#define BUILD_TIMESTAMP "2025-03-28 12:02:42"
```

Definition at line 12 of file version.h.

## 6.18 version.h

Go to the documentation of this file.

```
00001 // version.h.in
00002 #pragma once
00003
00004 // Auto-generated by CMake - DO NOT EDIT
00005 #define APP_NAME          "ZlibValidation"
00006 #define APP_VERSION_MAJOR 0
00007 #define APP_VERSION_MINOR 1
00008 #define APP_VERSION_PATCH 0
00009 #define APP_VERSION       "0.1.0"
00010 #define APP_AUTHOR        "Song Zixuan"
00011 #define APP_CONTACT       "cedar@zju.edu.cn"
00012 #define BUILD_TIMESTAMP   "2025-03-28 12:02:42"
```

## 6.19 README.md File Reference

## 6.20 src/compare.cpp File Reference

```
#include <chrono>
#include <filesystem>
#include <fstream>
#include <iostream>
#include "spdlog/spdlog.h"
#include <tabulate/markdown_exporter.hpp>
#include "compare.hpp"
#include "version.h"
```

Include dependency graph for compare.cpp:

# 6.21 compare.cpp

Go to the documentation of this file.

```
00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <iostream>
00005
00006 #include "spdlog/spdlog.h"
00007 #include <tabulate/markdown_exporter.hpp>
00008
00009 #include "compare.hpp"
00010 #include "version.h"
00011
00023 LibraryComparator::LibraryComparator(LibFile &ref_libfile, LibFile &comp_libfile, double reltol,
00024                                      double abstol)
00025     : reltol_(reltol), abstol_(abstol) {
00026   std::string ref_json_name = ref_libfile.basename_ + ".json";
00027   if (!std::filesystem::exists(ref_json_name)) {
00028     spdlog::info("Reference JSON file {} not found. Parsing first.", ref_json_name);
00029     ref_libfile.parse();
00030     ref_libfile.writeJsonToFile();
00031     ref_json_ = ref_libfile.lib_json_;
00032   } else {
00033     // Read the JSON file into json object
00034     std::ifstream ref_in(ref_json_name);
00035     if (!ref_in.is_open()) {
00036       spdlog::error("Could not open file '{}' for reading", ref_json_name);
00037       return;
00038     }
00039     try {
00040       ref_json_ = json::parse(ref_in);
00041     } catch (const json::parse_error &e) {
00042       spdlog::error("Error parsing reference JSON file '{}': {}", ref_json_name, e.what());
00043       return;
00044     }
00045   }
00046   std::string comp_json_name = comp_libfile.basename_ + ".json";
00047   if (!std::filesystem::exists(comp_json_name)) {
00048     spdlog::info("Comparison JSON file {} not found. Parsing first.", comp_json_name);
00049     comp_libfile.parse();
00050     comp_libfile.writeJsonToFile();
00051     comp_json_ = comp_libfile.lib_json_;
00052   } else {
00053     // Read the JSON file into json object
00054     std::ifstream comp_in(comp_json_name);
00055     if (!comp_in.is_open()) {
00056       spdlog::error("Could not open file '{}' for reading", comp_json_name);
00057       return;
00058     }
00059     try {
00060       comp_json_ = json::parse(comp_in);
00061     } catch (const json::parse_error &e) {
00062       spdlog::error("Error parsing comparison JSON file '{}': {}", comp_json_name, e.what());
00063       return;
00064     }
00065   }
00066
00067   ref_lib_path_ = ref_libfile.filepath_;
00068   comp_lib_path_ = comp_libfile.filepath_;
00069   spdlog::info("Successfully loaded JSON files for comparison");
00070 }
00071
00099 void LibraryComparator::compareLut(const std::string &cell_name, const std::string &pin_name,
00100                                    const std::string &timing_type, const std::string &related_pin,
00101                                    const std::string &arc_name, const json &ref_lut,
00102                                    const json &comp_lut, Table &table) {
00103   spdlog::info("Comparing LUT of '{}' ...", arc_name);
00104   std::vector<double> ref_index_1 = ref_lut["index_1"].get<std::vector<double>>();
00105   std::vector<double> ref_index_2 = ref_lut["index_2"].get<std::vector<double>>();
00106   std::vector<double> comp_index_1 = comp_lut["index_1"].get<std::vector<double>>();
00107   std::vector<double> comp_index_2 = comp_lut["index_2"].get<std::vector<double>>();
00108   if (ref_index_1 != comp_index_1 || ref_index_2 != comp_index_2) {
00109     spdlog::warn("Mismatch in LUT indices for '{}' in cell: '{}', pin: '{}', related_pin: '{}', "
00110                  "timing_type: '{}'",
00111                  arc_name, cell_name, pin_name, related_pin, timing_type);
00112     return;
00113   } else {
00114     spdlog::debug("LUT indices match for '{}'", arc_name);
00115
00116     std::vector<std::vector<double>> comp_value_matrix;
00117     // Generate a matrix of values from the JSON array for comparison library
00118     for (const auto &row_val : comp_lut["values"]) {
00119       std::vector<double> row_data;
00120       // Check if the row value is an array
```

```
00121       if (!row_val.is_array()) {
00122         spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in comp_lib '{}',
    "
00123                       "cell '{}', pin '{}', "
00124                       "related_pin '{}', timing_type '{}'",
00125                       arc_name, this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00126                       timing_type);
00127         return;
00128       }
00129       // Check if non-numeric values
00130       for (const auto &val : row_val) {
00131         if (val.is_number()) {
00132           row_data.push_back(val.get<double>());
00133         } else {
00134           spdlog::error(
00135               "Non-numeric value found in '{}'.values, skipping value: {} in comp_lib '{}', cell '{}',
    "
00136               "pin '{}', related_pin '{}', timing_type '{}'",
00137               arc_name, val.dump(), this->comp_lib_path_.string(), cell_name, pin_name, related_pin,
00138               timing_type);
00139           return;
00140         }
00141       }
00142       comp_value_matrix.push_back(row_data);
00143     }
00144     // Generate a matrix of values from the JSON array for reference library
00145     std::vector<std::vector<double» ref_value_matrix;
00146     for (const auto &row_val : ref_lut["values"]) {
00147       std::vector<double> row_data;
00148       // Check if the row value is an array
00149       if (!row_val.is_array()) {
00150         spdlog::error("Invalid LUT format: '{}' values should be an array of arrays in ref_lib '{}', "
00151                       "cell '{}', pin '{}', "
00152                       "related_pin '{}', timing_type '{}'",
00153                       arc_name, this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00154                       timing_type);
00155         return;
00156       }
00157       // Check if non-numeric values
00158       for (const auto &val : row_val) {
00159         if (val.is_number()) {
00160           row_data.push_back(val.get<double>());
00161         } else {
00162           spdlog::error(
00163               "Non-numeric value found in '{}'.values, skipping value: {} in ref_lib '{}', cell '{}',
    "
00164               "pin '{}', related_pin '{}', timing_type '{}'",
00165               arc_name, val.dump(), this->ref_lib_path_.string(), cell_name, pin_name, related_pin,
00166               timing_type);
00167           return;
00168         }
00169       }
00170       ref_value_matrix.push_back(row_data);
00171     }
00172     // Compare the matrix for relative tolerance
00173     if (!comp_value_matrix.empty() && !comp_value_matrix[0].empty()) {
00174       for (size_t i = 0; i < comp_value_matrix.size(); ++i) {
00175         for (size_t j = 0; j < comp_value_matrix[i].size(); ++j) {
00176           double ref_val = ref_value_matrix[i][j];
00177           double comp_val = comp_value_matrix[i][j];
00178           if (std::abs(ref_val - comp_val) > reltol_ * std::abs(ref_val) &&
00179               std::abs(ref_val - comp_val) > abstol_) {
00180             spdlog::debug("LUT value mismatch for '{}', index_1: {}, index_2: {}", arc_name,
00181                           ref_index_1[i], ref_index_2[j]);
00182             table.add_row({related_pin + "->" + pin_name, std::to_string(ref_val),
00183                            std::to_string(comp_val), std::to_string(comp_val - ref_val),
00184                            std::to_string((comp_val - ref_val) / ref_val * 100), timing_type,
    arc_name,
00185                            std::to_string(i + 1), std::to_string(ref_index_1[i]), std::to_string(j +
    1),
00186                            std::to_string(ref_index_2[j]), "<"});
00187             spdlog::debug("table size: {}", table.size());
00188           } else {
00189             spdlog::debug("LUT value match for '{}', index_1: {}, index_2: {}", arc_name,
    ref_index_1[i],
00190                           ref_index_2[j]);
00191           }
00192         }
00193       }
00194     } else {
00195       spdlog::error(
00196           "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00197           "timing_type: '{}', arc: '{}'",
00198           cell_name, pin_name, related_pin, timing_type, arc_name);
00199     }
00200   }
00201 }
```

```
00202
00221 void LibraryComparator::compareTimingArc(const std::string &cell_name, const std::string &pin_name,
00222                                          const std::string &timing_type, const json &ref_timing_arc,
00223                                          const json &comp_timing_arc, Table &table) {
00224   spdlog::info("Comparing timing type '{}' ...", timing_type);
00225
00226   std::string related_pin = ref_timing_arc["related_pin"].get<std::string>();
00227   spdlog::debug("Related pin: '{}'", related_pin);
00228
00229   std::vector<std::string> arc_names = {"cell_rise", "cell_fall", "rise_transition",
      "fall_transition"};
00230   for (auto arc_name : arc_names) {
00231     if (comp_timing_arc.contains(arc_name)) {
00232       spdlog::debug("Comparing timing arc: '{}'", arc_name);
00233       // Look for the arc in the reference JSON
00234       if (ref_timing_arc.contains(arc_name)) {
00235         // Found this arc
00236         spdlog::debug("Found timing arc: '{}'", arc_name);
00237         compareLut(cell_name, pin_name, timing_type, related_pin, arc_name, ref_timing_arc[arc_name],
00238                    comp_timing_arc[arc_name], table);
00239       } else {
00240         // arc not found in reference JSON
00241         spdlog::warn("Timing arc: '{}' not found in reference JSON", arc_name);
00242       }
00243     }
00244   }
00245 }
00246
00261 void LibraryComparator::comparePin(const std::string &cell_name, const std::string &pin_name,
00262                                    const json &ref_pin, const json &comp_pin, Table &table) {
00263   spdlog::info("Comparing pin '{}' ...", pin_name);
00264
00265   if (comp_pin.contains("timing_arcs")) {
00266     for (const auto &comp_arc : comp_pin["timing_arcs"]) {
00267       std::string timing_type = comp_arc["timing_type"].get<std::string>();
00268       spdlog::debug("Comparing timing type: '{}'", timing_type);
00269       // Look for the timing type in the reference JSON
00270       auto ref_arc_it = std::find_if(
00271           ref_pin["timing_arcs"].begin(), ref_pin["timing_arcs"].end(),
00272           [&timing_type](const json &ref_arc) { return ref_arc["timing_type"] == timing_type; });
00273       if (ref_arc_it != ref_pin["timing_arcs"].end()) {
00274         // Found this timing type
00275         spdlog::debug("Found timing type: '{}'", timing_type);
00276         compareTimingArc(cell_name, pin_name, timing_type, *ref_arc_it, comp_arc, table);
00277       } else {
00278         // timing arc not found in reference JSON
00279         spdlog::warn("Timing arc: '{}' not found in reference JSON", timing_type);
00280       }
00281     }
00282   } else {
00283     spdlog::info("No timing arcs found for pin: '{}'", comp_pin["pin_name"].get<std::string>());
00284   }
00285 }
00286
00300 void LibraryComparator::compareCell(const std::string &cell_name, const json &ref_cell,
00301                                     const json &comp_cell, Table &table) {
00302   spdlog::info("Comparing cell '{}' ...", cell_name);
00303
00304   if (comp_cell.contains("output_pins")) {
00305     for (const auto &comp_pin : comp_cell["output_pins"]) {
00306       std::string pin_name = comp_pin["pin_name"].get<std::string>();
00307       spdlog::debug("Comparing pin: '{}'", pin_name);
00308       // Look for the pin in the reference JSON
00309       auto ref_pin_it =
00310           std::find_if(ref_cell["output_pins"].begin(), ref_cell["output_pins"].end(),
00311                        [&pin_name](const json &ref_pin) { return ref_pin["pin_name"] == pin_name; });
00312       if (ref_pin_it != ref_cell["output_pins"].end()) {
00313         // Found this pin
00314         spdlog::debug("Found pin: '{}'", pin_name);
00315         comparePin(cell_name, pin_name, *ref_pin_it, comp_pin, table);
00316       } else {
00317         // pin not found in reference JSON
00318         spdlog::warn("Pin: '{}' not found in reference JSON", pin_name);
00319       }
00320     }
00321   } else {
00322     spdlog::info("No output pins found for cell: '{}'", cell_name);
00323   }
00324 }
00325
00342 void LibraryComparator::generateReport(const std::string &output_file) {
00343   std::ofstream outfile(output_file);
00344   outfile « "# LIBRARY comparison\n" « std::endl;
00345   outfile « "**Reference library: " « ref_lib_path_ « "**\n" « std::endl;
00346   outfile « "**Comparison library: " « comp_lib_path_ « "**\n" « std::endl;
00347   outfile « "**Absolute tolerance: " « abstol_ « "**\n" « std::endl;
00348   outfile « "**Relative tolerance: " « reltol_ « "**\n" « std::endl;
```

```
00349   outfile « "**Performed by " « APP_NAME « " v" « APP_VERSION « " from " « APP_AUTHOR;
00350   auto now = std::chrono::system_clock::now();
00351   std::time_t now_time_t = std::chrono::system_clock::to_time_t(now);
00352   std::tm *now_tm = std::localtime(&now_time_t);
00353   outfile « ". on: " « std::put_time(now_tm, "%c") « "**\n" « std::endl;
00354   outfile « "> Legend: < outlier, * scaled, ! indices switched, ^ slews extrapolated, ~ loads "
00355             "extrapolated,\n";
00356   outfile « "> \n";
00357   outfile
00358       « "> Legend: + padding added, /0 divide by zero, / slews interpolated, # loads interpolated\n";
00359   outfile « "> \n";
00360   outfile « "> Legend: « value is less but unknown, » value is more but unknown\n\n";
00361
00362   spdlog::info("Starting comparison report generation ...");
00363   for (const auto &comp_cell : comp_json_["cells"]) {
00364     std::string cell_name = comp_cell["cell_name"].get<std::string>();
00365     spdlog::debug("Comparing cell: '{}'", cell_name);
00366     // Look for the cell in the reference JSON
00367     auto ref_cell_it =
00368         std::find_if(ref_json_["cells"].begin(), ref_json_["cells"].end(),
00369                      [&cell_name](const json &ref_cell) { return ref_cell["cell_name"] == cell_name;
    });
00370     if (ref_cell_it != ref_json_["cells"].end()) {
00371       // Found this cell
00372       spdlog::debug("Found cell: '{}'", cell_name);
00373       Table table;
00374       table.add_row({"Pin Name", "Reference", "Comparison", "Diff", "Diff %", "Type", "Arc Name",
00375                      "Row #", "Index_1", "Col #", "Index_2", "Note"});
00376       // Header formatting
00377       for (size_t i = 0; i < table[0].size(); ++i) {
00378         table[0][i].format().font_color(Color::yellow).font_style({FontStyle::bold});
00379       }
00380       compareCell(cell_name, *ref_cell_it, comp_cell, table);
00381
00382       if (table.size() > 1) {
00383         // Data starts from row 1, row 0 is header
00384         size_t failed_count = table.size() - 1;
00385         double sum_diff = 0.0;
00386         double sum_diff_percent = 0.0;
00387         size_t outlier_count = 0;
00388         double max_diff = -1e9;
00389         double max_diff_percent = -1e9;
00390         std::vector<double> diff_values; // Store diff values for std deviation calculation
00391
00392         // Index of columns
00393         const int diff_index = 3;
00394         const int diff_percent_index = 4;
00395         const int note_index = 11;
00396
00397         for (size_t i = 1; i < table.size(); ++i) {
00398           try {
00399             double diff = std::stod(table[i][diff_index].get_text());
00400             double diff_percent = std::stod(table[i][diff_percent_index].get_text());
00401
00402             sum_diff += diff;
00403             sum_diff_percent += diff_percent;
00404             diff_values.push_back(diff_percent);
00405
00406             if (table[i][note_index].get_text() == "<") {
00407               outlier_count++;
00408             }
00409             max_diff = std::max(max_diff, diff);
00410             max_diff_percent = std::max(max_diff_percent, diff_percent);
00411
00412           } catch (const std::invalid_argument &e) {
00413             spdlog::error("Could not convert value to double: {}", e.what());
00414             continue; // Skip this row if there's an error
00415           }
00416         }
00417
00418         double avg_diff = sum_diff / failed_count;
00419         double avg_diff_percent = sum_diff_percent / failed_count;
00420
00421         outfile « "## " « cell_name « "\n\n";
00422         outfile « "Delay Comparison in ns\n\n";
00423         if (output_file.substr(output_file.size() - 3) == ".md") {
00424           MarkdownExporter exporter;
00425           auto markdown = exporter.dump(table);
00426           outfile « markdown « std::endl;
00427         } else {
00428           outfile « table « std::endl;
00429         }
00430         outfile « std::endl;
00431         std::cout « table « std::endl;
00432         outfile « cell_name « " delay SUMMARY (abstol: " « abstol_ « ", reltol: " « reltol_
00433                 « ")\n";
00434
```

```
00435         // Create summary table
00436         Table summary_table;
00437         summary_table.add_row({"Cell Name", "Data Type", "Failed Count", "Avg Diff", "Avg Diff%",
00438                                "Max Diff", "Max Diff%", "Outliers"});
00439         summary_table.add_row({cell_name, "delay(ns)", std::to_string(failed_count),
00440                                std::to_string(avg_diff), std::to_string(avg_diff_percent) + "%",
00441                                std::to_string(max_diff), std::to_string(max_diff_percent) + "%",
00442                                std::to_string(outlier_count)});
00443
00444         // Output summary table
00445         if (output_file.substr(output_file.size() - 3) == ".md") {
00446           MarkdownExporter exporter;
00447           auto markdown = exporter.dump(summary_table);
00448           outfile « markdown « std::endl;
00449         } else {
00450           outfile « summary_table « std::endl;
00451         }
00452         outfile « "Worst delay outlier: Max Abs: " « max_diff « ", Max Rel: " « max_diff_percent
00453                 « "%, Outliers: " « outlier_count « "\n\n";
00454       }
00455     } else {
00456       // cell not found in reference JSON
00457       spdlog::warn("Cell: '{}' not found in reference JSON", cell_name);
00458     }
00459   }
00460
00461   outfile.close();
00462 }
```

## 6.22 src/iterators.cpp File Reference

`#include "iterators.hpp"`

Include dependency graph for iterators.cpp:

## 6.23 iterators.cpp

Go to the documentation of this file.

```
00001 #include "iterators.hpp"
00002
00003 GroupsIterator::GroupsIterator(si2drGroupsIdT groups, si2drErrorT &err) : groups_(groups), err_(err) {
00004   group_ = si2drIterNextGroup(groups_, &err_);
00005 }
00006 GroupsIterator::~GroupsIterator() { si2drIterQuit(groups_, &err_); }
00007
00008 void GroupsIterator::next() { group_ = si2drIterNextGroup(groups_, &err_); }
00009 bool GroupsIterator::end() { return si2drObjectIsNull(group_, &err_); }
00010
00011 LibGroup GroupsIterator::get() { return LibGroup(group_, err_); }
00012
00013 AttributesIterator::AttributesIterator(si2drAttrsIdT attrs, si2drErrorT &err)
00014     : attrs_(attrs), err_(err) {
00015   attr_ = si2drIterNextAttr(attrs_, &err_);
00016 }
00017 AttributesIterator::~AttributesIterator() { si2drIterQuit(attrs_, &err_); }
00018
00019 void AttributesIterator::next() { attr_ = si2drIterNextAttr(attrs_, &err_); }
00020 bool AttributesIterator::end() { return si2drObjectIsNull(attr_, &err_); }
00021
00022 LibAttribute AttributesIterator::get() { return LibAttribute(attr_, err_); }
00023
00024 ValuesIterator::ValuesIterator(si2drValuesIdT values, si2drErrorT &err) : values_(values), err_(err) {
00025   si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00026 }
00027 ValuesIterator::~ValuesIterator() { si2drIterQuit(values_, &err_); }
00028
00029 void ValuesIterator::next() {
00030   si2drIterNextComplexValue(values_, &vtype_, &int_, &float_, &str_, &bool_, &exprp_, &err_);
00031 }
00032 bool ValuesIterator::end() { return vtype_ == SI2DR_UNDEFINED_VALUETYPE; }
```

## 6.24 src/json_utils.cpp File Reference

```
#include "spdlog/spdlog.h"
#include "iterators.hpp"
#include "json_utils.hpp"
```
Include dependency graph for json_utils.cpp:

### Functions

- std::vector< double > parseStringToVector (const std::string &str)

  *Parses a string representation of a vector of doubles into a vector of doubles.*
- json generateLutJson (LibGroup &lib_lut_group, si2drErrorT &err)

  *Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.*
- json generateTimingJson (LibGroup &lib_timing_group, si2drErrorT &err)

  *Generates a JSON representation of timing information from a library timing group.*
- json generatePowerJson (LibGroup &lib_power_group, si2drErrorT &err)

  *Converts a Liberty power group into a JSON representation.*
- std::pair< std::string, json > generatePinJson (LibGroup &lib_pin_group, si2drErrorT &err)

  *Generates a JSON representation of a Liberty pin group.*
- json generateCellJson (LibGroup &lib_cell_group, si2drErrorT &err)

  *Generates a JSON representation of a cell from a LibGroup object.*

### 6.24.1 Function Documentation

#### 6.24.1.1 generateCellJson()

```
json generateCellJson (
            LibGroup & lib_cell_group,
            si2drErrorT & err )
```

Generates a JSON representation of a cell from a LibGroup object.

This function processes a LibGroup representing a cell and converts it into a JSON object. It extracts the cell name and processes specific attributes such as area, cell_leakage_power, and cell_footprint. It also processes pins within the cell by categorizing them based on their direction (input, output, internal, inout).

**Parameters**

| | |
|---|---|
| *lib_cell_group* | The LibGroup object representing the cell |
| *err* | Reference to a si2drErrorT object for error handling |

**Returns**

json A JSON object containing the cell's information with the following structure:

- "cell_name": string - name of the cell

- "area": float (optional) - cell area if present
- "cell_leakage_power": float (optional) - cell leakage power if present
- "cell_footprint": string (optional) - cell footprint if present
- "input_pins": array - list of input pins
- "output_pins": array - list of output pins
- "internal_pins": array - list of internal pins
- "inout_pins": array - list of inout pins

Definition at line 272 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.24.1.2 generateLutJson()

```
json generateLutJson (
            LibGroup & lib_lut_group,
            si2drErrorT & err )
```

Generates a JSON object representation of a look-up table (LUT) from a LibGroup object.

This function iterates through the attributes of the provided LibGroup object representing a LUT and converts them into a JSON structure. It specifically handles the following attributes:

- "index_1": Converted to a vector and stored in the JSON
- "index_2": Converted to a vector and stored in the JSON
- "values": Each value is parsed into a vector and added to an array in the JSON

Any other attributes encountered will trigger a warning message.

**Parameters**

| | |
|---|---|
| *lib_lut_group* | The LibGroup object containing the LUT data to be converted |
| *err* | Reference to an si2drErrorT object to track any errors during processing |

**Returns**

json A JSON object representing the LUT data with keys for "index_1", "index_2", and "values"

Definition at line 62 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.24.1.3 generatePinJson()

```
std::pair< std::string, json > generatePinJson (
            LibGroup & lib_pin_group,
            si2drErrorT & err )
```

Generates a JSON representation of a Liberty pin group.

This function traverses a Liberty pin group, extracts relevant attributes and sub-groups, and converts them into a JSON object. It also determines the pin's direction.

Processes the following pin attributes:

- direction
- max_transition, capacitance, rise_capacitance, fall_capacitance, max_capacitance (float types)
- function, power_down_function, related_ground_pin, related_power_pin, three_state (string types)
- clock (boolean type)

Handles the following sub-groups:

- internal_power: Converted using generatePowerJson()
- timing: Converted using generateTimingJson()

**Parameters**

| | |
|---|---|
| *lib_pin_group* | The Liberty pin group to process |
| *err* | Reference to an error object for tracking Liberty API errors |

**Returns**

A pair containing the pin direction (string) and the JSON representation of the pin

Definition at line 206 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.24.1.4 generatePowerJson()

```
json generatePowerJson (
            LibGroup & lib_power_group,
            si2drErrorT & err )
```

Converts a Liberty power group into a JSON representation.

This function processes a Liberty power group and converts its attributes and subgroups into a JSON object. It handles attributes like "when", "related_pin", and "related_pg_pin", as well as "rise_power" and "fall_power" subgroups.

**Parameters**

| | |
|---|---|
| *lib_power_group* | The Liberty power group to convert |
| *err* | Reference to an si2drErrorT object for error handling |

**Returns**

json A JSON object representing the power group data

The function:

- Extracts string attributes (when, related_pin, related_pg_pin)
- Processes rise_power and fall_power subgroups by converting them to LUT JSON format
- Logs warnings for unknown power subgroup types

Definition at line 154 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.24.1.5 generateTimingJson()

```
json generateTimingJson (
            LibGroup & lib_timing_group,
            si2drErrorT & err )
```

Generates a JSON representation of timing information from a library timing group.

This function extracts timing attributes and sub-groups from a Liberty timing group and organizes them into a JSON object. It processes standard timing attributes like 'related_pin', 'timing_type', 'timing_sense', and 'when', as well as timing tables such as 'cell_fall', 'cell_rise', 'fall_transition', 'rise_transition', 'fall_constraint', and 'rise_constraint'.

**Parameters**

| | |
|---|---|
| *lib_timing_group* | The Liberty timing group to process |
| *err* | Reference to an si2drErrorT object for error reporting |

**Returns**

json A JSON object containing the extracted timing information

Definition at line 106 of file json_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.24.1.6 parseStringToVector()

```
std::vector< double > parseStringToVector (
            const std::string & str )
```

Parses a string representation of a vector of doubles into a vector of doubles.

This function takes a string containing comma-separated numbers, cleans it by removing backslashes and newline characters, and then converts each comma-separated value into a double that is added to the resulting vector.

**Parameters**

| | |
|---|---|
| *str* | The input string to be parsed, containing comma-separated numbers |

**Returns**

std::vector<double> A vector of doubles parsed from the input string

**Exceptions**

| | |
|---|---|
| *std::invalid_argument* | If the string contains values that cannot be converted to double |
| *std::out_of_range* | If the string contains values that are out of range for double |

Definition at line 28 of file json_utils.cpp.

Here is the caller graph for this function:

## 6.25 json_utils.cpp

Go to the documentation of this file.

```
00001 #include "spdlog/spdlog.h"
00002
00003 #include "iterators.hpp"
00004 #include "json_utils.hpp"
00005
00006 /*
00007 json
00008  []
00009  JSON  null
00010  {}
00011  JSON
00012 null
00013 */
00014
00028 std::vector<double> parseStringToVector(const std::string &str) {
00029   std::vector<double> result;
00030   std::string cleaned_str;
00031
00032   // Remove backslashes and newline characters
00033   for (char c : str) {
00034     if (c != '\\' && c != '\n') {
00035       cleaned_str += c;
00036     }
00037   }
00038
00039   std::stringstream ss(cleaned_str);
00040   std::string item;
00041   while (std::getline(ss, item, ',')) {
00042     result.push_back(std::stod(item));
00043   }
00044   return result;
00045 }
00046
00062 json generateLutJson(LibGroup &lib_lut_group, si2drErrorT &err) {
00063   json lut_json;
00064
00065   AttributesIterator attr_iter(lib_lut_group.getAttrs(), err);
00066   for (; !attr_iter.end(); attr_iter.next()) {
00067     LibAttribute lib_attr = attr_iter.get();
00068     std::string lut_attr_name = lib_attr.getName();
00069
00070     // spdlog::debug("LUT Attribute Name: {}", lib_attr.getName());
00071     // spdlog::debug("Is Complex? {}", lib_attr.isComplex());
00072
00073     if (lut_attr_name == "index_1" || lut_attr_name == "index_2") {
00074       ValuesIterator values_iter(lib_attr.getValues(), err);
00075       for (; !values_iter.end(); values_iter.next()) {
00076         // spdlog::debug("Type: {}", int(values_iter.vtype_)); // 5 is string
```

```
00077         // spdlog::debug("Str: {}", values_iter.str_);
00078         lut_json[lut_attr_name] = parseStringToVector(std::string(values_iter.str_));
00079       }
00080     } else if (lut_attr_name == "values") {
00081       ValuesIterator values_iter(lib_attr.getValues(), err);
00082       for (; !values_iter.end(); values_iter.next()) {
00083         // spdlog::debug("{}", values_iter.str_);
00084         lut_json["values"].push_back(parseStringToVector(std::string(values_iter.str_)));
00085       }
00086     } else {
00087       spdlog::warn("Unknown LUT attribute name: {}", lut_attr_name);
00088     }
00089   }
00090   return lut_json;
00091 }
00092
00106 json generateTimingJson(LibGroup &lib_timing_group, si2drErrorT &err) {
00107   json timing_json;
00108
00109   AttributesIterator attr_iter(lib_timing_group.getAttrs(), err);
00110   for (; !attr_iter.end(); attr_iter.next()) {
00111     LibAttribute lib_attr = attr_iter.get();
00112
00113     std::string attr_name = lib_attr.getName();
00114     if (attr_name == "related_pin" || attr_name == "timing_type" || attr_name == "timing_sense" ||
00115         attr_name == "when") {
00116       timing_json[attr_name] = lib_attr.getString();
00117     }
00118     // More timing attributes can be added here
00119   }
00120
00121   GroupsIterator timing_sub_group_iter(lib_timing_group.getGroups(), err);
00122   for (; !timing_sub_group_iter.end(); timing_sub_group_iter.next()) {
00123     LibGroup lib_timing_sub_group = timing_sub_group_iter.get();
00124
00125     std::string timing_sub_group_type = lib_timing_sub_group.getType();
00126     // std::string timing_sub_group_name = lib_timing_sub_group.getName();
00127     if (timing_sub_group_type == "cell_fall" || timing_sub_group_type == "cell_rise" ||
00128         timing_sub_group_type == "fall_transition" || timing_sub_group_type == "rise_transition" ||
00129         timing_sub_group_type == "fall_constraint" || timing_sub_group_type == "rise_constraint") {
00130       timing_json[timing_sub_group_type] = generateLutJson(lib_timing_sub_group, err);
00131     } else {
00132       spdlog::warn("Unknown timing sub group type: {}", timing_sub_group_type);
00133     }
00134   }
00135   return timing_json;
00136 }
00137
00154 json generatePowerJson(LibGroup &lib_power_group, si2drErrorT &err) {
00155   json power_json;
00156
00157   AttributesIterator attr_iter(lib_power_group.getAttrs(), err);
00158   for (; !attr_iter.end(); attr_iter.next()) {
00159     LibAttribute lib_attr = attr_iter.get();
00160
00161     std::string attr_name = lib_attr.getName();
00162     if (attr_name == "when" || attr_name == "related_pin" || attr_name == "related_pg_pin") {
00163       power_json[attr_name] = lib_attr.getString();
00164     }
00165     // More power attributes can be added here
00166   }
00167
00168   GroupsIterator power_sub_group_iter(lib_power_group.getGroups(), err);
00169   for (; !power_sub_group_iter.end(); power_sub_group_iter.next()) {
00170     LibGroup lib_power_sub_group = power_sub_group_iter.get();
00171
00172     std::string power_sub_group_type = lib_power_sub_group.getType();
00173     // std::string power_sub_group_name = lib_power_sub_group.getName();
00174     if (power_sub_group_type == "rise_power") {
00175       // spdlog::debug("Rise Power: {}", power_sub_group_name);
00176       power_json["rise_power"] = generateLutJson(lib_power_sub_group, err);
00177     } else if (power_sub_group_type == "fall_power") {
00178       power_json["fall_power"] = generateLutJson(lib_power_sub_group, err);
00179     } else {
00180       spdlog::warn("Unknown power sub group type: {}", power_sub_group_type);
00181     }
00182   }
00183   return power_json;
00184 }
00185
00206 std::pair<std::string, json> generatePinJson(LibGroup &lib_pin_group, si2drErrorT &err) {
00207   json pin_json;
00208   std::string direction;
00209   pin_json["pin_name"] = lib_pin_group.getName();
00210
00211   AttributesIterator attr_iter(lib_pin_group.getAttrs(), err);
00212   for (; !attr_iter.end(); attr_iter.next()) {
```

```
00213     LibAttribute lib_attr = attr_iter.get();
00214
00215     std::string attr_name = lib_attr.getName();
00216     if (attr_name == "direction") {
00217       direction = lib_attr.getString();
00218     } else if (attr_name == "max_transition" || attr_name == "capacitance" ||
00219                attr_name == "rise_capacitance" || attr_name == "fall_capacitance" ||
00220                attr_name == "max_capacitance") {
00221       pin_json[attr_name] = lib_attr.getFloat();
00222     } else if (attr_name == "function" || attr_name == "power_down_function" ||
00223                attr_name == "related_ground_pin" || attr_name == "related_power_pin" ||
00224                attr_name == "three_state") {
00225       pin_json[attr_name] = lib_attr.getString();
00226     } else if (attr_name == "clock") {
00227       pin_json[attr_name] = lib_attr.getBoolean();
00228     }
00229     // More pin attributes can be added here
00230   }
00231
00232   GroupsIterator pin_sub_group_iter(lib_pin_group.getGroups(), err);
00233   for (; !pin_sub_group_iter.end(); pin_sub_group_iter.next()) {
00234     LibGroup lib_pin_sub_group = pin_sub_group_iter.get();
00235
00236     std::string pin_sub_group_type = lib_pin_sub_group.getType();
00237     // std::string pin_sub_group_name = lib_pin_sub_group.getName();
00238     if (pin_sub_group_type == "internal_power") {
00239       json power_json = generatePowerJson(lib_pin_sub_group, err);
00240       pin_json["power_arcs"].push_back(power_json);
00241     } else if (pin_sub_group_type == "timing") {
00242       // spdlog::debug("Has Timing: {}", lib_pin_group.getName());
00243       json timing_json = generateTimingJson(lib_pin_sub_group, err);
00244       pin_json["timing_arcs"].push_back(timing_json);
00245     } else {
00246       spdlog::warn("Unknown pin sub group type: {}", pin_sub_group_type);
00247     }
00248   }
00249   return std::make_pair(direction, pin_json);
00250 }
00251
00272 json generateCellJson(LibGroup &lib_cell_group, si2drErrorT &err) {
00273   json cell_json;
00274   cell_json["cell_name"] = lib_cell_group.getName();
00275
00276   AttributesIterator attr_iter(lib_cell_group.getAttrs(), err);
00277   for (; !attr_iter.end(); attr_iter.next()) {
00278     LibAttribute lib_attr = attr_iter.get();
00279
00280     std::string attr_name = lib_attr.getName();
00281     if (attr_name == "area" || attr_name == "cell_leakage_power") {
00282       cell_json[attr_name] = lib_attr.getFloat();
00283     } else if (attr_name == "cell_footprint") {
00284       cell_json[attr_name] = lib_attr.getString();
00285     }
00286     // More cell attributes can be added here
00287   }
00288
00289   GroupsIterator cell_sub_group_iter(lib_cell_group.getGroups(), err);
00290   for (; !cell_sub_group_iter.end(); cell_sub_group_iter.next()) {
00291     LibGroup lib_cell_sub_group = cell_sub_group_iter.get();
00292
00293     std::string cell_sub_group_type = lib_cell_sub_group.getType();
00294     // std::string cell_sub_group_name = lib_cell_sub_group.getName();
00295
00296     // spdlog::debug("Cell Sub Group Type: {}", cell_sub_group_type);
00297     // spdlog::debug("Cell Sub Group Name: {}", cell_sub_group_name);
00298
00299     if (cell_sub_group_type == "pin") {
00300       auto [direction, pin_json] = generatePinJson(lib_cell_sub_group, err);
00301       if (direction == "input") {
00302         cell_json["input_pins"].push_back(pin_json);
00303       } else if (direction == "output") {
00304         cell_json["output_pins"].push_back(pin_json);
00305       } else if (direction == "internal") {
00306         cell_json["internal_pins"].push_back(pin_json);
00307       } else if (direction == "inout") {
00308         cell_json["inout_pins"].push_back(pin_json);
00309       } else {
00310         spdlog::warn("Unknown direction: {}", direction);
00311       }
00312     }
00313   }
00314   return cell_json;
00315 }
```

## 6.26 src/lib_attribute.cpp File Reference

```
#include "lib_attribute.hpp"
```
Include dependency graph for lib_attribute.cpp:

## 6.27 lib_attribute.cpp

Go to the documentation of this file.
```
00001 #include "lib_attribute.hpp"
00002
00003 LibAttribute::LibAttribute(si2drAttrIdT attr, si2drErrorT &err) : attr_(attr), err_(err) {}
00004
00005 LibAttribute::~LibAttribute() {}
00006
00007 std::string LibAttribute::getName() {
00008   si2drStringT name = si2drAttrGetName(attr_, &err_);
00009   return name ? std::string(name) : std::string();
00010 }
00011
00012 bool LibAttribute::isComplex() { return si2drAttrGetAttrType(attr_, &err_) ? 1 : 0; }
00013
00014 si2drValuesIdT LibAttribute::getValues() { return si2drComplexAttrGetValues(attr_, &err_); }
00015
00016 long int LibAttribute::getInt() { return si2drSimpleAttrGetInt32Value(attr_, &err_); }
00017
00018 double LibAttribute::getFloat() { return si2drSimpleAttrGetFloat64Value(attr_, &err_); }
00019
00020 std::string LibAttribute::getString() {
00021   si2drStringT str = si2drSimpleAttrGetStringValue(attr_, &err_);
00022   return str ? std::string(str) : std::string();
00023 }
00024
00025 bool LibAttribute::getBoolean() { return si2drSimpleAttrGetBooleanValue(attr_, &err_) ? 1 : 0; }
```

## 6.28 src/lib_file.cpp File Reference

```
#include <chrono>
#include <filesystem>
#include <fstream>
#include <sstream>
#include <unordered_set>
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "iterators.hpp"
#include "json_utils.hpp"
#include "lib_file.hpp"
#include "verilog_utils.hpp"
```
Include dependency graph for lib_file.cpp:

## 6.29 lib_file.cpp

Go to the documentation of this file.
```
00001 #include <chrono>
00002 #include <filesystem>
00003 #include <fstream>
00004 #include <sstream>
00005 #include <unordered_set>
00006
00007 #include "spdlog/sinks/basic_file_sink.h"
```

```
00008 #include "spdlog/sinks/stdout_color_sinks.h"
00009 #include "spdlog/spdlog.h"
00010
00011 #include "iterators.hpp"
00012 #include "json_utils.hpp"
00013 #include "lib_file.hpp"
00014 #include "verilog_utils.hpp"
00015
00028 LibFile::LibFile(const std::string &filepath, const std::string &loggername)
00029     : filepath_(filepath), loggername_(loggername) {
00030   filename_ = filepath_.string();
00031   basename_ = filepath_.stem().string();
00032   jsonname_ = basename_ + ".json";
00033
00034   auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00035   console_sink->set_level(spdlog::level::info);
00036
00037   auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(loggername_, true);
00038   file_sink->set_level(spdlog::level::debug);
00039
00040   std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00041   logger_ = std::make_shared<spdlog::logger>(loggername_, sinks.begin(), sinks.end());
00042   logger_->set_level(spdlog::level::debug);
00043
00044   logger_->info("Created LibFile object for '{}'", filename_);
00045   logger_->info("Debug log in: '{}'", loggername_);
00046 }
00047
00048 LibFile::~LibFile() { logger_->info("Closing file: '{}'", filename_); }
00049
00060 void LibFile::writeJsonToFile() {
00061   std::ofstream out(jsonname_);
00062   if (!out.is_open()) {
00063     logger_->error("Could not open file '{}' for writing", jsonname_);
00064     return;
00065   }
00066   out << lib_json_.dump(2);
00067   out.close();
00068   logger_->info("JSON data written to '{}'", jsonname_);
00069 }
00070
00091 void LibFile::read() {
00092   logger_->info("Reading '{}' ...", filename_);
00093
00094   auto start = std::chrono::high_resolution_clock::now();
00095   si2drReadLibertyFile(const_cast<char *>(filepath_.c_str()), &err_);
00096   auto end = std::chrono::high_resolution_clock::now();
00097   std::chrono::duration<double> duration = end - start;
00098
00099   if (err_ == SI2DR_INVALID_NAME) {
00100     logger_->error("Could not open file '{}' for parsing, quitting...", filename_);
00101     exit(301);
00102   } else if (err_ == SI2DR_SYNTAX_ERROR) {
00103     logger_->error("Syntax Errors were detected in the input file!");
00104     exit(401);
00105   } else {
00106     logger_->info("Done. Read time: {:.2f} seconds", duration.count());
00107   }
00108 }
00109
00130 void LibFile::parse() {
00131   si2drPIInit(&err_); // Initialize private error handler
00132   this->read();       // Read the Liberty file
00133
00134   logger_->info("Parsing '{}' ...", filename_);
00135   // Create a scope for the top-level groups iteration
00136   {
00137     GroupsIterator group_iter(si2drPIGetGroups(&err_), err_);
00138     for (; !group_iter.end(); group_iter.next()) {
00139       LibGroup lib_group = group_iter.get();
00140
00141       if (lib_group.getName() != "") {
00142         libname_ = lib_group.getName();
00143         lib_json_["library_name"] = this->libname_;
00144         logger_->info("Library Name: {}", libname_);
00145       } else {
00146         logger_->warn("Library Name: <NONAME>");
00147       }
00148       // Second level groups
00149       GroupsIterator sub_group_iter(lib_group.getGroups(), err_);
00150       for (; !sub_group_iter.end(); sub_group_iter.next()) {
00151         LibGroup lib_sub_group = sub_group_iter.get();
00152
00153         std::string sub_group_type = lib_sub_group.getType();
00154         std::string sub_group_name = lib_sub_group.getName();
00155
00156         if (sub_group_type == "cell") {
```

```
00157           logger_->debug("Cell Name: {}", sub_group_name);
00158           // if (sub_group_name != "AN2D0") {
00159           //   continue;
00160           // }
00161           // Handle each cell
00162           json cell_json = generateCellJson(lib_sub_group, err_);
00163           lib_json_["cells"].push_back(cell_json);
00164         } else if (sub_group_type == "operating_conditions") {
00165           logger_->info("Operating Conditions: {}", sub_group_name);
00166           // Get PVT from Operating Conditions
00167           AttributesIterator attr_iter(lib_sub_group.getAttrs(), err_);
00168           for (; !attr_iter.end(); attr_iter.next()) {
00169             LibAttribute lib_attr = attr_iter.get();
00170             logger_->debug("Attribute Name: {}", lib_attr.getName());
00171             logger_->debug("Int Value: {}", lib_attr.getInt());
00172             logger_->debug("Float Value: {}", lib_attr.getFloat());
00173             logger_->debug("String Value: {}", lib_attr.getString());
00174             if (lib_attr.getName() == "process") {
00175               process_ = lib_attr.getInt();
00176               lib_json_["process"] = process_;
00177             } else if (lib_attr.getName() == "voltage") {
00178               voltage_ = lib_attr.getFloat();
00179               // Round to 2 decimal places to avoid floating-point precision issues
00180               lib_json_["voltage"] = std::round(voltage_ * 100) / 100.0;
00181             } else if (lib_attr.getName() == "temperature") {
00182               temperature_ = lib_attr.getInt();
00183               lib_json_["temperature"] = temperature_;
00184             }
00185           }
00186           logger_->info("P: {}, V: {}, T: {}", process_, voltage_, temperature_);
00187         }
00188         // sub_group_iter's lifetime ends here, and the destructor is called
00189       }
00190     }
00191     // group_iter's lifetime ends here, and the destructor is called
00192   }
00193   si2drPIQuit(&err_);
00194 }
00195
00196 void LibFile::modify() { logger_->info("Modifying the file..."); }
00197
00219 bool LibFile::checkTimingArcMonotonicity(const json &cell, const json &pin, const json &arc,
00220                                          const std::string &timing_arc_name, const bool is_slew) {
00221   if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00222     logger_->debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}', when: '{}'",
00223                    cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00224                    arc["related_pin"].get<std::string>(), timing_arc_name,
00225                    arc["when"].get<std::string>());
00226   } else {
00227     logger_->debug("Checking cell: '{}', pin: '{}', related_pin: '{}', timing_arc: '{}'",
00228                    cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00229                    arc["related_pin"].get<std::string>(), timing_arc_name);
00230   }
00231   bool is_monotonic = true; // Assume the values are monotonic initially
00232   if (arc.contains(timing_arc_name) && arc[timing_arc_name].contains("values")) {
00233     std::vector<std::vector<double» value_matrix;
00234     // Generate a matrix of values from the JSON array
00235     for (const auto &row_val : arc[timing_arc_name]["values"]) {
00236       std::vector<double> row_data;
00237       // Check if the value is an array
00238       if (!row_val.is_array()) {
00239         logger_->error("Invalid format: '{}' values should be an array of arrays in cell '{}', pin "
00240                        "'{}', related_pin '{}'",
00241                        timing_arc_name, cell["cell_name"].get<std::string>(),
00242                        pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00243         return false;
00244       }
00245       // Check for non-numeric values
00246       for (const auto &val : row_val) {
00247         if (val.is_number()) {
00248           row_data.push_back(val.get<double>());
00249         } else {
00250           logger_->warn("Non-numeric value found in '{}'.values, skipping value: {} in cell '{}', "
00251                         "pin '{}', related_pin '{}'",
00252                         timing_arc_name, val.dump(), cell["cell_name"].get<std::string>(),
00253                         pin["pin_name"].get<std::string>(), arc["related_pin"].get<std::string>());
00254         }
00255       }
00256       value_matrix.push_back(row_data);
00257     }
00258     // Check the matrix for monotonicity
00259     if (!value_matrix.empty() && !value_matrix[0].empty()) {
00260       // Check the monotonic incrementality of rows (by output load capacitance, index 2)
00261       for (size_t i = 0; i < value_matrix.size(); ++i) {
00262         for (size_t j = 1; j < value_matrix[i].size(); ++j) {
00263           if ((value_matrix[i][j] < value_matrix[i][j - 1] && pin["pin_name"] != arc["related_pin"])
      ||
```

```
00264               (value_matrix[i][j] == 0 && value_matrix[i][j - 1] == 0)) {
00265             // If contains "when" key, log the when value
00266             if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00267               logger_->warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
00268                             "for Cell: {} Pin: {}->{} when: \"{}\"",
00269                             timing_arc_name, i, j, value_matrix[i][j], i, j - 1, value_matrix[i][j -
    1],
00270                             cell["cell_name"].get<std::string>(),
    arc["related_pin"].get<std::string>(),
00271                             pin["pin_name"].get<std::string>(), arc["when"].get<std::string>());
00272             } else {
00273               logger_->warn("Non-monotonic (by load) '{}' values: ({}, {}) {} < ({}, {}) {} "
00274                             "for Cell: {} Pin: {}->{}",
00275                             timing_arc_name, i, j, value_matrix[i][j], i, j - 1, value_matrix[i][j -
    1],
00276                             cell["cell_name"].get<std::string>(),
    arc["related_pin"].get<std::string>(),
00277                             pin["pin_name"].get<std::string>());
00278             }
00279             is_monotonic = false;
00280           }
00281         }
00282       }
00283       if (is_slew) {
00284         // Check the monotonic incrementality of columns (by input slew, index 1)
00285         for (size_t j = 0; j < value_matrix[0].size(); ++j) {
00286           for (size_t i = 1; i < value_matrix.size(); ++i) {
00287             if ((value_matrix[i][j] < value_matrix[i - 1][j] && pin["pin_name"] != arc["related_pin"])
    ||
00288                 (value_matrix[i][j] == 0 && value_matrix[i - 1][j] == 0)) {
00289               // If contains "when" key, log the when value
00290               if (arc.contains("when") && !arc["when"].get<std::string>().empty()) {
00291                 logger_->warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
00292                               "for Cell: {} Pin: {}->{} when: \"{}\"",
00293                               timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
00294                               value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00295                               arc["related_pin"].get<std::string>(),
    pin["pin_name"].get<std::string>(),
00296                               arc["when"].get<std::string>());
00297               } else {
00298                 logger_->warn("Non-monotonic (by slew) '{}' values: ({}, {}) {} < ({}, {}) {} "
00299                               "for Cell: {} Pin: {}->{}",
00300                               timing_arc_name, i, j, value_matrix[i][j], i - 1, j,
00301                               value_matrix[i - 1][j], cell["cell_name"].get<std::string>(),
00302                               arc["related_pin"].get<std::string>(),
    pin["pin_name"].get<std::string>());
00303               }
00304               is_monotonic = false;
00305             }
00306           }
00307         }
00308       }
00309     } else {
00310       logger_->warn(
00311           "Empty or invalid 'values' array found for cell: '{}', pin: '{}', related_pin: '{}', "
00312           "timing_arc: '{}'",
00313           cell["cell_name"].get<std::string>(), pin["pin_name"].get<std::string>(),
00314           arc["related_pin"].get<std::string>(), timing_arc_name);
00315     }
00316   }
00317   return is_monotonic;
00318 }
00319
00340 void LibFile::mono(const bool is_slew) {
00341   /*
00342    * Use this command to check the following data in the current library to ensure
00343    * the tables are monotonically increasing with respect to output load:
00344    * cell_rise retaining_rise
00345    * cell_fall retaining_fall
00346    * rise_transition retain_rise_slew
00347    * fall_transition retain_fall_slew
00348    * mpw
00349    */
00350   logger_->info("Monotonicity check of '{}' starting ...", filename_);
00351
00352   if (!std::filesystem::exists(jsonname_)) {
00353     logger_->info("JSON file not found. Parsing Liberty file first.");
00354     this->parse();
00355     this->writeJsonToFile();
00356   } else {
00357     // Read the JSON file into json object
00358     std::ifstream in(jsonname_);
00359     if (!in.is_open()) {
00360       logger_->error("Could not open file '{}' for reading", jsonname_);
00361       return;
00362     }
00363     try {
```

```
00364       lib_json_ = json::parse(in);
00365     } catch (const json::parse_error &e) {
00366       logger_->error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00367       in.close();
00368       return;
00369     }
00370     in.close();
00371   }
00372
00373   std::map<std::string, bool> cell_monotonicity_status; // Track pass/fail status for each cell
00374   std::vector<std::string> failed_cells;                // List of failed cell names
00375   int total_cells = 0;
00376   int passed_cells = 0;
00377
00378   // Check the monotonicity of delay values
00379   // The index 1 (time values) must be monotonically increasing and >= 0
00380   for (const auto &cell : lib_json_["cells"]) {
00381     total_cells++;
00382     bool cell_is_monotonic = true; // Assume cell is monotonic initially
00383     std::string cell_name = cell["cell_name"].get<std::string>();
00384     cell_monotonicity_status[cell_name] =
00385         true; // Initialize to pass, will be set to false if any check fails
00386
00387     if (cell.contains("output_pins")) {
00388       for (const auto &pin : cell["output_pins"]) {
00389         if (pin.contains("timing_arcs")) {
00390           for (const auto &arc : pin["timing_arcs"]) {
00391             // Check 4 timing arc names and accumulate monotonicity status
00392             for (const auto &name : {"cell_rise", "cell_fall", "rise_transition", "fall_transition"})
    {
00393               if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00394                 cell_is_monotonic = false;
00395               }
00396             }
00397           }
00398         }
00399       }
00400     }
00401     if (cell.contains("input_pins")) {
00402       for (const auto &pin : cell["input_pins"]) {
00403         if (!pin.contains("clock")) {
00404           continue; // Skip non-clock pins
00405         }
00406         // logger_->debug("Clock pin: {}", pin["pin_name"].get<std::string>());
00407         if (pin.contains("timing_arcs")) {
00408           for (const auto &arc : pin["timing_arcs"]) {
00409             if (arc.contains("timing_type")) {
00410               // logger_->debug("Timing type: {}", arc["timing_type"].get<std::string>());
00411               if (arc["timing_type"].get<std::string>() == "min_pulse_width") {
00412                 for (const auto &name : {"rise_constraint", "fall_constraint"}) {
00413                   if (!checkTimingArcMonotonicity(cell, pin, arc, name, is_slew)) {
00414                     cell_is_monotonic = false;
00415                   }
00416                 }
00417               }
00418             }
00419           }
00420         }
00421       }
00422     }
00423     // Update cell status based on all checks
00424     cell_monotonicity_status[cell_name] = cell_is_monotonic;
00425     if (cell_is_monotonic) {
00426       passed_cells++;
00427     } else {
00428       failed_cells.push_back(cell_name);
00429     }
00430   }
00431   logger_->info("Monotonicity check of '{}' completed.", filename_);
00432
00433   // Output summary statistics
00434   logger_->info("validate_monotonicity : {} out of {} cells passed", passed_cells, total_cells);
00435   logger_->info("validate_monotonicity : {} out of {} cells failed", total_cells - passed_cells,
00436                 total_cells);
00437   if (!failed_cells.empty()) {
00438     std::stringstream failed_cells_ss;
00439     for (size_t i = 0; i < failed_cells.size(); ++i) {
00440       failed_cells_ss « failed_cells[i];
00441       if (i < failed_cells.size() - 1) {
00442         failed_cells_ss « ", "; // Add comma if not the last element
00443       }
00444     }
00445     logger_->info("Failed cell list : {}", failed_cells_ss.str());
00446   }
00447 }
00448
00467 void LibFile::supercell(const int chain_length, const std::vector<std::string> &cell_names) {
```

```
00468   /*
00469    * Supercells are named as follows:
00470    *   <cellname>__X<chain_length>__<input_pin>__<output_pin>
00471    */
00472   logger_->info("Creating supercells for '{}'", filename_);
00473
00474   if (!std::filesystem::exists(jsonname_)) {
00475     logger_->info("JSON file not found. Parsing Liberty file first.");
00476     this->parse();
00477     this->writeJsonToFile();
00478   } else {
00479     // Read the JSON file into json object
00480     std::ifstream in(jsonname_);
00481     if (!in.is_open()) {
00482       logger_->error("Could not open file '{}' for reading", jsonname_);
00483       return;
00484     }
00485     try {
00486       lib_json_ = json::parse(in);
00487     } catch (const json::parse_error &e) {
00488       logger_->error("JSON parsing error in file '{}': {}", jsonname_, e.what());
00489       in.close();
00490       return;
00491     }
00492     in.close();
00493   }
00494
00495   // write to .map file
00496   std::ofstream out(basename_ + ".map");
00497   if (!out.is_open()) {
00498     logger_->error("Could not open file '{}' for writing", basename_ + ".map");
00499     return;
00500   }
00501   // if celll_names is empty, create supercells for all cells
00502   // else create supercells for only the specified cells
00503
00504   // Check if we're processing specific cells or all cells
00505   bool process_all_cells = cell_names.empty();
00506
00507   if (process_all_cells) {
00508     logger_->info("Creating supercells for ALL cells in '{}'", filename_);
00509   } else {
00510     logger_->info("Creating supercells for {} specified cells in '{}'", cell_names.size(), filename_);
00511   }
00512
00513   // Create a set for faster lookups if we have specific cell names
00514   std::unordered_set<std::string> cell_set;
00515   std::unordered_set<std::string> found_cells;
00516   if (!process_all_cells) {
00517     cell_set.insert(cell_names.begin(), cell_names.end());
00518   }
00519
00520   for (const auto &cell : lib_json_["cells"]) {
00521     bool is_sequential = false;
00522     std::string cell_name = cell["cell_name"].get<std::string>();
00523
00524     // Skip cells not in the specified list
00525     if (!process_all_cells && cell_set.find(cell_name) == cell_set.end()) {
00526       continue;
00527     }
00528
00529     // Mark this cell as found
00530     if (!process_all_cells) {
00531       found_cells.insert(cell_name);
00532     }
00533
00534     logger_->debug("Creating supercells for cell: '{}'", cell_name);
00535
00536     std::vector<std::string> output_pins;
00537     std::vector<std::string> input_pins;
00538     // Extract input and output pins
00539     if (cell.contains("output_pins")) {
00540       for (const auto &pin : cell["output_pins"]) {
00541         output_pins.push_back(pin["pin_name"].get<std::string>());
00542       }
00543     }
00544     if (cell.contains("input_pins")) {
00545       for (const auto &pin : cell["input_pins"]) {
00546         if (pin.contains("clock")) {
00547           is_sequential = true;
00548           // clock pin not use for supercell
00549           continue;
00550         }
00551         input_pins.push_back(pin["pin_name"].get<std::string>());
00552       }
00553     }
00554
```

```
00555     // create supercells for all combinations of input and output pins
00556     for (const auto &output_pin : output_pins) {
00557       for (const auto &input_pin : input_pins) {
00558         if (is_sequential) {
00559           const int chain_len = 1;
00560           std::string supercell_name =
00561               cell_name + "__X" + std::to_string(chain_len) + "__" + input_pin + "__" + output_pin;
00562           out << cell_name << " " << supercell_name << std::endl;
00563         } else {
00564           std::string supercell_name =
00565               cell_name + "__X" + std::to_string(chain_length) + "__" + input_pin + "__" + output_pin;
00566           out << cell_name << " " << supercell_name << std::endl;
00567         }
00568       }
00569     }
00570   }
00571
00572   // Check for cells that were specified but not found
00573   if (!process_all_cells) {
00574     std::vector<std::string> not_found_cells;
00575     for (const auto &requested_cell : cell_names) {
00576       if (found_cells.find(requested_cell) == found_cells.end()) {
00577         not_found_cells.push_back(requested_cell);
00578       }
00579     }
00580
00581     // Output warning for cells that weren't found
00582     if (!not_found_cells.empty()) {
00583       std::string missing_cells = not_found_cells[0];
00584       for (size_t i = 1; i < not_found_cells.size(); ++i) {
00585         missing_cells += ", " + not_found_cells[i];
00586       }
00587       logger_->warn("{} specified cell{} not found in the library: {}", not_found_cells.size(),
00588                     not_found_cells.size() > 1 ? "s were" : " was", missing_cells);
00589     }
00590   }
00591
00592   out.close();
00593   logger_->info("Supercell creation complete in '{}'", basename_ + ".map");
00594 }
00595
00596 void LibFile::verilog(const int chain_length, const std::vector<std::string> &cell_names) {
00597   logger_->info("Creating Verilog for '{}'", filename_);
00598
00599   this->supercell(chain_length, cell_names);
00600
00601   // write to .v file
00602   std::ofstream out(basename_ + ".v");
00603   if (!out.is_open()) {
00604     logger_->error("Could not open file '{}' for writing", basename_ + ".v");
00605     return;
00606   }
00607
00608   // Read the .map file into a vector to preserve all entries and their order
00609   std::vector<std::pair<std::string, std::string>> supercell_entries;
00610   std::ifstream in(basename_ + ".map");
00611   if (!in.is_open()) {
00612     logger_->error("Could not open file '{}' for reading", basename_ + ".map");
00613     return;
00614   }
00615
00616   std::string line;
00617   while (std::getline(in, line)) {
00618     std::istringstream iss(line);
00619     std::string cell_name, supercell_name;
00620     if (!(iss >> cell_name >> supercell_name)) {
00621       logger_->warn("Invalid line format in map file: '{}'", line);
00622       continue;
00623     }
00624     supercell_entries.push_back({cell_name, supercell_name});
00625   }
00626   in.close();
00627   logger_->info("Read {} supercells from '{}'", supercell_entries.size(), basename_ + ".map");
00628
00629   // Generate verilog module for each supercell
00630   for (const auto &supercell_entry : supercell_entries) {
00631     // Check if the cell is sequential
00632     bool is_sequential = false;
00633     // Count the number of instances will be created in verilog
00634     int instance_count = 0;
00635     // Get original cell name from pair
00636     const std::string &cell_name = supercell_entry.first;
00637     // Get supercell name(as well as module name) from pair
00638     const std::string &module_name = supercell_entry.second;
00639
00640     logger_->info("Creating Module: '{}' from Cell '{}", module_name, cell_name);
00641
```

```
00642     // Get the cell JSON object
00643     json cell_json;
00644     for (const auto &cell : lib_json_["cells"]) {
00645       if (cell["cell_name"].get<std::string>() == cell_name) {
00646         cell_json = cell;
00647         break;
00648       }
00649     }
00650
00651     // Get input/output pins from the cell JSON object
00652     std::vector<std::string> input_pins;
00653     std::vector<std::string> output_pins;
00654     std::stringstream input_pins_ss;
00655     std::stringstream output_pins_ss;
00656     if (cell_json.contains("input_pins")) {
00657       for (const auto &pin : cell_json["input_pins"]) {
00658         if (pin.contains("clock")) {
00659           is_sequential = true;
00660         }
00661         input_pins.push_back(pin["pin_name"].get<std::string>());
00662         input_pins_ss << pin["pin_name"].get<std::string>() << ", ";
00663       }
00664     }
00665     if (cell_json.contains("output_pins")) {
00666       for (const auto &pin : cell_json["output_pins"]) {
00667         output_pins.push_back(pin["pin_name"].get<std::string>());
00668         output_pins_ss << pin["pin_name"].get<std::string>() << ", ";
00669       }
00670     }
00671     logger_->debug("Input pins set: {}", input_pins_ss.str());
00672     logger_->debug("Output pins set: {}", output_pins_ss.str());
00673
00674     // Sequential cells will have only one instance
00675     // Combinational cells will have instances equal to chain length
00676     if (is_sequential) {
00677       instance_count = 1;
00678     } else {
00679       instance_count = chain_length;
00680     }
00681
00682     // Create ANSI port list
00683     std::string port_list_text = "(";
00684     // Add input ports
00685     bool isFirstPort = true;
00686     for (const auto &input_pin : input_pins) {
00687       if (!isFirstPort) {
00688         port_list_text += ", ";
00689       }
00690       port_list_text += "input " + input_pin;
00691       isFirstPort = false;
00692     }
00693     // Add output ports
00694     for (const auto &output_pin : output_pins) {
00695       if (!isFirstPort) {
00696         port_list_text += ", ";
00697       }
00698       port_list_text += "output " + output_pin;
00699       isFirstPort = false;
00700     }
00701     port_list_text += ")";
00702     logger_->debug("ANSI Port list: {}", port_list_text);
00703
00704     // Creat full module header text
00705     std::string fullModuleText = "module " + module_name + port_list_text + ";\nendmodule";
00706     logger_->debug("Full module text: {}", fullModuleText);
00707
00708     // Generate the syntax tree for the module using slang library
00709     auto tree = slang::syntax::SyntaxTree::fromText(fullModuleText);
00710     if (tree) {
00711       // Add ports to the syntax tree
00712       ModuleRewriter rewriter(input_pins, output_pins, supercell_entry, instance_count, logger_);
00713
00714       tree = rewriter.transform(tree);
00715       rewriter.visit(tree->root());
00716
00717       // Output the transformed syntax tree
00718       out << "`timescale 1ns/1ps" << std::endl;
00719       out << slang::syntax::SyntaxPrinter::printFile(*tree) << std::endl << std::endl;
00720     } else {
00721       logger_->error("Failed to create syntax tree for module '{}'", module_name);
00722       continue;
00723     }
00724   }
00725
00726   out.close();
00727   logger_->info("Verilog creation complete in '{}'", basename_ + ".v");
00728 }
```

## 6.30 src/lib_group.cpp File Reference

```
#include "lib_group.hpp"
```
Include dependency graph for lib_group.cpp:

## 6.31 lib_group.cpp

Go to the documentation of this file.
```
00001 #include "lib_group.hpp"
00002
00003 LibGroup::LibGroup(si2drGroupIdT group, si2drErrorT &err) : group_(group), err_(err) {}
00004
00005 LibGroup::~LibGroup() {}
00006
00007 std::string LibGroup::getName() {
00008   si2drNamesIdT names = si2drGroupGetNames(group_, &err_);
00009   si2drStringT name = si2drIterNextName(names, &err_);
00010   si2drIterQuit(names, &err_);
00011   return name ? std::string(name) : std::string();
00012 }
00013
00014 std::string LibGroup::getType() {
00015   si2drStringT type = si2drGroupGetGroupType(group_, &err_);
00016   return type ? std::string(type) : std::string();
00017 }
00018
00019 si2drAttrsIdT LibGroup::getAttrs() { return si2drGroupGetAttrs(group_, &err_); }
00020
00021 si2drGroupsIdT LibGroup::getGroups() { return si2drGroupGetGroups(group_, &err_); }
```

## 6.32 src/main.cpp File Reference

```
#include <filesystem>
#include <iostream>
#include <string>
#include <thread>
#include <vector>
#include "CLI/CLI.hpp"
#include "si2dr_liberty.h"
#include "spdlog/sinks/basic_file_sink.h"
#include "spdlog/sinks/stdout_color_sinks.h"
#include "spdlog/spdlog.h"
#include "compare.hpp"
#include "lib_file.hpp"
#include "verilog_utils.hpp"
#include "version.h"
```
Include dependency graph for main.cpp:

### Functions

- void printInfo ()

  *Sets up and configures the global logger and prints application information.*
- void parseLibFile (const std::string &library_path, const std::string log_file_name)

  *Parses a library file and generates corresponding output.*
- void monoCheckLibFile (const std::string &library_path, const std::string log_file_name, bool is_slew)

  *Performs a monotonicity check on a library file.*

- void supercellLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)

  *Creates supercell map structures from a Liberty library file.*
- void verilogLibFile (const std::string &library_path, const std::string &log_file_name, int chain_length, const std::vector< std::string > &cell_names)

  *Generates Verilog files from a library file for specified cells.*
- void compareLibFiles (const std::string &ref_lib, const std::string &comp_lib, const double reltol, const double abstol, std::string &report_file_name)

  *Compares two library files and generates a detailed comparison report.*
- void funcLibFile (const std::string &ref_file, const std::string &comp_file, const std::vector< std::string > &cell_names)
- int main (int argc, char *argv[])

## 6.32.1 Function Documentation

### 6.32.1.1 compareLibFiles()

```
void compareLibFiles (
            const std::string & ref_lib,
            const std::string & comp_lib,
            const double reltol,
            const double abstol,
            std::string & report_file_name )
```

Compares two library files and generates a detailed comparison report.

This function compares a reference library file with another library file, performing validation checks based on specified tolerance parameters. The comparison results are written to a report file in markdown or text format.

**Parameters**

| | |
|---|---|
| *ref_lib* | Path to the reference library file to use as the baseline |
| *comp_lib* | Path to the library file to compare against the reference |
| *reltol* | Relative tolerance for numerical comparisons (must be >= 0.0) |
| *abstol* | Absolute tolerance for numerical comparisons |
| *report_file_name* | [in,out] Name of the file to write the comparison report to. If empty, defaults to [comp_lib_basename].cmp.md. If provided but doesn't end with .txt or .md, .md will be appended. |

**Note**

The function will log an error and return without comparing if reltol is invalid.

Log files will be created with the naming pattern [library_basename].cmp.log

The function uses the LibraryComparator class to perform the actual comparison.

Definition at line 216 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.32.1.2 funcLibFile()

```
void funcLibFile (
            const std::string & ref_file,
            const std::string & comp_file,
            const std::vector< std::string > & cell_names )
```

Definition at line 242 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.32.1.3 main()

```
int main (
            int argc,
            char * argv[] )
```

Definition at line 260 of file main.cpp.

Here is the call graph for this function:

#### 6.32.1.4 monoCheckLibFile()

```
void monoCheckLibFile (
            const std::string & library_path,
            const std::string log_file_name,
            bool is_slew )
```

Performs a monotonicity check on a library file.

This function loads a library file and performs monotonicity validation on it. Results are logged to a file. If no log file name is provided, it generates one based on the library file name.

**Parameters**

| | |
|---|---|
| *library_path* | Path to the library file to check |
| *log_file_name* | Name of the log file (optional - if empty, generates a name based on library file) |
| *is_slew* | Boolean flag indicating whether to check slew monotonicity (true) or other parameters (false) |

**Exceptions**

| | |
|---|---|
| *Any* | exceptions from the monotonicity check are caught and logged |

Definition at line 97 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.32.1.5 parseLibFile()

```
void parseLibFile (
```

```
const std::string & library_path,
const std::string log_file_name )
```

Parses a library file and generates corresponding output.

This function processes the given library file, parsing its contents and generating a JSON output. It also logs the parsing process to a specified log file or creates a default log file if none is provided.

**Parameters**

| | |
|---|---|
| *library_path* | Path to the library file that needs to be parsed |
| *log_file_name* | Optional name for the log file. If empty, a default name is generated based on the library filename with ".parse.log" extension |

**Exceptions**

| | |
|---|---|
| *The* | function catches and logs any exceptions but doesn't propagate them |

Definition at line 65 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.32.1.6 printInfo()

```
void printInfo ( )
```

Sets up and configures the global logger and prints application information.

This function performs the following operations:

1. Creates a console sink for logging with level set to INFO
2. Creates a file sink for logging with level set to DEBUG, saving to [APP_NAME].log
3. Configures a logger with both sinks and sets it as the default logger
4. Outputs basic application information:
   - Version and build timestamp
   - Author information
   - Log file location

**Note**

Uses spdlog library for logging functionality

Depends on APP_NAME, APP_VERSION, BUILD_TIMESTAMP, APP_AUTHOR, and APP_CONTACT macros

Definition at line 34 of file main.cpp.

Here is the caller graph for this function:

#### 6.32.1.7 supercellLibFile()

```
void supercellLibFile (
            const std::string & library_path,
            const std::string & log_file_name,
            int chain_length,
            const std::vector< std::string > & cell_names )
```

Creates supercell map structures from a Liberty library file.

This function reads a Liberty file, creates supercell structures based on the specified chain length and cell names, and logs the process to a file.

**Parameters**

| | |
|---|---|
| *library_path* | Path to the Liberty file to process |
| *log_file_name* | Name of the log file (if empty, defaults to "[library_name].supercell.log") |
| *chain_length* | The length of chains to create (must be >= 1) |
| *cell_names* | Vector of cell names to process for supercell generation |

**Exceptions**

| | |
|---|---|
| *May* | pass through exceptions from the LibFile::supercell method |

**Note**

The function validates the chain length and logs all activities including errors that might occur during processing

Definition at line 130 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

#### 6.32.1.8 verilogLibFile()

```
void verilogLibFile (
            const std::string & library_path,
            const std::string & log_file_name,
            int chain_length,
            const std::vector< std::string > & cell_names )
```

Generates Verilog files from a library file for specified cells.

This function processes a library file and generates Verilog representation for the specified cell names with a given chain length. The operation results are logged to a specified or default log file.

**Parameters**

| | |
|---|---|
| *library_path* | Path to the library file to process |
| *log_file_name* | Name for the log file (if empty, a default name will be generated) |
| *chain_length* | Number of cells to chain together, must be >= 1 |
| *cell_names* | Vector of cell names to generate Verilog for |

**Exceptions**

| *Catches* | any exceptions from the verilog generation process and logs them |
|---|---|

Definition at line 171 of file main.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

## 6.33 main.cpp

Go to the documentation of this file.

```
00001 // ./src/main.cpp
00002 #include <filesystem>
00003 #include <iostream>
00004 #include <string>
00005 #include <thread>
00006 #include <vector>
00007
00008 #include "CLI/CLI.hpp"
00009 #include "si2dr_liberty.h"
00010 #include "spdlog/sinks/basic_file_sink.h"
00011 #include "spdlog/sinks/stdout_color_sinks.h"
00012 #include "spdlog/spdlog.h"
00013
00014 #include "compare.hpp"
00015 #include "lib_file.hpp"
00016 #include "verilog_utils.hpp"
00017 #include "version.h"
00018
00034 void printInfo() {
00035   // Set Global Logger
00036   auto console_sink = std::make_shared<spdlog::sinks::stdout_color_sink_mt>();
00037   console_sink->set_level(spdlog::level::info);
00038   std::string app_name = APP_NAME;
00039   auto file_sink = std::make_shared<spdlog::sinks::basic_file_sink_mt>(app_name + ".log", true);
00040   file_sink->set_level(spdlog::level::debug);
00041
00042   std::vector<spdlog::sink_ptr> sinks{console_sink, file_sink};
00043   auto logger = std::make_shared<spdlog::logger>(APP_NAME, sinks.begin(), sinks.end());
00044   logger->set_level(spdlog::level::debug);
00045   spdlog::set_default_logger(logger);
00046
00047   spdlog::info("Version {}, Built: {}", APP_VERSION, BUILD_TIMESTAMP);
00048   spdlog::info("Author: {}, Email: {}", APP_AUTHOR, APP_CONTACT);
00049   spdlog::info("Global log file in: '{}'", app_name + ".log");
00050 }
00051
00065 void parseLibFile(const std::string &library_path, const std::string log_file_name) {
00066   std::string logname = log_file_name.empty()
00067                             ? std::filesystem::path(library_path).stem().string() + ".parse.log"
00068                             : log_file_name;
00069   LibFile libfile(library_path, logname);
00070
00071   libfile.logger_->info("Starting parse for file: '{}' ...", library_path);
00072
00073   try {
00074     libfile.parse();
00075     libfile.writeJsonToFile();
00076     libfile.logger_->info("Successfully parsed file: '{}'", library_path);
00077   } catch (const std::exception &e) {
00078     libfile.logger_->error("Error parsing file '{}': {}", library_path, e.what());
00079   }
00080 }
00081
00097 void monoCheckLibFile(const std::string &library_path, const std::string log_file_name, bool is_slew)
      {
00098   std::string logname = log_file_name.empty()
00099                             ? std::filesystem::path(library_path).stem().string() + ".mono.log"
00100                             : log_file_name;
00101   LibFile libfile(library_path, logname);
00102
00103   libfile.logger_->info("Starting monotonicity check for file: '{}', input slew: {}", library_path,
00104                         is_slew);
00105
00106   try {
00107     libfile.mono(is_slew);
```

```
00108     libfile.logger_->info("Successfully completed monotonicity check for file: '{}'", library_path);
00109   } catch (const std::exception &e) {
00110     libfile.logger_->error("Error during monotonicity check for file '{}': {}", library_path,
      e.what());
00111   }
00112 }
00113
00130 void supercellLibFile(const std::string &library_path, const std::string &log_file_name,
00131                       int chain_length, const std::vector<std::string> &cell_names) {
00132   std::string logname = log_file_name.empty()
00133                             ? std::filesystem::path(library_path).stem().string() + ".supercell.log"
00134                             : log_file_name;
00135   LibFile libfile(library_path, logname);
00136
00137   // Check chain length validity
00138   if (chain_length < 1) {
00139     libfile.logger_->error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00140     return;
00141   }
00142   std::stringstream ss;
00143   for (const auto &cell : cell_names) {
00144     ss « cell « ", ";
00145   }
00146   libfile.logger_->info("Starting supercell generation for file: '{}', chain length: {}, cells: {}",
00147                         library_path, chain_length, ss.str());
00148   try {
00149     libfile.supercell(chain_length, cell_names);
00150     libfile.logger_->info("Successfully generated supercells for file: '{}'", library_path);
00151   } catch (const std::exception &e) {
00152     libfile.logger_->error("Error during supercell generation for file '{}': {}", library_path,
00153                            e.what());
00154   }
00155 }
00156
00171 void verilogLibFile(const std::string &library_path, const std::string &log_file_name, int
      chain_length,
00172                     const std::vector<std::string> &cell_names) {
00173   std::string logname = log_file_name.empty()
00174                             ? std::filesystem::path(library_path).stem().string() + ".verilog.log"
00175                             : log_file_name;
00176   LibFile libfile(library_path, logname);
00177
00178   // Check chain length validity
00179   if (chain_length < 1) {
00180     libfile.logger_->error("Invalid chain length: {}. Chain length must be >= 1.", chain_length);
00181     return;
00182   }
00183   std::stringstream ss;
00184   for (const auto &cell : cell_names) {
00185     ss « cell « ", ";
00186   }
00187   libfile.logger_->info("Starting Verilog generation for file: '{}', chain length: {}, cells: {}",
00188                         library_path, chain_length, ss.str());
00189   try {
00190     libfile.verilog(chain_length, cell_names);
00191     libfile.logger_->info("Successfully generated Verilog for file: '{}'", library_path);
00192   } catch (const std::exception &e) {
00193     libfile.logger_->error("Error during Verilog generation for file '{}': {}", library_path,
      e.what());
00194   }
00195 }
00196
00216 void compareLibFiles(const std::string &ref_lib, const std::string &comp_lib, const double reltol,
00217                      const double abstol, std::string &report_file_name) {
00218   std::string ref_logname = std::filesystem::path(ref_lib).stem().string() + ".cmp.log";
00219   std::string comp_logname = std::filesystem::path(comp_lib).stem().string() + ".cmp.log";
00220   LibFile ref_libfile(ref_lib, ref_logname), comp_libfile(comp_lib, comp_logname);
00221
00222   // Check relative tolerance validity
00223   if (reltol < 0.0) {
00224     spdlog::error("Invalid relative tolerance: {}. Relative tolerance must be >= 0.0.", reltol);
00225     return;
00226   }
00227   if (report_file_name.empty()) {
00228     report_file_name = comp_libfile.basename_ + ".cmp.md";
00229   } else if (std::filesystem::path(report_file_name).extension() != ".txt" &&
00230              std::filesystem::path(report_file_name).extension() != ".md") {
00231     // report file name not end in .txt or .md, warn user and append .md
00232     report_file_name = report_file_name + ".md";
00233     spdlog::warn("Report file name does not end in .txt or .md. Report will be written to '{}'",
00234                  report_file_name);
00235   }
00236
00237   LibraryComparator comparator(ref_libfile, comp_libfile, reltol, abstol);
00238   comparator.generateReport(report_file_name);
00239   spdlog::info("Comparison completed. Report written to: '{}'", report_file_name);
00240 }
```

```
00241
00242 void funcLibFile(const std::string &ref_file, const std::string &comp_file,
00243                   const std::vector<std::string> &cell_names) {
00244   // Check if the files are Liberty or Verilog
00245   std::string ref_ext = std::filesystem::path(ref_file).extension();
00246   std::string comp_ext = std::filesystem::path(comp_file).extension();
00247   for (const auto &cell : cell_names) {
00248     spdlog::info("Checking functional equivalence for cell: '{}'", cell);
00249     if (ref_ext == ".v") {
00250       spdlog::info("Reference file in Verilog format.");
00251       getAST(ref_file, cell);
00252     }
00253     if (comp_ext == ".v") {
00254       spdlog::info("Comparison file in Verilog format.");
00255       getAST(comp_file, cell);
00256     }
00257   }
00258 }
00259
00260 int main(int argc, char *argv[]) {
00261   // Parse command line arguments
00262   std::vector<std::string> library_paths; // Support multiple files
00263   std::string log_file_name = "";
00264
00265   // Command line parameter parsing using CLI11
00266   CLI::App app{APP_NAME};
00267   app.set_version_flag("-v,--version", APP_VERSION);
00268
00269   // Add subcommand for parse mode
00270   CLI::App *parse_cmd = app.add_subcommand("parse", "Parse the Liberty file and write JSON to a
      file");
00271   parse_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00272       ->check(CLI::ExistingFile)
00273       ->required();
00274   parse_cmd->add_option("-l,--log", log_file_name,
00275                         "Specify the log file name. Default: <basename>.parse.log");
00276   parse_cmd->callback([&] {
00277     printInfo();
00278     // Check if multi files
00279     if (library_paths.size() > 1) {
00280       spdlog::info("Running sequential parsing for {} files.", library_paths.size());
00281       spdlog::info("Each library will write to its own log file.");
00282       // Sequential parsing
00283       for (const auto &library_path : library_paths) {
00284         parseLibFile(library_path, log_file_name = "");
00285       }
00286     } else {
00287       parseLibFile(library_paths[0], log_file_name);
00288     }
00289   });
00290
00291   // Add subcommand for mono check mode
00292   bool is_slew = false;
00293   CLI::App *mono_cmd = app.add_subcommand("mono", "Check the monotonicity of timing arc values");
00294   mono_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00295       ->check(CLI::ExistingFile)
00296       ->required();
00297   mono_cmd->add_option("-l,--log", log_file_name,
00298                        "Specify the log file name. Default: <basename>.mono.log");
00299   mono_cmd->add_flag("-s,--slew", is_slew, "Specify that monotonicity checks also include input
      slew.");
00300   mono_cmd->callback([&] {
00301     printInfo();
00302     // Check if multi files
00303     if (library_paths.size() > 1) {
00304       spdlog::info("Running multi-threaded monotonicity check for {} files.", library_paths.size());
00305       spdlog::info("Each thread will write to its own log file.");
00306
00307       // Sequential json file check
00308       for (const auto &library_path : library_paths) {
00309         if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00310           spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00311           parseLibFile(library_path, log_file_name = "");
00312         }
00313       }
00314       spdlog::info("All JSON files prepared.");
00315       // Parallel monotonicity check
00316       std::vector<std::thread> threads;
00317       for (const auto &library_path : library_paths) {
00318         threads.emplace_back(monoCheckLibFile, library_path, log_file_name = "", is_slew);
00319       }
00320       for (auto &thread : threads) {
00321         thread.join();
00322       }
00323       spdlog::info("All threads completed.");
00324     } else {
00325       monoCheckLibFile(library_paths[0], log_file_name, is_slew);
```

```
00326     }
00327   });
00328
00329   // Add subcommand for compare mode
00330   std::string ref_lib, comp_lib, report_file_name;
00331   CLI::App *compare_cmd = app.add_subcommand(
00332       "compare", "Compare the comparison library against the reference library and report
      differences");
00333   compare_cmd->add_option("--ref", ref_lib, "Specify the reference library file")
00334       ->check(CLI::ExistingFile)
00335       ->required();
00336   compare_cmd->add_option("--comp", comp_lib, "Specify the comparison library file")
00337       ->check(CLI::ExistingFile)
00338       ->required();
00339   double abstol = 0.002;
00340   compare_cmd->add_option("--abstol", abstol,
00341                           "Specify the absolute tolerance for comparison. Default: 0.002ns");
00342   double reltol = 0.02;
00343   compare_cmd->add_option("--reltol", reltol,
00344                           "Specify the relative tolerance for comparison. Default: 0.02/2.0%");
00345   compare_cmd->add_option("--report", report_file_name,
00346                           "Specify the report file name. Default: <comp_lib>.cmp.md");
00347   compare_cmd->callback([&] {
00348     printInfo();
00349     compareLibFiles(ref_lib, comp_lib, reltol, abstol, report_file_name);
00350   });
00351
00352   // Add subcommand for supercell generation
00353   int chain_length = 1;
00354   CLI::App *supercell_cmd =
00355       app.add_subcommand("supercell", "Generate supercells for the given Liberty file");
00356   supercell_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00357       ->check(CLI::ExistingFile)
00358       ->required();
00359   supercell_cmd->add_option("-l,--log", log_file_name,
00360                             "Specify the log file name. Default: <basename>.supercell.log");
00361   supercell_cmd->add_option("-c,--chain", chain_length,
00362                             "Specify the chain length for supercell generation. Default: 1");
00363   std::vector<std::string> cell_names = {}; // "CMPE42D1" "AN2D0", "DFQD1"
00364   supercell_cmd->add_option("--cells", cell_names, "Specify the cell names to generate supercells
      for");
00365   supercell_cmd->callback([&] {
00366     printInfo();
00367     // Check if multi files
00368     if (library_paths.size() > 1) {
00369       spdlog::info("Running multi-threaded supercell generation for {} files.", library_paths.size());
00370       spdlog::info("Each library will write to its own log file.");
00371
00372       // Sequential json file check
00373       for (const auto &library_path : library_paths) {
00374         if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00375           spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00376           parseLibFile(library_path, log_file_name = "");
00377         }
00378       }
00379       spdlog::info("All JSON files prepared.");
00380       // Parallel supercell generation
00381       std::vector<std::thread> threads;
00382       for (const auto &library_path : library_paths) {
00383         threads.emplace_back(supercellLibFile, library_path, log_file_name = "", chain_length,
00384                              cell_names);
00385       }
00386       for (auto &thread : threads) {
00387         thread.join();
00388       }
00389       spdlog::info("All threads completed.");
00390     } else {
00391       supercellLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00392     }
00393   });
00394
00395   // Add subcommand for zlibboost
00396   CLI::App *zlibboost_cmd =
00397       app.add_subcommand("zlibboost", "ZlibBoost - Multi-threaded Library Processing Tool");
00398   std::string config_dir = "/home/songzx/Projects/zlibboost/config.tcl";
00399   std::string python_dir = "/home/guocj/anaconda3/envs/myenv/bin/python";
00400   std::string main_py_dir = "/home/songzx/Projects/zlibboost/zlibboost.py";
00401   zlibboost_cmd->add_option(
00402       "-c, --config", config_dir,
00403       "Specify the configuration TCL file. Default: /home/songzx/Projects/zlibboost/config.tcl");
00404   zlibboost_cmd->add_option(
00405       "--python", python_dir,
00406       "Specify the python directory. Default: /home/guocj/anaconda3/envs/myenv/bin/python");
00407   zlibboost_cmd->add_option(
00408       "--main", main_py_dir,
00409       "Specify the main python script directory. Default:
      /home/songzx/Projects/zlibboost/zlibboost.py");
```

```
00410   zlibboost_cmd->callback([&] {
00411     printInfo();
00412     // Run the ZlibBoost tool
00413     std::string command = python_dir + " " + main_py_dir + " -c " + config_dir;
00414     spdlog::info("Running ZlibBoost with command: '{}'", command);
00415     int ret = std::system(command.c_str());
00416     if (ret == 0) {
00417       spdlog::info("ZlibBoost completed successfully.");
00418     } else {
00419       spdlog::error("ZlibBoost failed with return code: {}", ret);
00420     }
00421   });
00422
00423   // Add subcommand for clear
00424   CLI::App *clear_cmd =
00425       app.add_subcommand("clear", "Clear the log, JSON, map, markdown files in this directory");
00426   clear_cmd->callback([&] {
00427     printInfo();
00428     std::filesystem::path current_dir = std::filesystem::current_path();
00429     for (const auto &entry : std::filesystem::directory_iterator(current_dir)) {
00430       if (entry.path().extension() == ".log" || entry.path().extension() == ".json" ||
00431           entry.path().extension() == ".map" || entry.path().extension() == ".md") {
00432         spdlog::info("Removing file: '{}'", entry.path().string());
00433         std::filesystem::remove(entry.path());
00434       }
00435     }
00436     spdlog::info("All log, JSON, map, markdown files cleared.");
00437   });
00438
00439   // Add subcommand for Verilog generation
00440   CLI::App *verilog_cmd = app.add_subcommand("verilog", "Generate Verilog file for given Liberty
      file");
00441   verilog_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00442       ->check(CLI::ExistingFile)
00443       ->required();
00444   verilog_cmd->add_option("-l,--log", log_file_name,
00445                           "Specify the log file name. Default: <basename>.verilog.log");
00446   verilog_cmd->add_option("-c,--chain", chain_length,
00447                           "Specify the chain length for verilog generation. Default: 1");
00448   verilog_cmd->add_option("--cells", cell_names, "Specify the cell names to generate Verilog for");
00449   verilog_cmd->callback([&] {
00450     printInfo();
00451     // Check if multi files
00452     if (library_paths.size() > 1) {
00453       spdlog::info("Running multi-threaded Verilog generation for {} files.", library_paths.size());
00454       spdlog::info("Each library will write to its own log file.");
00455
00456       // Sequential json file check
00457       for (const auto &library_path : library_paths) {
00458         if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00459           spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00460           parseLibFile(library_path, log_file_name = "");
00461         }
00462       }
00463       spdlog::info("All JSON files prepared.");
00464       // Parallel Verilog generation
00465       std::vector<std::thread> threads;
00466       for (const auto &library_path : library_paths) {
00467         threads.emplace_back(verilogLibFile, library_path, log_file_name = "", chain_length,
      cell_names);
00468       }
00469       for (auto &thread : threads) {
00470         thread.join();
00471       }
00472       spdlog::info("All threads completed.");
00473     } else {
00474       verilogLibFile(library_paths[0], log_file_name, chain_length, cell_names);
00475     }
00476   });
00477
00478   // Add subcommand for SPICE generation
00479   CLI::App *spice_cmd = app.add_subcommand("spice", "Generate SPICE file for given Liberty file");
00480   spice_cmd->add_option("library_path", library_paths, "Specify the library file to process")
00481       ->check(CLI::ExistingFile)
00482       ->required();
00483   spice_cmd->add_option("-l,--log", log_file_name,
00484                         "Specify the log file name. Default: <basename>.spice.log");
00485   spice_cmd->add_option("--cells", cell_names, "Specify the cell names to generate SPICE for");
00486   spice_cmd->callback([&] {
00487     printInfo();
00488     // Check if multi files
00489     if (library_paths.size() > 1) {
00490       spdlog::info("Running multi-threaded SPICE generation for {} files.", library_paths.size());
00491       spdlog::info("Each library will write to its own log file.");
00492
00493       // Sequential json file check
00494       for (const auto &library_path : library_paths) {
```

```
00495         if (!std::filesystem::exists(std::filesystem::path(library_path).stem().string() + ".json")) {
00496           spdlog::info("JSON file not found for '{}'. Parsing Liberty file first.", library_path);
00497           parseLibFile(library_path, log_file_name = "");
00498         }
00499       }
00500       spdlog::info("All JSON files prepared.");
00501       // Parallel SPICE generation
00502       std::vector<std::thread> threads;
00503       for (const auto &library_path : library_paths) {
00504         // threads.emplace_back(spiceLibFile, library_path, log_file_name = "", cell_names);
00505       }
00506       for (auto &thread : threads) {
00507         thread.join();
00508       }
00509       spdlog::info("All threads completed.");
00510     } else {
00511       // spiceLibFile(library_paths[0], log_file_name, cell_names);
00512     }
00513   });
00514
00515   // Add subcommand for funtional equivalence check
00516   CLI::App *func_cmd =
00517       app.add_subcommand("func", "Check functional equivalence of two Liberty files or Verilog
      files");
00518   std::string ref_file, comp_file;
00519   func_cmd->add_option("--ref", ref_file, "Specify the reference Liberty or Verilog file")
00520       ->check(CLI::ExistingFile)
00521       ->required();
00522   func_cmd->add_option("--comp", comp_file, "Specify the comparison Liberty or Verilog file")
00523       ->check(CLI::ExistingFile)
00524       ->required();
00525   func_cmd->add_option("--cells", cell_names,
00526                        "Specify the cell names to check functional equivalence for");
00527   func_cmd->callback([&] {
00528     printInfo();
00529     funcLibFile(ref_file, comp_file, cell_names);
00530   });
00531
00532   CLI11_PARSE(app, argc, argv);
00533
00534   // End of program
00535   char hostname[256];
00536   gethostname(hostname, sizeof(hostname));
00537
00538   auto now = std::chrono::system_clock::now();
00539   auto time_now = std::chrono::system_clock::to_time_t(now);
00540   std::stringstream ss;
00541   ss « std::put_time(std::localtime(&time_now), "%c");
00542
00543   spdlog::info("ZlibValidation exited on '{}' at {}", hostname, ss.str());
00544   return 0;
00545 }
```

## 6.34 src/verilog_utils.cpp File Reference

```
#include <unordered_set>
#include "spdlog/spdlog.h"
#include "verilog_utils.hpp"
```

Include dependency graph for verilog_utils.cpp:

### Functions

- void getAST (const std::string &verilog_file, const std::string &cell)

### 6.34.1 Function Documentation

#### 6.34.1.1 getAST()

```
void getAST (
              const std::string & verilog_file,
              const std::string & cell )
```

Definition at line 410 of file verilog_utils.cpp.

Here is the call graph for this function: Here is the caller graph for this function:

## 6.35 verilog_utils.cpp

Go to the documentation of this file.

```
00001 #include <unordered_set>
00002
00003 #include "spdlog/spdlog.h"
00004
00005 #include "verilog_utils.hpp"
00006
00007 // Adding a generic handler method to record all visited nodes and increment the depth counter
00008 void VerilogVisitor::handle(const slang::syntax::SyntaxNode &node) {
00009   std::string indent(depth_ * 2, ' ');
00010   spdlog::debug("{}Node: {}", indent, slang::syntax::toString(node.kind));
00011
00012   // Increase depth, visit child nodes, then decrease depth
00013   depth_++;
00014   visitDefault(node);
00015   depth_--;
00016 }
00017
00018 // Handle module declarations
00019 void VerilogVisitor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00020   std::string indent(depth_ * 2, ' ');
00021
00022   if (module.header && module.header->name.valid()) {
00023     std::string_view moduleName = module.header->name.valueText();
00024     spdlog::info("{}Module Name:{}", indent, moduleName);
00025
00026     inTargetModule_ = !targetCell_.empty() && moduleName == targetCell_;
00027     if (!inTargetModule_) {
00028       return;
00029     }
00030
00031     // If a specific module name is specified, check if it matches
00032     if (inTargetModule_) {
00033       spdlog::info("{}Found target module: {}", indent, targetCell_);
00034       // Print the module ports
00035       if (module.header->ports) {
00036         spdlog::info("{}Ports: {}", indent, module.header->ports->toString());
00037       }
00038     }
00039   }
00040
00041   // Continue processing child nodes
00042   depth_++;
00043   visitDefault(module);
00044   depth_--;
00045 }
00046
00047 // Handle port declarations
00048 void VerilogVisitor::handle(const slang::syntax::PortDeclarationSyntax &portDecl) {
00049   if (!inTargetModule_) {
00050     return;
00051   }
00052
00053   std::string indent = std::string(depth_ * 2, ' ');
00054
00055   // Get port direction
00056   std::string direction = "unknown";
00057   if (portDecl.header) {
00058     // Try to convert the header to different types of port headers
00059     if (auto *varPort = portDecl.header->as_if<slang::syntax::VariablePortHeaderSyntax>()) {
00060       if (varPort->direction) {
00061         direction = varPort->direction.valueText();
00062       }
00063     } else if (auto *netPort = portDecl.header->as_if<slang::syntax::NetPortHeaderSyntax>()) {
```

```
00064       if (netPort->direction) {
00065         direction = netPort->direction.valueText();
00066       }
00067     }
00068
00069     // Get port name
00070     for (const auto &declarator : portDecl.declarators) {
00071       if (declarator->name.valid()) {
00072         std::string_view portName = declarator->name.valueText();
00073         spdlog::info("{}Port Name: {} ({})", indent, portName, direction);
00074       }
00075     }
00076   }
00077 }
00078
00079 // Handle hierarchical instantiation (module instance)
00080 void VerilogVisitor::handle(const slang::syntax::HierarchyInstantiationSyntax &hierarchyInst) {
00081   if (!inTargetModule_) {
00082     return;
00083   }
00084
00085   std::string indent(depth_ * 2, ' ');
00086
00087   if (hierarchyInst.type.valid()) {
00088     std::string_view instType = hierarchyInst.type.valueText();
00089     spdlog::info("{}Hierarchy Instance Type: {}", indent, instType);
00090
00091     // Safely print parameter values (if any)
00092     try {
00093       if (hierarchyInst.parameters) {
00094         spdlog::info("{}Parameters:", indent);
00095         for (const auto &param : hierarchyInst.parameters->parameters) {
00096           if (!param)
00097             continue; // Null pointer check
00098
00099           if (auto *orderedParam = param->as_if<slang::syntax::OrderedParamAssignmentSyntax>()) {
00100             if (orderedParam->expr) {
00101               spdlog::info("{}  Parameter: {}", indent, orderedParam->expr->toString());
00102             }
00103           } else if (auto *namedParam = param->as_if<slang::syntax::NamedParamAssignmentSyntax>()) {
00104             if (namedParam->name.valid() && namedParam->expr) {
00105               spdlog::info("{}  Parameter {}: {}", indent, namedParam->name.valueText(),
00106                            namedParam->expr->toString());
00107             }
00108           }
00109         }
00110       }
00111     } catch (const std::exception &e) {
00112       spdlog::warn("{}Exception while processing parameters: {}", indent, e.what());
00113     }
00114
00115     // Safely handle instances
00116     try {
00117       for (const auto &instance : hierarchyInst.instances) {
00118         if (!instance) {
00119           spdlog::warn("{}Invalid instance", indent);
00120           continue;
00121         } else if (!instance->decl) {
00122           spdlog::warn("{}Invalid instance declaration", indent);
00123           // continue;
00124         } else {
00125           try {
00126             if (instance->decl->name.valid()) {
00127               std::string_view instName = instance->decl->name.valueText();
00128               spdlog::info("{}Instance Name: {}", indent, instName);
00129             }
00130           } catch (...) {
00131             spdlog::debug("{}Error printing name", indent);
00132           }
00133
00134           // Safely print dimensions
00135           try {
00136             if (!instance->decl->dimensions.empty()) {
00137               spdlog::info("{}  Dimensions: {}", indent, instance->decl->dimensions.toString());
00138             }
00139           } catch (...) {
00140             spdlog::debug("{}Error printing dimensions", indent);
00141           }
00142         }
00143
00144         // Safely print port connections
00145         spdlog::info("{}  Port connections:", indent);
00146         for (const auto &conn : instance->connections) {
00147           if (!conn) {
00148             spdlog::warn("{}    Null connection", indent);
00149             continue; // Null pointer check
00150           }
```

```
00151
00152           try {
00153             // Use as_if method for safe type conversion
00154             if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00155               if (ordered->expr) {
00156                 spdlog::info("{}    Ordered connection: {}", indent, ordered->expr->toString());
00157               } else {
00158                 spdlog::info("{}    Ordered connection: <empty>", indent);
00159               }
00160             } else if (auto *named = conn->as_if<slang::syntax::NamedPortConnectionSyntax>()) {
00161               if (named->name.valid()) {
00162                 std::string exprStr = named->expr ? named->expr->toString() : "<empty>";
00163                 spdlog::info("{}    .{}({})", indent, named->name.valueText(), exprStr);
00164               }
00165             } else if (conn->as_if<slang::syntax::EmptyPortConnectionSyntax>()) {
00166               spdlog::info("{}    <empty connection>", indent);
00167             } else if (conn->as_if<slang::syntax::WildcardPortConnectionSyntax>()) {
00168               spdlog::info("{}    .* (wildcard connection)", indent);
00169             } else {
00170               spdlog::info("{}    Unknown connection type: {}", indent,
00171                            slang::syntax::toString(conn->kind));
00172             }
00173           } catch (const std::exception &e) {
00174             spdlog::warn("{}Exception processing connection: {}", indent, e.what());
00175           } catch (...) {
00176             spdlog::warn("{}Unknown exception processing connection", indent);
00177           }
00178         }
00179       }
00180     } catch (const std::exception &e) {
00181       spdlog::warn("{}Exception while processing instances: {}", indent, e.what());
00182     }
00183
00184     // Continue processing child nodes
00185     depth_++;
00186     visitDefault(hierarchyInst);
00187     depth_--;
00188   }
00189 }
00190
00191 // Handle primitive gate instantiation
00192 void VerilogVisitor::handle(const slang::syntax::PrimitiveInstantiationSyntax &primitiveInst) {
00193   if (!inTargetModule_) {
00194     return;
00195   }
00196
00197   std::string indent(depth_ * 2, ' ');
00198
00199   // Get primitive gate type
00200   if (primitiveInst.type.valid()) {
00201     std::string_view gateType = primitiveInst.type.valueText();
00202     spdlog::info("{}Primitive Gate: {}", indent, gateType);
00203
00204     // Print delay information (if any)
00205     if (primitiveInst.delay) {
00206       spdlog::info("{}Delay: {}", indent, primitiveInst.delay->toString());
00207     }
00208
00209     // Print strength information (if any)
00210     if (primitiveInst.strength) {
00211       spdlog::info("{}Strength: {}", indent, primitiveInst.strength->toString());
00212     }
00213
00214     // Print each instance
00215     for (const auto &instance : primitiveInst.instances) {
00216       // Primitive gates usually don't have names, but if they do, print them
00217       if (instance->decl && instance->decl->name.valid()) {
00218         std::string_view instName = instance->decl->name.valueText();
00219         spdlog::info("{}  Gate instance: {}", indent, instName);
00220       }
00221
00222       // Print connections
00223       spdlog::info("{}  Gate connections:", indent);
00224       for (size_t i = 0; i < instance->connections.size(); ++i) {
00225         const auto &conn = instance->connections[i];
00226         // For gate-level instantiation, connections are usually ordered
00227         if (auto *ordered = conn->as_if<slang::syntax::OrderedPortConnectionSyntax>()) {
00228           if (ordered->expr) {
00229             // The first is usually the output, the rest are inputs
00230             std::string portType = (i == 0) ? "output" : "input";
00231             spdlog::info("{}    {} {}: {}", indent, portType, i, ordered->expr->toString());
00232           }
00233         }
00234       }
00235     }
00236
00237     // Continue to traverse deeper when specific standard gate type
```

```
00238     // Create a set of safe standard gate types
00239     static const std::unordered_set<std::string_view> safeGateTypes = {
00240         "and",    "or",     "nand",   "nor",    "xor",      "xnor",    "not",      "buf",
      "bufif0",
00241         "bufif1", "notif0", "notif1", "pullup", "pulldown", "cmos",    "rcmos",    "nmos",    "pmos",
00242         "rnmos",  "rpmos",  "tran",   "rtran",  "tranif0",  "tranif1", "rtranif0", "rtranif1"};
00243
00244     if (safeGateTypes.find(gateType) != safeGateTypes.end()) {
00245       // Continue processing child nodes
00246       depth_++;
00247       visitDefault(primitiveInst);
00248       depth_--;
00249     } else {
00250       spdlog::warn("{}Skipping deeper traversal of primitive: {}", indent, gateType);
00251     }
00252
00253   } else {
00254     spdlog::warn("{}Primitive instantiation missing gate type", indent);
00255   }
00256 }
00257
00258 // Handle specify block
00259 void VerilogVisitor::handle(const slang::syntax::SpecifyBlockSyntax &specifyBlock) {
00260   if (!inTargetModule_) {
00261     return;
00262   }
00263
00264   std::string indent(depth_ * 2, ' ');
00265   spdlog::info("{}Specify Block:", indent);
00266
00267   // Iterate through all path declarations
00268   for (const auto &item : specifyBlock.items) {
00269     if (auto *pathDecl = item->as_if<slang::syntax::PathDeclarationSyntax>()) {
00270       if (pathDecl->desc) {
00271         std::string pathSrc;
00272         std::string pathDst;
00273
00274         // Get path source
00275         if (!pathDecl->desc->inputs.empty()) {
00276           // Get the first input
00277           if (auto *identifier =
00278                   pathDecl->desc->inputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00279             if (identifier->identifier.valid()) {
00280               pathSrc = identifier->identifier.valueText();
00281             }
00282           }
00283         }
00284
00285         // Get path destination
00286         if (pathDecl->desc->suffix) {
00287           if (auto *simpleSuffix =
00288                   pathDecl->desc->suffix->as_if<slang::syntax::SimplePathSuffixSyntax>()) {
00289             if (!simpleSuffix->outputs.empty()) {
00290               if (auto *identifier =
00291                       simpleSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00292                 if (identifier->identifier.valid()) {
00293                   pathDst = identifier->identifier.valueText();
00294                 }
00295               }
00296             } else if (auto *edgeSuffix = pathDecl->desc->suffix
00297                                               ->as_if<slang::syntax::EdgeSensitivePathSuffixSyntax>())
      {
00298               if (!edgeSuffix->outputs.empty()) {
00299                 if (auto *identifier =
00300                         edgeSuffix->outputs[0]->as_if<slang::syntax::IdentifierNameSyntax>()) {
00301                   if (identifier->identifier.valid()) {
00302                     pathDst = identifier->identifier.valueText();
00303                   }
00304                 }
00305               }
00306             }
00307           }
00308
00309           // Construct path string
00310           std::string pathStr = pathSrc + " => " + pathDst;
00311
00312           // Get path delay
00313           std::string delayStr = "(";
00314           for (size_t i = 0; i < pathDecl->delays.size(); ++i) {
00315             if (i > 0)
00316               delayStr += ", ";
00317             delayStr += pathDecl->delays[i]->toString();
00318           }
00319           delayStr += ")";
00320
00321           spdlog::info("{}  Path: {} = {}", indent, pathStr, delayStr);
00322         }
```

```
00323       }
00324       // Can add handling for ConditionalPathDeclarationSyntax and IfNonePathDeclarationSyntax
00325       else if (auto *condPath = item->as_if<slang::syntax::ConditionalPathDeclarationSyntax>()) {
00326         if (condPath->path && condPath->predicate) {
00327           spdlog::info("{}  Conditional Path (if {})", indent, condPath->predicate->toString());
00328           // Recursively handle internal path
00329           handle(*condPath->path);
00330         }
00331       } else if (auto *ifNonePath = item->as_if<slang::syntax::IfNonePathDeclarationSyntax>()) {
00332         if (ifNonePath->path) {
00333           spdlog::info("{}  If-None Path", indent);
00334           // Recursively handle internal path
00335           handle(*ifNonePath->path);
00336         }
00337       }
00338     }
00339
00340     // Continue processing child nodes
00341     depth_++;
00342     visitDefault(specifyBlock);
00343     depth_--;
00344   }
00345 }
00346
00347 // Handle module declarations, only keep the target module
00348 void CellExtractor::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00349   if (module.header && module.header->name.valid()) {
00350     std::string_view moduleName = module.header->name.valueText();
00351
00352     // If it is the target module, mark as found, otherwise remove it
00353     if (!targetCell_.empty() && moduleName == targetCell_) {
00354       foundTarget_ = true;
00355       // Do not modify, keep this module
00356     } else {
00357       // Remove non-target modules
00358       remove(module);
00359     }
00360   }
00361 }
00362
00363 // Get result
00364 bool CellExtractor::foundTargetCell() const { return foundTarget_; }
00365
00366 // Handle module declarations
00367 void CellPrinter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00368   if (module.header && module.header->name.valid()) {
00369     std::string_view moduleName = module.header->name.valueText();
00370
00371     // Check if it is the target module
00372     if (!targetCell_.empty() && moduleName == targetCell_) {
00373       foundTarget_ = true;
00374
00375       // Print module definition
00376       out_ << "`timescale 1ns/10ps\n";
00377       out_ << module.toString();
00378
00379       return; // Do not traverse further
00380     }
00381   }
00382
00383   // Continue traversing other modules
00384   visitDefault(module);
00385 }
00386
00387 void ModuleRewriter::handle(const slang::syntax::SyntaxNode &node) {
00388   std::string indent(depth_ * 2, ' ');
00389   logger_->debug("{}Node: {}", indent, slang::syntax::toString(node.kind));
00390
00391   // Continue processing child nodes
00392   depth_++;
00393   visitDefault(node);
00394   depth_--;
00395 }
00396
00397 void ModuleRewriter::handle(const slang::syntax::ModuleDeclarationSyntax &module) {
00398   logger_->info("Processing module: {}", module.header->name.valueText());
00399   // auto &newNode = parse("\n" + this->cellName_ + " I" + this->moduleName_ + "( );");
00400   // logger_->debug("New node: {}", newNode.toString());
00401   for (int i = 0; i < instance_count_ - 1; i++) {
00402     auto &newNetNode = parse("\n  wire OP_" + std::to_string(i) + ";");
00403     insertAtBack(module.members, newNetNode);
00404   }
00405
00406   // TODO: Add instance to the module
00407
00408 }
00409
```

```
00410 void getAST(const std::string &verilog_file, const std::string &cell) {
00411   try {
00412     spdlog::info("Starting get AST from Verilog file: '{}'", verilog_file);
00413     auto result = slang::syntax::SyntaxTree::fromFile(verilog_file);
00414     if (result) {
00415       spdlog::info("Successfully parsed Verilog file.");
00416 
00417       try {
00418         // First, use the visitor to print basic information
00419         VerilogVisitor visitor(cell);
00420         visitor.visit(result.value()->root());
00421 
00422         // If a target cell is specified, use the rewriter to extract the relevant code
00423         if (!cell.empty()) {
00424           // Create a rewriter to only keep the target cell related code
00425           CellExtractor extractor(cell);
00426           auto extractedTree = extractor.transform(result.value());
00427 
00428           if (extractor.foundTargetCell()) {
00429             // Use SyntaxPrinter to print the extracted code
00430             std::string extractedCode = slang::syntax::SyntaxPrinter::printFile(*extractedTree);
00431 
00432             // Save to a file named after the cell name
00433             std::string outputFile = cell + ".v";
00434             std::ofstream cellOut(outputFile);
00435             if (cellOut) {
00436               cellOut << extractedCode;
00437               cellOut.close();
00438               spdlog::info("Extracted '{}' cell code to '{}'", cell, outputFile);
00439             } else {
00440               spdlog::error("Failed to write extracted cell code to '{}'", outputFile);
00441             }
00442           } else {
00443             spdlog::warn("Target cell '{}' not found in the Verilog file", cell);
00444           }
00445         }
00446 
00447         spdlog::info("Print full source code to 'full_source_code.v'");
00448         // Optionally save the entire syntax tree
00449         std::string fullOutput = slang::syntax::SyntaxPrinter::printFile(*result.value());
00450         std::ofstream out("full_source_code.v");
00451         out << fullOutput;
00452         out.close();
00453 
00454         spdlog::info("Print target cell code to 'cell_code.v'");
00455         // Use CellPrinter to print the target cell's code
00456         std::ofstream cellOut("cell_code.v");
00457         CellPrinter cellPrinter(cell, cellOut);
00458         cellPrinter.visit(result.value()->root());
00459         cellOut.close();
00460 
00461       } catch (const std::exception &e) {
00462         spdlog::error("Exception during AST traversal: {}", e.what());
00463       } catch (...) {
00464         spdlog::error("Unknown exception during AST traversal");
00465       }
00466     } else {
00467       spdlog::error("Error parsing Verilog file.");
00468     }
00469   } catch (const std::exception &e) {
00470     spdlog::error("Exception during Verilog parsing: {}", e.what());
00471   } catch (...) {
00472     spdlog::error("Unknown exception during Verilog parsing");
00473   }
00474 }
```

# Index